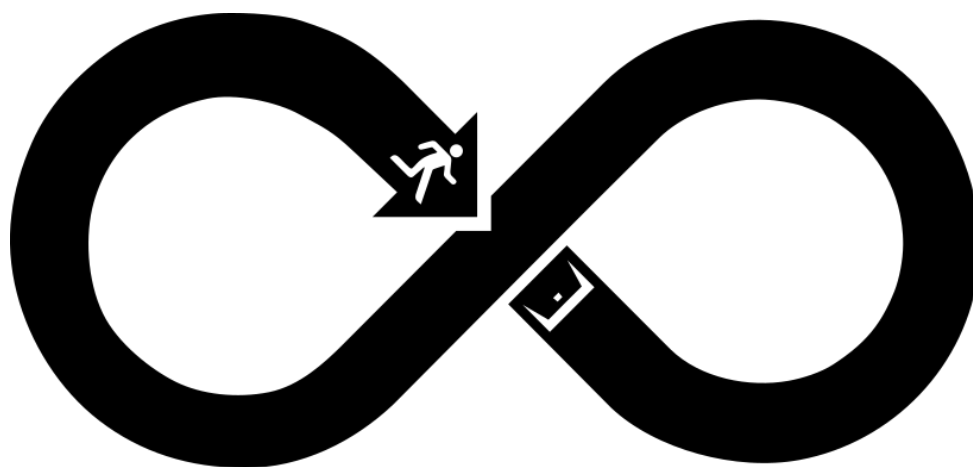


GRADO EN INGENIERÍA INFORMÁTICA DE GESTIÓN Y
SISTEMAS DE INFORMACIÓN

TRABAJO FIN DE GRADO

LOOP ESCAPE



Estudiante: Etxaniz Monge, Eneko

Director/Directora: Villamañe Girones, Mikel

Curso: 2024-2025

Fecha: Bilbao, 18 de junio de 2025

Resumen

A fin de experimentar un desarrollo que suponga un desafío en múltiples áreas, desde diseño gráfico a programación, se ha decidido desarrollar un videojuego, este se llama Loop Escape. Loop Escape es un videojuego desarrollado en Unity para dispositivos Windows donde se controla un personaje que derrota enemigos y avanza entre niveles. En este videojuego del género *roguelike* se han implementado múltiples enemigos y armas con diferentes comportamientos. Además, se ha realizado el diseño de este para tratar de hacerlo accesible a un amplio público. Actualmente, junio de 2025, el juego se puede descargar desde itch.io: <https://etxaniz100.itch.io/loopescape>.

Palabras clave: Unity, Videojuego, Roguelike, Accesibilidad

In order to experience a development that involves a challenge in multiple areas, from graphic design to coding, it has been decided to develop a video game, it is called Loop Escape. Loop Escape is a video game developed in Unity for Windows devices where a character is controlled that defeats enemies and advances between levels. In this video game of the roguelike genre, multiple enemies and weapons with different behaviours have been implemented. In addition, this game has been designed to be accessible to a wide public. Currently, June 2025, the game can be downloaded from itch.io: <https://etxaniz100.itch.io/loopescape>.

Keywords: Unity, Video game, Roguelike, Accessibility

-Testu hau automatikoki itzulita dago gaztelaniazko bertsiotik abiatuta.

Hainbat arlotan, diseinu grafikotik programaziora, erronka izango den garapena probatzeko asmoz, bideo-joko bat garatzea erabaki da. Loop Escape du izena bideo-jokoak. Loop Escape Windows gailuetarako Unityn garatutako bideo-joko bat da. Etsaiak garaitu eta mailen artean aurrera egiten duen pertsonaia bat kontrolatzen da. *Roguelike* generoko bideojoko honetan etsai eta arma anitz inplementatu dira, portaera ezberdinekin. Gainera, horren diseinua egin da, publiko zabalaren eskura jartzen saiatzeko. Gaur egun, 2025eko ekainean, jokoa itch.io bidez deskarga daiteke: <https://etxaniz100.itch.io/loopescape>.

Hitz gakoak: Unity, Bideo-jokoa, Roguelike, Irisgarritasuna

Índice general

1. Introducción	9
1.1. Objetivos de desarrollo sostenible y valores democráticos	10
2. Glosario	11
3. Planteamiento inicial	13
3.1. Objetivos	13
3.2. Alcance	14
3.2.1. Ciclo de vida	14
3.2.2. Estructura de descomposición del trabajo, EDT	15
3.2.3. Paquetes de trabajo	19
3.3. Planificación temporal	29
3.4. Herramientas	31
3.4.1. Herramientas software	31
3.4.2. Herramientas físicas	33
3.5. Gestión de riesgos	34
3.5.1. Riesgos informáticos	35
3.5.2. Riesgos físicos y de gestión	36
3.6. Evaluación económica	38
3.6.1. Rentabilidad económica	39
4. Antecedentes	41
5. Captura de requisitos	43
5.1. Jerarquía de actores	43
5.2. Modelo de casos de uso	43
5.3. Modelo de dominio	46
6. Análisis y diseño	47
6.1. Diagrama de clases	47
7. Desarrollo y evaluación	51
7.0. El entorno de desarrollo de Unity	51
7.1. Iteración 1	54
7.1.1. Movimiento del personaje	54
7.1.2. Movimiento de la cámara	58
7.1.3. Tileset	59
7.1.4. Enemigos	60
7.1.5. Armas	63
7.1.6. Combate	65
7.1.7. HUD	66
7.1.8. Controlador del juego	67

7.1.9.	Apartado artístico	67
7.1.10.	Pruebas	68
7.2.	Iteración 2	69
7.2.1.	Composición	69
7.2.2.	Armas	71
7.2.3.	Gestión de equipamiento	73
7.2.4.	Botín	73
7.2.5.	Enemigos	75
7.2.6.	Animaciones y apartado gráfico	77
7.2.7.	Pruebas	78
7.3.	Iteración 3	80
7.3.1.	HUD	80
7.3.2.	Menús	84
7.3.3.	Niveles y progreso	86
7.3.4.	Altares	87
7.3.5.	Sonidos	88
7.3.6.	Pruebas	89
7.4.	Iteración 4	92
7.4.1.	Calamar Vampiro	92
7.4.2.	Mandoble Volcánico	95
7.4.3.	Interfaz	97
7.4.4.	Nuevos niveles	97
7.4.5.	Pruebas	98
7.5.	Iteración 5	99
7.5.1.	Sistema de guardado de partidas	99
7.5.2.	Selección de personaje	100
7.5.3.	Progresión	101
7.5.4.	Ajustes y accesibilidad	102
7.5.5.	Pruebas	104
8.	Conclusiones	105
8.1.	Objetivos	105
8.2.	Planificación	107
8.3.	Desafíos surgidos	109
8.4.	Trabajo futuro	110
9.	Bibliografía	113
10.	Anexo	115
10.1.	Diagramas de secuencia	115
10.2.	Diagrama de clases	125

Índice de figuras

3.1. Paquetes principales del EDT	15
3.2. EDT: Organización e Iteración 1	16
3.3. EDT: Iteración 2 e Iteración 3	17
3.4. EDT: Iteración 4 e Iteración 5	18
3.5. EDT: Iteración 6 y Documentación y planificación	19
3.6. Diagrama de Gantt	31
5.1. Jerarquía de actores	43
5.2. Casos de uso	44
5.3. Subcasos de uso de “Cargar Nivel”	45
5.4. Modelo de dominio	46
6.1. Diagrama de clases, parte del personaje	48
6.2. Diagrama de clases, clases generales	49
6.3. Diagrama de clases, derrota de enemigo	49
6.4. Diagrama de clases, controlador	50
6.5. Diagrama de clases	50
7.1. Ciclo de ejecución simplificado	52
7.2. Demostración Coyote Time	56
7.3. Demostración detección de suelo	57
7.4. Demostración <i>tileset</i>	59
7.5. Escenario creado utilizando dos <i>tilesets</i>	60
7.6. Máquina de estados para un enemigo	61
7.7. Sensores de un enemigo	62
7.8. Posibles direcciones de ataques	64
7.9. Captura de pantalla del juego	67
7.10. Evolución de los <i>sprites</i>	68
7.11. Diferentes objetos utilizando diferentes componentes	70
7.12. Diferentes proyectiles utilizando diferentes modificadores	71
7.13. Rango de recogida de botín	74
7.14. Tipos de enemigos	75
7.15. Visión del esqueleto y <i>slime</i>	76
7.16. <i>Collider</i> de los ataques de enemigos	77
7.17. <i>Spritesheet</i> del personaje corriendo	77
7.18. HUD en Pac-Man, Super Mario Bros y Minecraft	80
7.19. Tres capas mostradas en perspectiva	81
7.20. Esquina izquierda del HUD mostrando información sobre la vida y armas	82
7.21. Rotación del contador de enemigos	83
7.22. Menú principal	84
7.23. Menú de pausa	85

7.24. Tipos de altares	87
7.25. Calamar Vampiro : Jefe y animal	93
7.26. Ataques del calamar vampiro	94
7.27. Comportamiento del mandoble volcánico	95
7.28. Detección de paredes o techos	96
7.29. Rastro de la espada	97
7.30. Personajes seleccionables	100
7.31. Menú de selección de personaje	101
7.32. Interfaz de opciones	102
8.1. Gráfico de Likert sobre la encuesta final	106
8.2. Estimación horaria contra realidad	108
8.3. Distribución horaria	109
8.4. Diseños sin implementar	111
10.1. Diagrama de secuencia: Continuar partida	115
10.2. Diagrama de secuencia: Iniciar nueva partida	116
10.3. Diagrama de secuencia: Ajustes	117
10.4. Diagrama de secuencia: Cargar nivel	118
10.5. Diagrama de secuencia: Inicializar personaje	119
10.6. Diagrama de secuencia: Inicializar interfaz	120
10.7. Diagrama de secuencia: Pausar	120
10.8. Diagrama de secuencia: Gestionar equipamiento	121
10.9. Diagrama de secuencia: Inicializar enemigo	122
10.10. Diagrama de secuencia: Derrotar enemigo	123
10.11. Diagrama de secuencia: Moverse	124
10.12. Diagrama de clases	125

Índice de tablas

3.1. Estimaciones horarias del proyecto	30
3.2. Retraso en el proyecto dependiendo del riesgo	34
3.3. Probabilidad de aparición dependiendo del riesgo	34
3.4. Coste relativo de los materiales	38
3.5. Costes totales estimados del proyecto	39

1. Introducción

Los videojuegos son un tipo de *software* de la industria del entretenimiento que mueve a billones de personas a lo largo del mundo. Estableciéndose un sector propio, en los últimos años los videojuegos se han convertido en uno de los pasatiempos más compartidos por personas de todas las edades.

Con el avance de los años y la evolución de esta industria, la barrera de entrada a la creación de este tipo de contenido es más accesible que nunca en la historia. Existiendo miles de herramientas y recursos gratuitos para llevar a cabo el desarrollo, la creación de videojuegos ha dejado de ser un sueño para muchos, para convertirse en una realidad y poder implementar un producto que otros puedan disfrutar.

Siendo tal la importancia del sector hoy en día, experimentar de primera mano un desarrollo de este tipo es una gran oportunidad. El videojuego a desarrollar en este TFG, *Loop Escape*, plantea una experiencia de partidas individuales, donde en cada partida el progreso es parcialmente reiniciado, a este género de juegos se le conoce como *roguelike*.

Los videojuegos pueden atraer personas a la informática, siendo el mío uno de los casos. Empecé a aprender por mi cuenta mediante programación por bloques y más adelante me adentre al código con diferentes lenguajes y herramientas, aprovechando ese conocimiento para hacer juegos simples.

Posteriormente, en la propia universidad se proponen proyectos relacionados con la creación de videojuegos. En las asignaturas de Programación Modular Orientada a Objetos e Ingeniería del Software se realizan desarrollos de esta temática, originando en el primero de ellos el nombre *Loop Escape*.

Fuera de la universidad, eventos como las *Game Jam*, eventos de corta duración donde se propone un tema para crear un videojuego desde cero, impulsaron más mis ganas por este tipo de desarrollo. Participar en varias, siendo una la *Euskal Game Jam*, creada en colaboración con la UPV/EHU, resultó en buenas experiencias tanto educativas como recreativas, causando que finalmente me decida por crear un videojuego como trabajo de fin de grado.

Loop Escape es un videojuego de un jugador para PC, *Personal Computer*, donde el jugador tratará de avanzar por diferentes salas generadas proceduralmente, moviéndose por un espacio de dos dimensiones mientras se enfrenta a enemigos que impiden el paso y salta entre plataformas que ofrecen apoyo.

1.1. Objetivos de desarrollo sostenible y valores democráticos

Siguiendo las guías de la agenda 2030, es posible crear un proyecto de forma sostenible a fin de colaborar en un mejor futuro. En este proyecto se encuentran presentes dos objetivos de los 17+1¹ concretados, ODS 9: “Industria, innovación e infraestructura” y ODS 10: “Reducción de las desigualdades”.

El ODS 9 está presente en este TFG en la parte de innovación e industria porque los videojuegos son una industria que está en constante innovación, haciendo uso de las últimas tecnologías para aprovecharlas en el entretenimiento. La aportación de este TFG al ODS 10 será en lo relacionado con la accesibilidad. Se usará la guía de *Game Accessibility Guidelines*² para tratar de hacer el juego accesible al mayor público posible. Se seguirán las recomendaciones de la guía y de este modo, se intentará reducir la desigualdad facilitando disfrutar de este juego a personas con necesidades especiales.

Los valores democráticos de la universidad establecen una serie de normas e ideales las cuales aseguran un ambiente de respeto y convivencia entre estudiantes, profesorado y otras entidades que se puedan encontrar en el ámbito universitario. Este proyecto está mayormente relacionado con el tercer principio general de las normas básicas de convivencia. Entre los principios generales [1], se establece: “Las personas que integran la comunidad universitaria procurarán participar activamente en la vida universitaria, para fomentar el compromiso social y el desarrollo personal”.

Los videojuegos proponen un espacio donde poder socializar y encontrar gustos comunes, pudiendo considerarse como parte de la vida universitaria de muchas personas. En varios eventos de la universidad o relacionados con esta se han propuesto actividades relacionadas con los videojuegos o la creación de estos, como la Euskal Game Jam³, un evento donde se trata de crear un videojuego en unos pocos días.

¹<https://www.ehu.eus/es/web/iraunkortasuna/ehuagenda-2030>

²<https://gameaccessibilityguidelines.com/full-list/>

³https://www.ehu.eus/es/home/-/asset_publisher/1MM8Xlt2aJEZ/content/euskal-game-jam-1/1600093

2. Glosario

A lo largo de la documentación se usará terminología específica al desarrollo de videojuegos, ya sea con el uso de siglas, anglicismos o términos con un significado diferente al habitual.

- **Botín/Loot:** Recompensas que el personaje puede obtener, estas varían de tipo en función del juego.
- **Entidad:** Elemento dentro del grupo formado por el personaje, enemigos y jefes.
- **Fotograma/Frame:** Un fotograma es un instante del juego. En un fotograma se lleva a cabo un ciclo de ejecución y se renderiza la imagen que se muestra en pantalla.
- **FPS:** Fotogramas Por Segundo, cuántos fotogramas se renderizan en un segundo.
- **Gráfico rasterizado:** Imagen formada por una cuadrícula de píxeles.
- **HUD:** Head-Up Display, información que se muestra al jugador sobre su estado durante la partida en forma de gráficos visuales.
- **Jefe:** Término utilizado para referirse a desafíos más difíciles de lo normal. Comúnmente se encuentran como finales a secciones de juegos en forma de un enemigo especial más fuerte de lo normal.
- **Jugador/a:** Persona que está jugando al juego. Aunque en el mundo de los videojuegos este término también suele usarse para referirse al carácter/muñeco que la persona controla, en esta documentación y para evitar ambigüedades, en este caso se usará el término personaje.
- **Nivel:** Conjunto de salas por las que hay que avanzar, normalmente de forma lineal.
- **Personaje:** Personaje que está controlando el jugador.
- **Pixel art:** Estilo artístico para la creación de gráficos rasterizados, normalmente dibujando a nivel de pixel.
- **Retroalimentación/Feedback:** Detalles audiovisuales que informan al jugador de las consecuencias de sus acciones, por ejemplo, vibración de la pantalla como consecuencia de atacar.
- **Sala:** Formada por un escenario donde se coloca al personaje, normalmente con una salida y un objetivo que cumplir.
- **Sprite:** Imágenes bidimensionales utilizadas en videojuegos para representar elementos.

3. Planteamiento inicial

Antes de comenzar un proyecto, es una buena práctica planificar inicialmente qué se va a hacer en este. Tanto definir el resultado buscado, como definir las herramientas que se utilizarán en el proceso son partes esenciales de un buen desarrollo.

3.1. Objetivos

El principal objetivo de este proyecto consiste en desarrollar e implementar un videojuego ejecutable en sistemas Windows. Teniendo como género principal el sistema *roguelike*, se jugará en 2D, con gran énfasis en la individualidad de cada partida haciendo uso de algoritmos aleatorios para generar las salas y llevar a cabo eventos del juego.

Apoyando al objetivo principal, se busca que la experiencia de juego sea fluida, para que quien lo juegue sienta que sus acciones se ven reflejadas correctamente en la partida. Se hará uso de efectos de sonido y visuales como apoyo a la retroalimentación de la experiencia.

Como objetivo secundario, se quiere obtener una gran variedad de elementos en el software, desde enemigos hasta armas. Los diferentes tipos de enemigos se diferenciarán por sus aspectos y comportamientos. Esto mismo se aplica a los jefes, con la diferencia de que sus comportamientos serán más avanzados a fin de suponer un verdadero obstáculo al jugador. Además, se podrán encontrar diferentes tipos de escenarios con la intención de ofrecer una mayor variedad visual.

En caso de haber cumplido todos los objetivos previos, se establecen otros posibles objetivos:

Como cada persona tiene una forma diferente de jugar, se implementarán ajustes de accesibilidad, para facilitar la experiencia a un mayor público. Por ejemplo, añadiendo un selector de idioma.

A fin de aumentar la variedad en el flujo de juego, se tiene como objetivo opcional el añadido de diferentes tipos de salas. Estas podrían ser simplemente salas con un cofre donde obtener algo de botín, o tiendas donde comprar armas a cambio de algún tipo de divisa.

3.2. Alcance

Para un correcto desarrollo de un proyecto, definir el alcance es una parte esencial, dando a conocer costes tanto temporales como económicos a la vez que sirve de mapa para guiar el progreso hacia un satisfactorio cierre.

3.2.1. Ciclo de vida

Debido a la forma en la que se estructuran los proyectos de desarrollo de videojuegos, se ha decidido establecer un ciclo de vida incremental. De esta forma, se irán obteniendo versiones jugables del juego, cada una con más contenido y funcionalidades que la anterior. Se referirá a cada incremento como iteración, y cada una de las cuales constará de diferentes mecánicas a implementar que harán avanzar el desarrollo.

En la **primera iteración**, se obtendrá una versión simplificada del juego. Esta constará de una única sala, con las diferentes plataformas y obstáculos colocados a mano. El personaje poseerá la capacidad de moverse, desplazarse horizontalmente y saltar, además de poder atacar enemigos con un arma. Debe haber al menos un tipo de enemigo que ataque al personaje y que pueda recibir ataques de este. Se considera que la primera iteración está finalizada una vez sea posible iniciar el juego, derrotar a todos los enemigos, existiendo la posibilidad de que el personaje sea derrotado, y acceder a la salida de la sala, haciendo aparecer un mensaje que informa al jugador de su victoria. Además, los elementos contarán con *sprites* para ser reconocibles.

En la **segunda iteración** el centro de atención pasa a la variedad de juego, siendo su objetivo la posibilidad de obtener, equipar y utilizar diferentes tipos de armas, con sus respectivos accesorios. Además, deberá haber más de un tipo de enemigo contra el que enfrentarse, los cuales, al ser derrotados, existirá la posibilidad de que dejen equipamiento para el personaje en forma de botín o *loot*. En esta iteración los elementos deberán tener animaciones.

Durante la **tercera iteración**, el enfoque estará en los menús y el avance del juego. Se desarrollará el sistema que permitirá avanzar entre salas y se crearán varias salas para recorrer. Se diseñarán e implementarán los menús de inicio, derrota y pausa. También se implementará el HUD, *Head-Up Display*, el cual mostrará datos como la salud del personaje. Para mejorar la inmersión, también se añadirán sonidos.

La **cuarta iteración** se usará para estudiar e implementar la generación procedural de salas.

La **quinta iteración** se centrará en los jefes. Se desarrollarán sus salas, comportamientos y botón especial.

La **sexta y última iteración** tratará de añadir el sistema de guardado de partidas junto con un sistema de progresión y opciones de accesibilidad, como la selección de idioma.

3.2.2. Estructura de descomposición del trabajo, EDT

Para evitar problemas a futuro, es importante planificar con anterioridad las diferentes tareas que se llevarán a cabo. Gracias a la distribución por iteraciones del proyecto, la asignación de tareas se puede hacer de una forma modular simplificando la gestión. Para ello el EDT presenta las tareas de forma gráfica haciendo sencilla su organización.

En total, las tareas se han agrupado en 8 secciones: organización, iteraciones del 1 al 6 y documentación y presentación. La figura 3.1 muestra el primer nivel de descomposición y las siguientes expanden los detalles de cada una de las tareas.

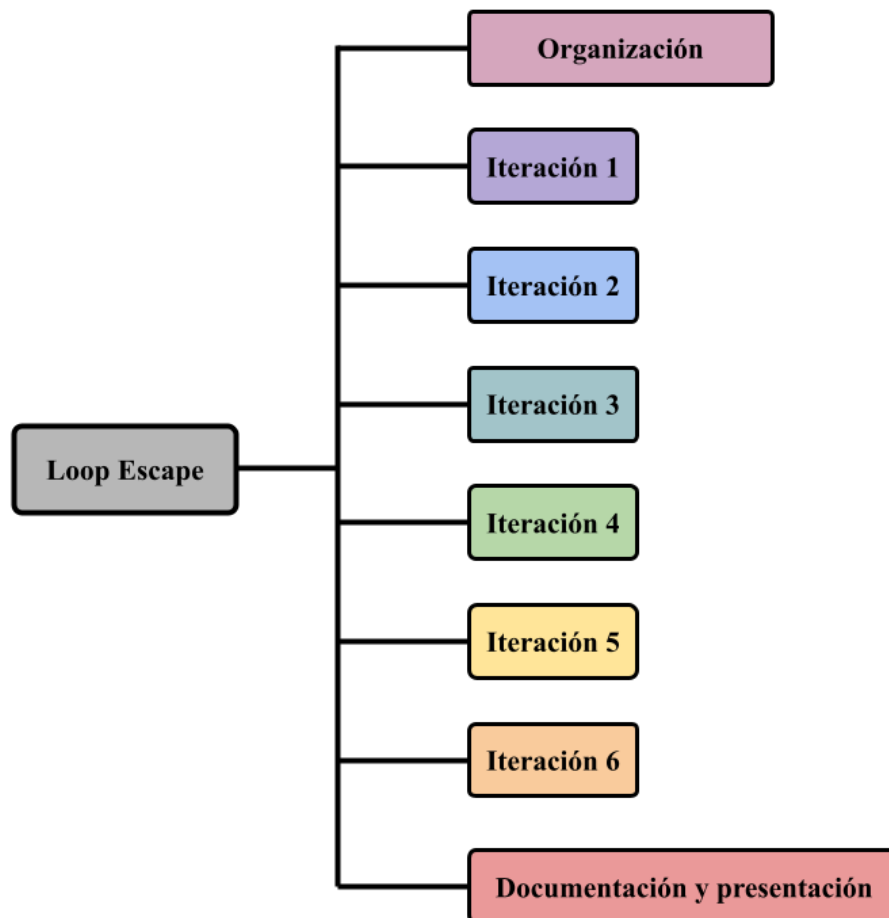


Figura 3.1: Paquetes principales del EDT

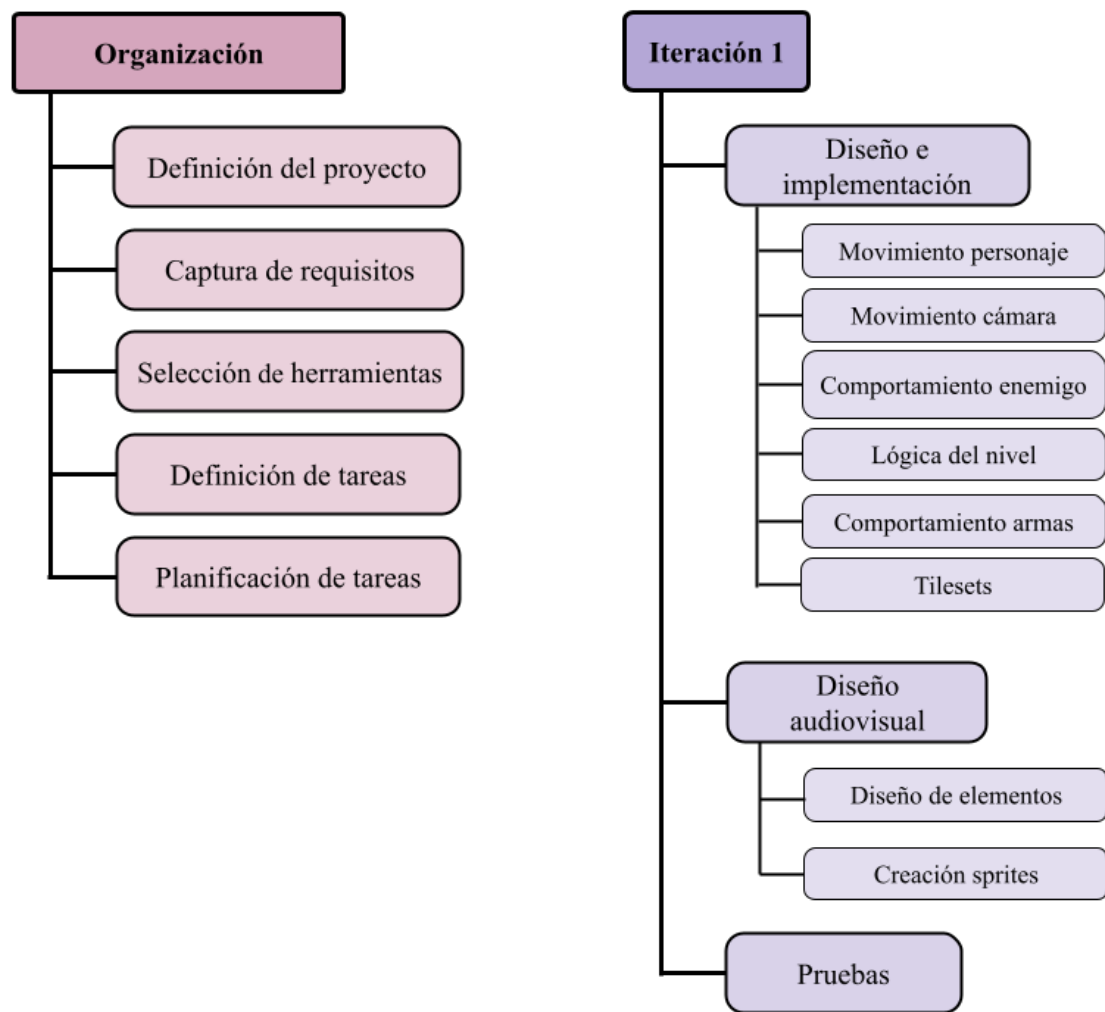


Figura 3.2: EDT: Organización e Iteración 1

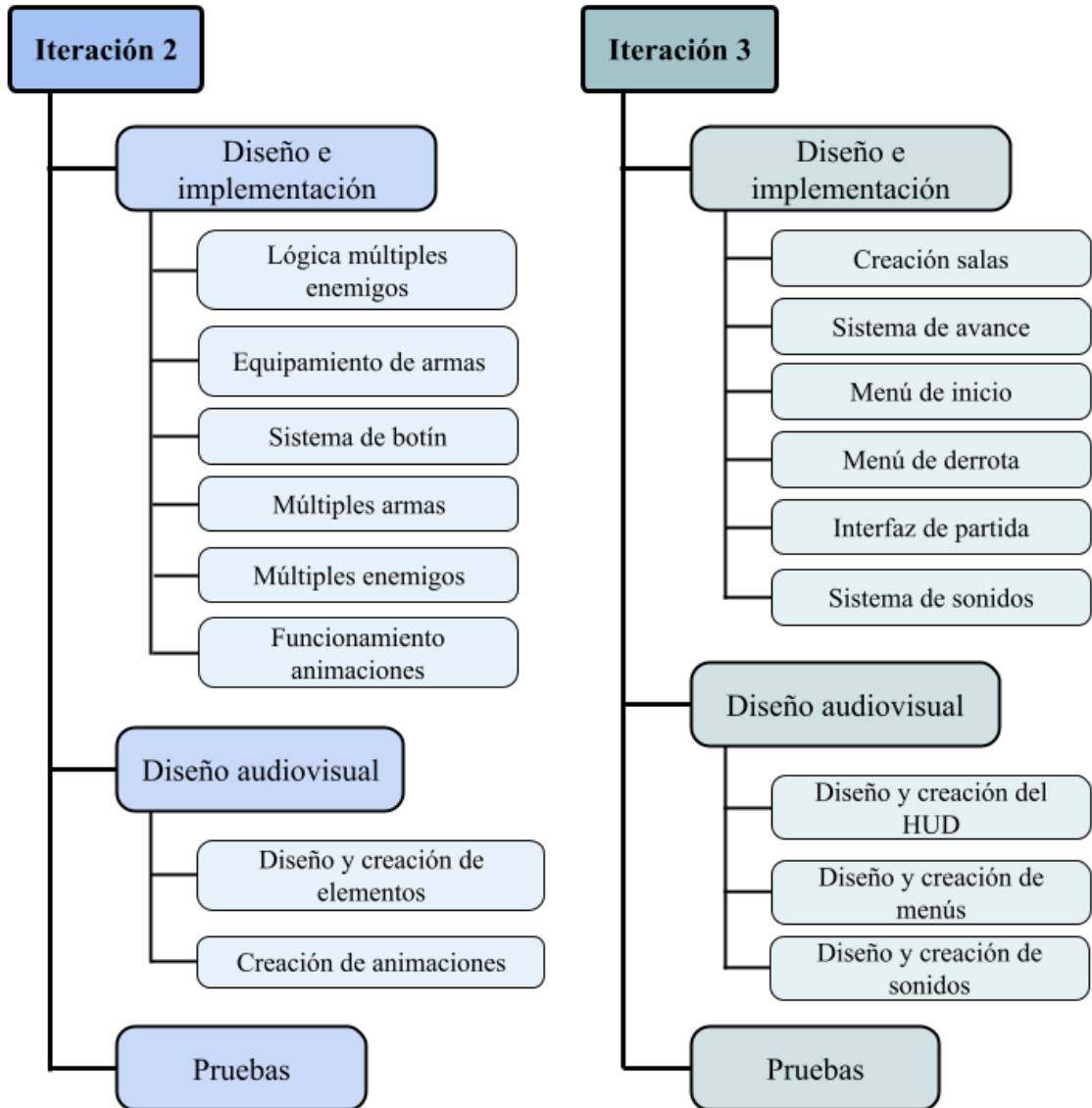


Figura 3.3: EDT: Iteración 2 e Iteración 3

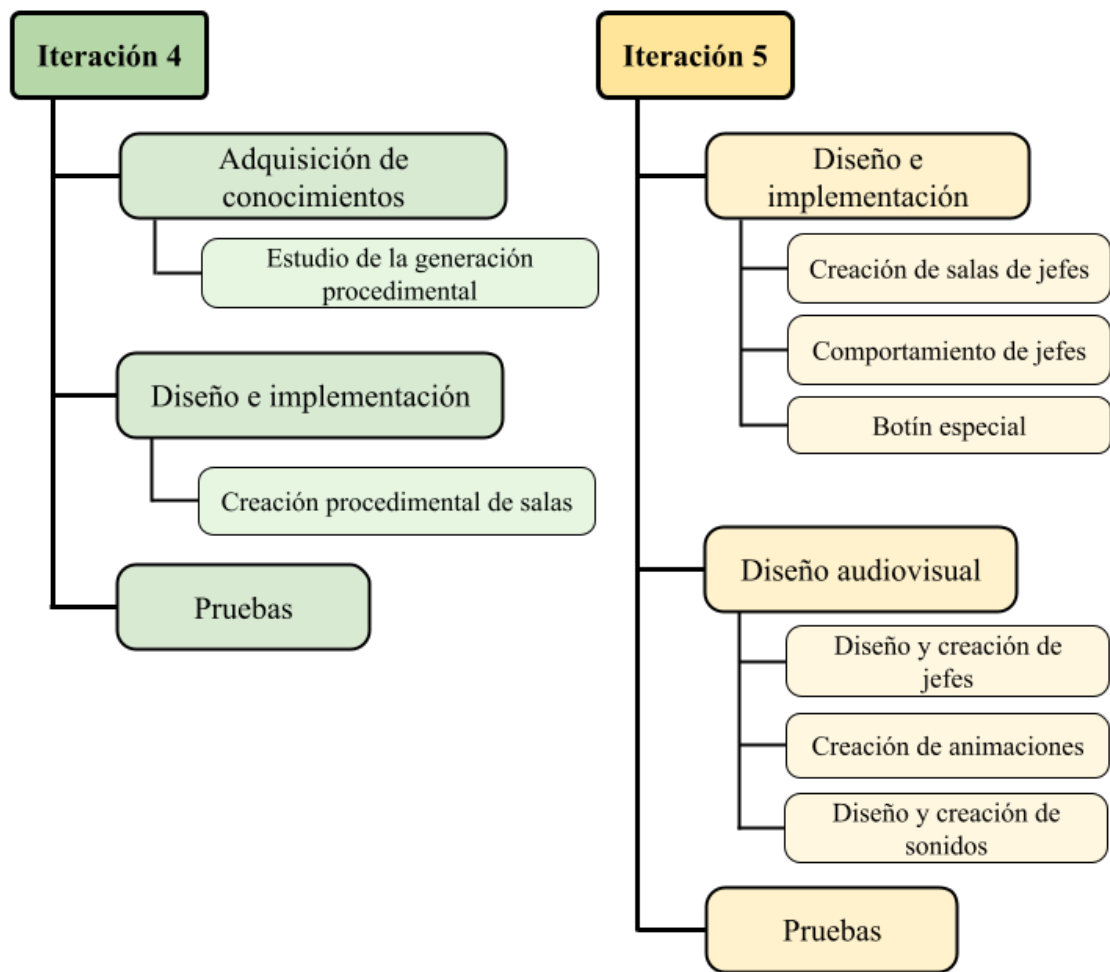


Figura 3.4: EDT: Iteración 4 e Iteración 5

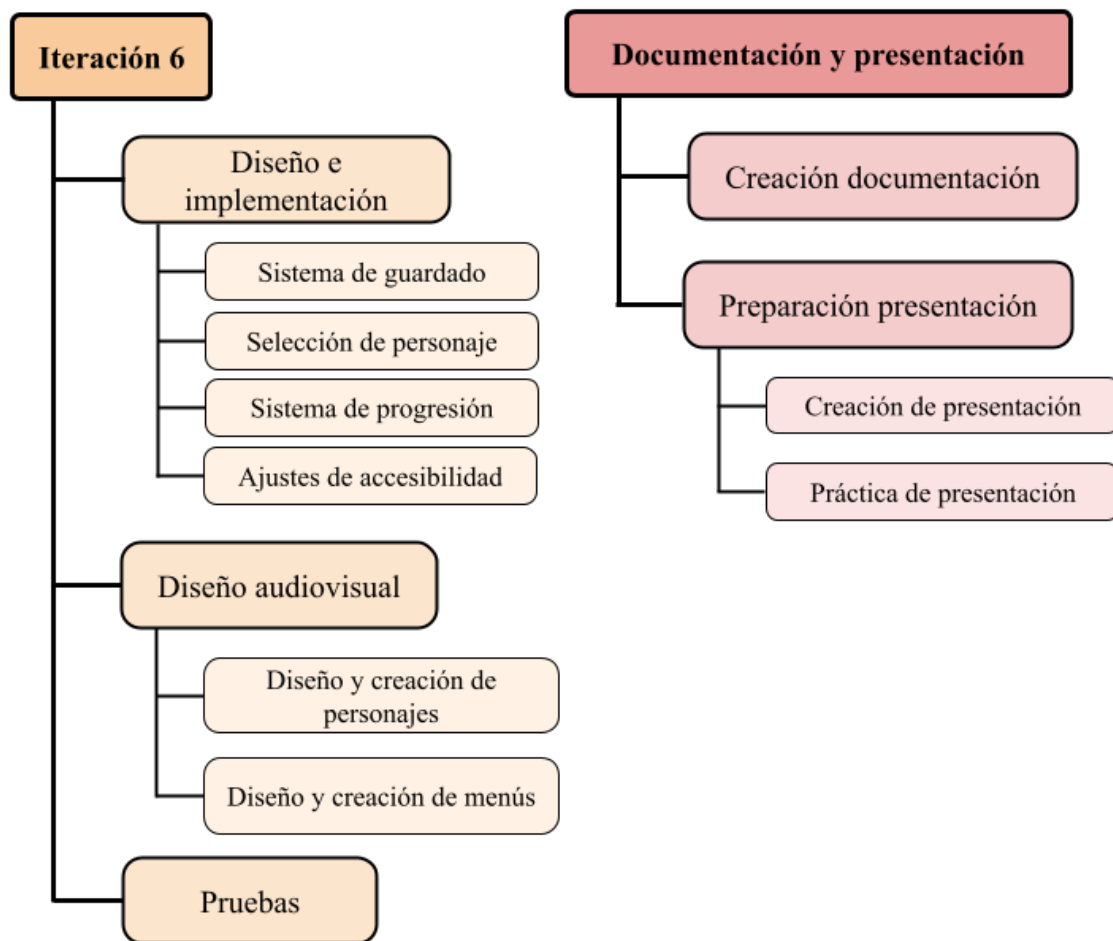


Figura 3.5: EDT: Iteración 6 y Documentación y planificación

3.2.3. Paquetes de trabajo

Cada una de las tareas del EDT consiste en un paquete de trabajo. Estos contienen información sobre la duración, descripción y requerimientos, guiando y dando una estimación del esfuerzo que supone cada tarea.

3.2.3.1. Organización

Paquete de trabajo: 0.1 Definición del proyecto.
Duración: 4 horas
Descripción: Definición de cómo será el juego y qué elementos tendría.
Salidas/Entregables: Documento con descripción general del juego.

Paquete de trabajo: 0.2 Captura de requisitos.
Duración: 6 horas
Descripción: Realizar un análisis donde queden registradas las necesidades para el correcto funcionamiento del juego planteado.
Salidas/Entregables: Requisitos funcionales y no funcionales.
Recursos necesarios: Documento con descripción general del juego.

Paquete de trabajo: 0.3 Selección de herramientas.
Duración: 1 hora
Descripción: Búsqueda de herramientas populares en el ámbito del desarrollo de videojuegos y selección de aquellas que se usarán para llevar a cabo el proyecto.
Recursos necesarios: Documento con descripción general del juego.

Paquete de trabajo: 0.4 Definición de tareas.
Duración: 6 horas
Descripción: Estructuración en forma de tareas de todas aquellas acciones que se deban tomar para realizar el proyecto, al igual que una estimación horaria de cada una.
Salidas/Entregables: EDT y paquetes de trabajo.
Recursos necesarios: Requisitos funcionales y no funcionales.

Paquete de trabajo: 0.5 Planificación temporal.
Duración: 3 horas
Descripción: Distribución temporal de las tareas a lo largo del tiempo de desarrollo.
Salidas/Entregables: Diagrama de Gantt.
Recursos necesarios: EDT y paquetes de trabajo.

3.2.3.2. Iteración 1

1.1 Diseño e implementación

La salida del grupo de tareas “Diseño e implementación” de todas las iteraciones es una versión ejecutable del juego.

Paquete de trabajo: 1.1.1 Movimiento del personaje.
Duración: 7.5 horas
Descripción: Se plantea e implementa cómo se moverá el personaje por el entorno del juego.

Paquete de trabajo: 1.1.2 Movimiento cámara.
Duración: 2.5 horas
Descripción: Se crea el algoritmo que sigue la cámara a la hora de seguir al personaje.

Paquete de trabajo: 1.1.3 Comportamiento enemigo.
Duración: 5.5 horas
Descripción: Se plantea e implementa un enemigo básico que atacará al personaje y podrá recibir ataques de este.

Paquete de trabajo: 1.1.4 Comportamiento armas.
Duración: 6 horas
Descripción: Desarrollo e implementación del sistema que permite al personaje derrotar enemigos.

Paquete de trabajo: 1.1.5 Lógica del nivel.
Duración: 3.5 horas
Descripción: Implementación del funcionamiento interno del juego, para saber cuando el nivel ha sido superado.

Paquete de trabajo: 1.1.6 Tilesets.
Duración: 1 hora
Descripción: Utilizando un módulo de Unity, se implementa un sistema para poder crear escenarios con facilidad.

1.2 Diseño gráfico

Paquete de trabajo: 1.2.1 Diseño de elementos.
Duración: 2 horas
Descripción: Se diseña el personaje, enemigos y <i>tilesets</i> .
Salidas/Entregables: Bocetos del personaje, enemigos y <i>tilesets</i> .

Paquete de trabajo: 1.2.2 Creación de <i>sprites</i> .
Duración: 8 horas
Descripción: Se dibujan los <i>sprites</i> diseñados empleando técnicas de <i>pixelart</i> .
Salidas/Entregables: <i>Sprites</i> del personaje, enemigos y <i>tilesets</i> .
Recursos necesarios: Bocetos del personaje, enemigos y <i>tilesets</i> .

1.3 Pruebas

Paquete de trabajo: 1.3 Pruebas.
Duración: 5 horas
Descripción: Se prueban las diferentes mecánicas del juego y se crea una encuesta para obtener opiniones de personas ajenas al proyecto. En caso de haber errores, se corrigen.
Salidas/Entregables: Informe de pruebas.
Recursos necesarios: Versión ejecutable de la iteración 1.

3.2.3.3. Iteración 2

2.1 Diseño e implementación

Paquete de trabajo: 2.1.1 Lógica múltiples enemigos.
Duración: 3 horas
Descripción: Modificación de la lógica de los enemigos para permitir y facilitar diferentes tipos de estos.

Paquete de trabajo: 2.1.2 Equipamiento de armas.
Duración: 10 horas
Descripción: Desarrollar un sistema para que el personaje pueda recoger armas, y alternar entre las dos que puede poseer. Además, se añaden los accesorios de las armas.

Paquete de trabajo: 2.1.3 Sistema de botón.
Duración: 2 horas
Descripción: Implementar el sistema a través del cual cuando los enemigos son derrotados hacen aparecer armas para el personaje.

Paquete de trabajo: 2.1.4 Múltiples armas.
Duración: 6 horas
Descripción: Se crean tres armas que el personaje pueda usar.

Paquete de trabajo: 2.1.5 Múltiples enemigos.
Duración: 8 horas
Descripción: Se diseñan e implementan tres enemigos.

Paquete de trabajo: 2.1.6 Funcionamiento animaciones.
Duración: 3 horas
Descripción: Aprovechando el sistema de animaciones de Unity, se añaden animaciones a diferentes acciones de los enemigos, armas y personaje.

2.2 Diseño audiovisual

Paquete de trabajo: 2.2.1 Diseño y creación de elementos.
Duración: 8 horas
Descripción: Se diseñan los nuevos enemigos y se crean sus <i>sprites</i> básicos. También se crean los diseños de las armas.
Salidas/Entregables: <i>Sprites</i> de enemigos y armas.

Paquete de trabajo: 2.2.2 Diseño y creación de animaciones.
Duración: 15 horas
Descripción: Se dibujan las animaciones para el personaje, enemigos y ataques de las armas.
Salidas/Entregables: Animaciones de enemigos, personaje y ataques.
Recursos necesarios: <i>Sprites</i> de enemigos y armas.

2.3 Pruebas

Paquete de trabajo: 2.3 Pruebas.
Duración: 6 horas
Descripción: Se realizan pruebas para asegurar el funcionamiento en conjunto de las nuevas implementaciones. En caso de haber errores, se corrigen.
Salidas/Entregables: Informe de pruebas.
Recursos necesarios: Versión ejecutable de la iteración 2.

3.2.3.4. Iteración 3

3.1 Diseño e implementación

Paquete de trabajo: 3.1.1 Creación de salas.
Duración: 3 horas
Descripción: Utilizando los <i>tilesets</i> previamente implementados, se crean varias salas.

Paquete de trabajo: 3.1.2 Sistema de avance.
Duración: 2 horas
Descripción: Se implementa el sistema que permite avanzar entre salas.

Paquete de trabajo: 3.1.3 Menú de inicio.
Duración: 6 horas
Descripción: Se diseña y crea el menú que aparece al abrir el juego.

Paquete de trabajo: 3.1.4 Menú de derrota.
Duración: 4 horas
Descripción: Se diseña y crea el menú que aparece al ser derrotado.

Paquete de trabajo: 3.1.5 Interfaz de partida.
Duración: 5 horas
Descripción: Se diseña y crea el HUD.

Paquete de trabajo: 3.1.6 Sistema de sonidos.
Duración: 6 horas
Descripción: Diseñar e implementar la forma de reproducir sonidos acorde a las acciones del juego.

3.2 Diseño audiovisual

Paquete de trabajo: 3.2.1 Diseño y creación del HUD.
Duración: 3 horas
Descripción: Diseño y creación de los diferentes elementos a mostrar en el HUD.
Salidas/Entregables: <i>Sprites</i> del HUD.

Paquete de trabajo: 3.2.2 Diseño y creación de menús.
Duración: 6 horas
Descripción: Diseño y creación de los diferentes elementos a mostrar en los menús, como fondos, opciones.
Salidas/Entregables: Imágenes del menú.

Paquete de trabajo: 3.2.3 Diseño y creación de sonidos.
Duración: 4 horas
Descripción: Se diseñan y producen los sonidos para cada acción del juego.
Salidas/Entregables: Archivos de audio con los sonidos.

3.3 Pruebas

Paquete de trabajo: 3.3 Pruebas.
Duración: 7 horas
Descripción: Se prueban las diferentes mecánicas del juego en conjunto y se crea una encuesta para obtener opiniones de personas ajenas al proyecto. En caso de haber errores, se corrigen.
Salidas/Entregables: Informe de pruebas.
Recursos necesarios: Versión ejecutable de la iteración 3.

3.2.3.5. Iteración 4

4.1 Adquisición de conocimientos

Paquete de trabajo: 4.1 Estudio de la generación procedimental.
Duración: 10 horas
Descripción: Se estudia el funcionamiento y técnicas comúnmente usadas en la generación procedimental de escenarios..

4.2 Diseño e implementación

Paquete de trabajo: 4.2 Creación procedimental de salas.
Duración: 20 horas
Descripción: Se implementa un algoritmo de generación procedimental de salas.

4.3 Pruebas

Paquete de trabajo: 4.3 Pruebas.
Duración: 7 horas
Descripción: Se llevan a cabo pruebas para asegurar la creación de niveles correctos. En caso de haber errores, se corrigen.
Salidas/Entregables: Informe de pruebas.
Recursos necesarios: Versión ejecutable de la iteración 4.

3.2.3.6. Iteración 5

5.1 Diseño e implementación

Paquete de trabajo: 5.1.1 Creación de salas de jefes.
Duración: 3 horas
Descripción: Se crean dos salas especiales donde combatir a los jefes.

Paquete de trabajo: 5.1.2 Comportamiento de jefes.
Duración: 25 horas
Descripción: Se diseñan e implementan dos combates contra jefes.

Paquete de trabajo: 5.1.3 Botín especial.
Duración: 4 horas
Descripción: Se añaden armas especiales para cada jefe.

5.2 Diseño audiovisual

Paquete de trabajo: 5.2.1 Diseño y creación de jefes.
Duración: 5 horas
Descripción: Se diseña el aspecto de los jefes.
Salidas/Entregables: <i>Sprites</i> de referencia para los jefes.

Paquete de trabajo: 5.2.2 Creación de animaciones.
Duración: 10 horas
Descripción: Se crean las animaciones de los jefes.
Salidas/Entregables: Animaciones de los jefes.
Recursos necesarios: <i>Sprites</i> de referencia para los jefes.

Paquete de trabajo: 5.2.3 Diseño y creación de sonidos.
Duración: 3 horas
Descripción: Se diseñan y crean los sonidos que aparecerán durante las batallas contra los jefes.
Salidas/Entregables: Archivos de audio con los sonidos.

5.3 Pruebas

Paquete de trabajo: 5.3 Pruebas.
Duración: 7 horas
Descripción: Se llevan a cabo pruebas para asegurar que los combates contra jefes estén balanceados y se crea una encuesta para obtener opiniones de personas ajenas al proyecto. En caso de haber errores, se corrigen.
Salidas/Entregables: Informe de pruebas.
Recursos necesarios: Versión ejecutable de la iteración 5.

3.2.3.7. Iteración 6

6.1 Diseño e implementación

Paquete de trabajo: 6.1.1 Sistema de guardado.
Duración: 8 horas
Descripción: Implementación de un sistema para guardar y cargar el progreso de la partida.

Paquete de trabajo: 6.1.2 Selección de personaje.
Duración: 5 horas
Descripción: Se diseña e implementa una función para que el jugador pueda escoger diferentes personajes al iniciar la partida y así modificar el inicio de la partida.

Paquete de trabajo: 6.1.3 Sistema de progresión.
Duración: 7 horas
Descripción: Se añade un sistema que desbloquea las armas y personajes a medida que el jugador juega varias partidas.

Paquete de trabajo: 6.1.4 Ajustes de accesibilidad.
Duración: 15 horas
Descripción: Diseño e implementación de un sistema que permita cambiar el idioma del juego a elección del jugador, configurar las teclas, tamaño de texto y otros ajustes de accesibilidad.

6.2 Diseño audiovisual

Paquete de trabajo: 6.2.1 Diseño y creación de personajes.
Duración: 3 horas
Descripción: Se diseña el aspecto de los nuevos personajes.
Salidas/Entregables: <i>Sprites</i> de los nuevos personajes.

Paquete de trabajo: 6.2.2 Diseño y creación de menús.
Duración: 3 horas
Descripción: Se diseñan y crean los elementos del menú de personajes y nuevas opciones de accesibilidad.
Salidas/Entregables: Imágenes del menú.

6.3 Pruebas

Paquete de trabajo: 6.3 Pruebas.
Duración: 6 horas
Descripción: Se llevan a cabo pruebas para asegurar que el progreso del juego se guarde bien y se haga ameno para los jugadores. En caso de errores se corrigen.
Salidas/Entregables: Informe de pruebas.
Recursos necesarios: Ejecutable de la iteración 6.

3.2.3.8. Documentación y presentación

Paquete de trabajo: 7.1 Creación de documentación.
Duración: 70 horas
Descripción: Se desarrollan los diferentes apartados de la documentación.
Salidas/Entregables: Documentación TFG.

7.2 Preparación presentación

Paquete de trabajo: 7.2.1 Creación de presentación.
Duración: 15 horas
Descripción: Se crean las diapositivas y la explicación que será presentada al tribunal.
Salidas/Entregables: Guion y diapositivas para la presentación.

Paquete de trabajo: 7.2.2 Práctica de presentación.
Duración: 4 horas
Descripción: Se practica la presentación previamente creada.
Recursos necesarios: Guion y diapositivas para la presentación.

3.3. Planificación temporal

Para un cierre satisfactorio de un proyecto, una pieza clave es acabarlo en la fecha estimada. Una correcta planificación temporal ayuda a organizar las tareas en el tiempo y es una forma fácil de medir el avance del desarrollo en comparación con la estimación inicial.

En la tabla mostrada a continuación, tabla 3.1, se puede ver el tiempo estimado de cada grupo de tareas y cómo afecta al conjunto total de horas.

Tabla 3.1

Estimaciones horarias del proyecto

	Nombre	Duración (horas)
0	Organización	20 [5 %]
1	Iteración 1	[10 %]
1.1	Diseño e implementación	26
1.2	Diseño audiovisual	10
1.3	Pruebas	5
	Total	41
2	Iteración 2	[15 %]
2.1	Diseño e implementación	32
2.2	Diseño audiovisual	23
2.3	Pruebas	6
	Total	61
3	Iteración 3	[12 %]
3.1	Diseño e implementación	26
3.2	Diseño audiovisual	13
3.3	Pruebas	7
	Total	46
4	Iteración 4	[10 %]
4.1	Adquisición de conocimientos	10
4.2	Diseño e implementación	20
4.3	Pruebas	7
	Total	37
5	Iteración 5	[14 %]
5.1	Diseño e implementación	32
5.2	Diseño audiovisual	18
5.3	Pruebas	7
	Total	57
6	Iteración 6	[12 %]
6.1	Diseño e implementación	35
6.2	Diseño audiovisual	6
6.3	Pruebas	7
	Total	47
7	Documentación y presentación	89 [22 %]
	Horas totales estimadas	398

A la hora de planificar cuándo se realizará cada tarea, se ha tomado como referencia que durante el primer cuatrimestre, debido a una mayor carga lectiva, se trabajarán aproximadamente 6 horas a la semana. Durante el segundo cuatrimestre, desde finales de enero, se ha supuesto que se realizarán 12 horas semanales. Este punto cuadrará aproximadamente con el inicio de la tercera iteración.

En total se estiman 398 horas de trabajo, iniciando el proyecto el 9 de septiembre de 2024 y cerrándolo el 18 de mayo de 2025. Esta fecha, la cual se considerará como la fecha de cierre del proyecto, no incluye la posterior creación de la defensa del proyecto en la fecha final.

A continuación se puede ver en el diagrama de Gantt, figura 3.6, el orden de los grupos de tareas y las fechas en las que se estima realizarlas. A excepción de la documentación, la cual se realiza a lo largo de todo el proyecto, las tareas se realizan de forma secuencial dado que se trata de un proyecto individual.

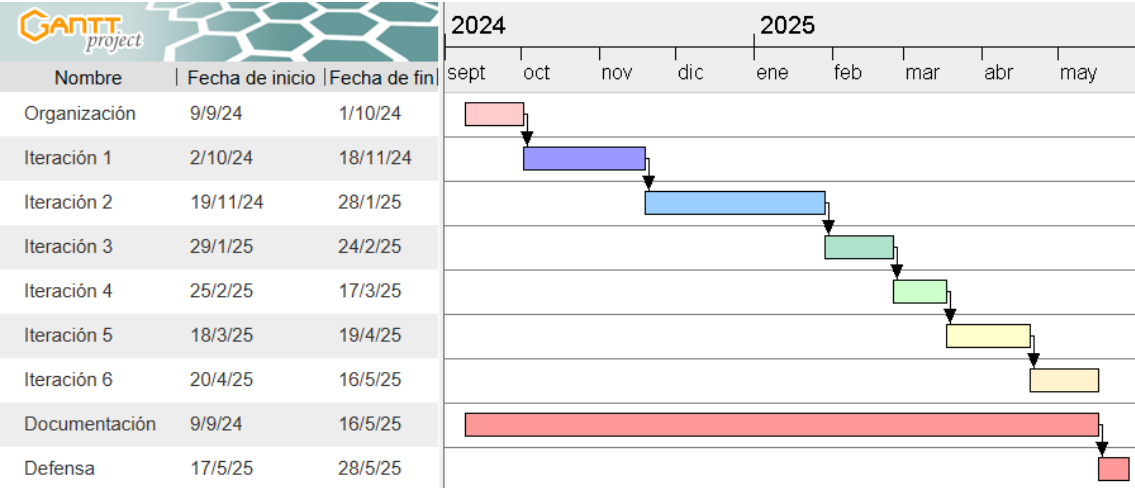


Figura 3.6: Diagrama de Gantt

3.4. Herramientas

Los proyectos hacen uso de herramientas para facilitar el trabajo o por necesidades de este, por lo que son una pieza clave en cualquier desarrollo. Para este proyecto se utilizarán herramientas de variadas utilidades las cuales se describirán a continuación.

3.4.1. Herramientas software

Durante el proyecto, la gran mayoría de herramientas que se usarán serán de software debido a la naturaleza del propio proyecto.

3.4.1.1. Google Drive

Google Drive¹ es un servicio de almacenamiento en la nube que se usará para tener accesibles varios archivos necesarios para la gestión del proyecto. Además de eso, se usarán las herramientas integradas como Documentos de Google para escribir documentos, Formularios de Google para llevar a cabo encuestas e incluso Dibujos de Google para la creación de ilustraciones como apoyo a explicaciones técnicas.

3.4.1.2. Overleaf

Overleaf² es un servicio en línea de creación y edición de archivos usando LaTeX, un sistema de composición tipográfica. Es la herramienta con la que se desarrollará la memoria y documentación.

3.4.1.3. Unity

Unity³ es un motor de videojuegos multiplataforma usado a nivel mundial en el desarrollo de videojuegos tanto 2D como 3D. En este se desarrollará el videojuego aprovechando los diferentes módulos y herramientas que proporciona.

3.4.1.4. Mega

Mega⁴ es un sitio web de almacenamiento donde se almacenarán los archivos relacionados con el juego, tanto el código, imágenes como el proyecto de Unity. Se usará su herramienta de sincronización para poder trabajar en el proyecto en varios dispositivos.

3.4.1.5. Visual Studio Community

Visual Studio Community⁵ es un IDE, *Integrated Development Environment*, que se usará para editar el código durante el desarrollo.

3.4.1.6. Aseprite

Aseprite⁶ es un editor de gráficos rasterizados, es decir, hechos con píxeles en vez de vectores. Esta herramienta se usará para la creación de las imágenes que se verán en el juego. Tanto el personaje, como los fondos y enemigos.

¹<https://www.google.com/intl/es-es/drive/>

²<http://www.overleaf.com>

³<https://unity.com/es>

⁴<https://mega.io/es>

⁵<https://visualstudio.microsoft.com/es/vs/community/>

⁶<https://www.aseprite.org/>

3.4.1.7. Audacity

Audacity⁷ es una aplicación informática multiplataforma libre de grabación y edición de audio. En este caso se usará para trabajar los efectos de sonido que aparezcan en el juego.

3.4.2. Herramientas físicas

Estos son los diferentes dispositivos y herramientas auxiliares que se usarán durante el desarrollo.

3.4.2.1. Ordenador de sobremesa

El ordenador en el que principalmente se trabajará. Este cuenta con las siguientes características:

- Procesador, AMD Ryzen 5 2600X 3.6 GHz
- RAM, DDR4 8 GB HyperX x2 (total 16GB)
- Disco, 1 TB de memoria SSD
- Tarjeta gráfica, Gigabyte GeForce GTX 1650 OC 4GB GDDR5
- Sistema operativo, Windows 10 Pro

3.4.2.2. Ordenador portátil

Un portátil, Lenovo IdeaPad 3 Gen 6, para poder trabajar aun no estando en el lugar de trabajo habitual. Este cuenta con las siguientes características:

- Procesador, Intel Core i5-1155G7
- RAM, 16 GB
- Disco, 512 GB de memoria SSD
- Sistema operativo, Windows 11 Home / Ubuntu

3.4.2.3. Tableta gráfica

Una tableta gráfica Star G640 de XP Pen⁸. Usada para la creación de *sprites* y otras tareas relacionadas con el apartado artístico durante el desarrollo.

⁷<https://www.audacityteam.org/>

⁸<https://www.xp-pen.es/product/236.html>

3.5. Gestión de riesgos

Durante un desarrollo, muchos contratiempos pueden ocurrir. Es por ese motivo que es importante tener soluciones preparadas para diferentes problemas, o todavía mejor, prevenir los problemas.

Cuando sucede un incidente, la planificación temporal realizada se ve comprometida. Dependiendo de la situación, tareas que deberían haber sido acabadas en cierto plazo, pueden verse atrasadas y en consecuencia, retrasar las siguientes. Es por este motivo que resalta la importancia de una buena prevención. Para evitar que un pequeño retraso impacte a todo el proyecto, se añadirán pequeñas holguras entre tareas, a fin de tener un pequeño intervalo de seguridad.

Dependiendo de cuánto retraso suponga un incidente, será clasificado por su impacto. En la tabla 3.2 se puede ver la clasificación de impactos.

Tabla 3.2

Retraso en el proyecto dependiendo del riesgo

Impacto del riesgo	Retraso
Bajo	< 1 día
Medio	1 ~ 2 días
Alto	3 ~ 7 días
Muy alto	8 ~ 30 días

Además del impacto que un riesgo puede suponer, es importante medir su probabilidad. En la tabla 3.3 se puede ver la estimación del porcentaje de que el riesgo en cuestión ocurra a lo largo del desarrollo.

Tabla 3.3

Probabilidad de aparición dependiendo del riesgo

Probabilidad del riesgo	Porcentaje
Bajo	< 15 %
Medio	15 % ~ 30 %
Alto	30 % ~ 70 %
Muy alto	> 70 %

Teniendo todo esto en cuenta, se pueden listar y clasificar los riesgos. Cabe mencionar, que estos se dividirán en riesgos informáticos y físicos y de gestión. La primera categoría consistirá en aquellos riesgos cuya causa sea un evento físico, ya sea robo de material, enfermedad o riesgos asociados a la gestión del proyecto. En el caso de los informáticos, se encuentran los riesgos relacionados con el entorno del software, como la pérdida de datos o cambios de políticas de software.

3.5.1. Riesgos informáticos

Pérdida de datos.
Descripción: Pérdida de archivos por fallos de sistemas, borrado accidental o inaccesibilidad al hardware.
Medidas de prevención: Llevar un registro de todos los documentos y archivos generados. Asignar nivel de importancia a los archivos y hacer copias de seguridad en consecuencia. Se harán varias copias, tanto en la nube como en almacenamiento físico.
Plan de contingencia: Recuperar archivo de su última copia de seguridad y rehacer el trabajo perdido.
Probabilidad: Media.
Impacto: Bajo.
Cambio de políticas del software.
Descripción: Cambio en las políticas de software de una aplicación clave para el desarrollo de forma que afecten directamente al proyecto.
Medidas de prevención: Antes de decidirse por un software investigar la historia de este para buscar casos parecidos y evitar ese software.
Plan de contingencia: Ajustarse a las nuevas normas y políticas o cambiar de software, dependiendo del impacto del cambio.
Probabilidad: Baja.
Impacto: Medio / Muy alto.

<i>Overscoping.</i>
Descripción: Riesgo común en el desarrollo de videojuegos. Consiste en intentar añadir demasiado contenido y funciones a un proyecto, siendo más de lo que se puede abarcar.
Medidas de prevención: Analizar el coste de cada funcionalidad para asegurarse de no causar una sobrecarga de trabajo.
Plan de contingencia: Reevaluar la planificación y eliminar funcionalidades que puedan considerarse innecesarias.
Probabilidad: Medio.
Impacto: Medio.

Pruebas poco eficaces.
Descripción: A causa de una realización de pruebas deficiente, el producto final no se encuentra en las condiciones esperadas.
Medidas de prevención: Realizar pruebas exhaustivas y compartir la aplicación con un grupo de personas para que prueben el producto, a fin de encontrar errores inesperados.
Plan de contingencia: Volver a la etapa de desarrollo y realizar pruebas hasta tener un producto en condiciones óptimas.
Probabilidad: Baja.
Impacto: Alto.

3.5.2. Riesgos físicos y de gestión

Falta de acceso al hardware.
Descripción: Se da en caso de no poder acceder al hardware comúnmente utilizado para el desarrollo del proyecto. Se puede dar por el robo de un portátil o un fallo en las piezas que incapacitan su uso.
Medidas de prevención: Un buen mantenimiento del hardware, con revisiones de funcionalidad y un buen cuidado de este, no dejándolo desatendido en lugares poco fiables.
Plan de contingencia: Utilizar un hardware similar al que se tenga acceso, como puede ser un ordenador de repuesto al que se le hayan cargado los datos de las copias de seguridad.
Probabilidad: Baja.
Impacto: Medio.

Estimación incorrecta de duración de tarea.
Descripción: A la hora de planear la duración de las tareas, es posible subestimar el trabajo que conlleva una tarea y acabar teniendo que retrasar las siguientes.
Medidas de prevención: Comparar tareas con proyectos pasados para realizar una estimación lo más correcta posible.
Plan de contingencia: Retrasar las tareas siguientes.
Probabilidad: Alto.
Impacto: Medio.

Casos similares: Imposibilidad personal de trabajo

Dentro de los riesgos, se pueden dar varios casos con resoluciones y consecuencias similares. En este caso, los planes de contingencia de los 3 siguientes riesgos acababan siendo retrasar tareas.

Sobrecarga de trabajo.
Descripción: Debido a la realización del proyecto, estudios y trabajo, acabar sobrecargado y no poder realizar las tareas planeadas en el plazo previsto.
Medidas de prevención: Planificar las tareas teniendo en cuenta la carga laboral fuera del proyecto.
Plan de contingencia: Revaluar la planificación temporal.
Probabilidad: Bajo.
Impacto: Medio.

Retraso por enfermedad.
Descripción: Imposibilidad de trabajar debido a enfermedades o condiciones médicas inesperadas. Las tareas planificadas se ven retrasadas en consecuencia.
Medidas de prevención: Mantener una higiene adecuada y tomar precauciones en entornos contaminados.
Plan de contingencia: Priorizar la recuperación de la salud a fin de no prolongar el malestar.
Probabilidad: Medio.
Impacto: Alto.

Causas personales.
Descripción: A lo largo del desarrollo puede ocurrir algún evento personal que impida o dificulte el trabajo.
Medidas de prevención: Dado que son hechos inesperados, no se puede prevenir.
Plan de contingencia: Reevaluar la planificación temporal.
Probabilidad: Medio.
Impacto: Alto.

3.6. Evaluación económica

Una pieza clave a la hora de considerar si llevar a cabo un proyecto es el apartado económico. En este proyecto, los costes estimados serán medidos tanto por los materiales usados como por las horas de trabajo.

En cuanto a materiales, se encuentran los costes de los ordenadores, herramientas y las aplicaciones usadas. En la tabla 3.4 se muestran los costes de estos. Dado que estas herramientas no se usarán durante toda su vida útil para este proyecto, es importante tener en cuenta el coste relativo a los 9 meses de duración del trabajo. La vida útil de cada herramienta son estimaciones generales del tipo de producto, dado que es una duración que varía en gran medida dependiendo de su cuidado y de factores externos, como puede ser un accidente.

Tabla 3.4

Coste relativo de los materiales

Material	Coste	Vida útil	Coste relativo
Ordenador de sobremesa	1000 €	78 meses	115.4 €
Portátil	550 €	45 meses	110 €
Tableta gráfica	30 €	120 meses	2.25 €
		Total	227.65 €

Todas las herramientas usadas son gratuitas o disponen de alguna licencia sin coste para uso personal. En el caso de Unity, se dispone de una licencia gratuita dependiendo de los beneficios obtenidos por su uso. Según las políticas de Unity, con su uso personal se deberá comprar una licencia a partir de generar 200.000\$ en beneficios, como se puede ver en su web oficial [2].

Las horas de trabajo se contabilizarán usando el salario promedio de un programador/diseñador. Se ha estimado un coste de aproximadamente 15 € por hora. Este coste se ha tomado de un estudio realizado por el Servicio Público de Empleo Estatal [3]. Con un total de 398 horas de trabajo, el coste asciende a 5970 €.

Además, se han incluido otros costes como la electricidad usada. Esta se ha estimado como un 5 % del coste del proyecto. El resultado total se puede ver en la tabla 3.5.

Tabla 3.5

Costes totales estimados del proyecto

Resumen	Coste
Materiales	227.65 €
Recursos humanos	5970 €
Otros costes	309.88 €
Total	6507.53 €

3.6.1. Rentabilidad económica

A la hora de completar un videojuego, la forma más directa de rentabilizarlo es publicarlo en una página de venta de videojuegos. De esta forma no es necesario preocuparse por la distribución ni por sistemas de pago.

Hoy en día, las dos plataformas de distribución digital de videojuegos principales son Steam y Epic Games, siendo Steam la predominante. En cuanto a videojuegos indie, Itch.io es la plataforma con más renombre.

Económicamente hablando, a la hora de escoger una plataforma es necesario tener en cuenta los costes de publicación y el reparto de beneficios. En el caso de Itch.io, no tiene ningún coste de publicación y el reparto de beneficios es decidido por el propio usuario al publicar el juego, pudiendo establecerlo desde 0 % hasta 100 %. Originalmente no existía la posibilidad, e Itch.io no se llevaba nada del dinero, pero al popularizarse y aumentar los costes de mantenimiento, decidieron crear la opción para apoyar a la plataforma. Esto se puede ver en su página de noticias [4].

En el caso de Steam [5] y Epic Games[6], ambas requieren un pago inicial de 100\$ por la publicación del juego y luego se llevan un 30 % y un 12 % de los beneficios de este respectivamente.

Suponiendo que el juego se publica en Steam, a un precio de 5 €, sería necesario vender aproximadamente 1888 copias del juego. En el caso de Epic Games 1502 copias y en Itch.io, con una donación de beneficios del 5 %, 1391.

Conseguir esas cifras es complicado en cualquiera de las plataformas, pero como se trata de un trabajo de fin de grado, la falta de viabilidad económica no es un impedimento para desarrollarlo.

4. Antecedentes

Como previamente se ha mencionado, los videojuegos son una industria que lleva años en crecimiento y no parece que vaya a parar. La creación de un videojuego no es algo trivial, y consta de muchas partes. En un videojuego a gran escala trabajan personas especializadas en programación, audiovisuales y hasta psicología. Aunque este desarrollo se centre principalmente en el apartado técnico, es importante e interesante conocer el trabajo que conllevan este tipo de proyectos, en todos los ámbitos que engloban.

Una vez decidido que se hará un videojuego, toca decidir el tipo de juego, la forma más común siendo la asignación de un género. Con el tiempo aparecen videojuegos que sientan precedente y crean un nuevo género de videojuegos, una fórmula que se repetirá y expandirá en juegos futuros. Actualmente hay cientos de géneros, desde plataformas, como *Super Mario* donde saltas por plataformas, hasta *Metroidvania*, género que toma conceptos tanto del videojuego *Metroid* como *Castlevania*. En el caso de *roguelike*, proviene de *Rogue*, un videojuego lanzado en 1980, el cual se considera el origen del género. La elección del género se basa en la simple premisa de este, si pierdes vuelves a empezar. Esta es una idea simple, pero puede tener interesantes ramificaciones.

En cuanto a juegos similares, se ha tomado inspiración de *Hollow Knight*, *Blasphemous* y *Soul Knight*. Los dos primeros videojuegos pertenecen al género Metroidvania 2D, el segundo habiendo sido creado por un estudio español. En estos está basado el movimiento del personaje, por su fluidez y comodidad de los controles. De *Soul Knight*, un *roguelike*, se ha tomado inspiración para las armas. En este juego se pueden encontrar hasta 483 armas, las cuales dan una gran variedad de posibilidades. En este juego se pueden llevar dos armas y este es un concepto que se ha aprovechado para este proyecto dado que permite al jugador realizar combinaciones interesantes.

En la mayoría de videojuegos se suelen separar los objetos clasificados como armas y aquellos que aportan beneficios al jugador, como puede ser desplazarse más rápido o realizar más daño. Para este se ha planteado unir las dos cosas. A estos se les referirá como accesorios. Se ha mencionado que se podrán llevar dos armas, una que estará en uso y la otra en reserva. Las armas tendrán bonificaciones, pero estas únicamente se activarán mientras estén en reserva, es decir, se combinan la función de arma y accesorio. De esta forma, si el jugador encuentra un buen arma, tendrá que decidir si usarla como arma principal o mantenerla en reserva, dando la posibilidad de plantear estrategias con ciertas combinaciones.

El proyecto está pensado para ser jugado en PC. Hoy en día, los videojuegos se pueden dividir en tres grandes grupos basándose en la plataforma donde se juegan: teléfono, PC y consola. El mercado móvil suele estar centrado en juegos gratuitos con anuncios o microtransacciones. En consolas, los videojuegos requieren el software específico de la consola. Por último, en PC, gracias a la facilidad de publicación y de desarrollo de videojuegos, es donde se encuentra la mayoría del mercado de videojuegos independientes o *indie*, grupo en el que se categorizaría este proyecto.

Para el desarrollo de videojuegos en PC existe gran variedad de herramientas tanto gratuitas como de pago, de las cuales destacan tres: Unity, Unreal Engine y Godot. De estos tres Unreal Engine queda descartado dado que se especializa en el desarrollo de videojuegos 3D, como el conocido videojuego *Fornite*.

Godot es un motor de videojuegos de código libre, mayormente centrado en los juegos 2D aunque está ganando popularidad en los años recientes.

A pesar de todo, el motor por excelencia y el seleccionado para este desarrollo suele ser Unity. Los motivos para la elección de este son varios, desde la versatilidad, la gran cantidad de documentación que posee, o más importante, la experiencia previa. En la introducción queda mencionada la creación de juegos previos a este y siendo realizados con Unity, otorgan una familiaridad con el entorno que facilita el desarrollo.

5. Captura de requisitos

En esta sección se analizan los requisitos que debe cumplir el videojuego, es decir, recopilar las diferentes funcionalidades que se esperan del proyecto.

5.1. Jerarquía de actores

A la hora de analizar los tipos de usuario de un videojuego, dependen mucho del tipo de videojuego y su implementación. Si es un videojuego multijugador, puede haber diferentes roles entre los jugadores, donde una persona puede tener más control sobre los ajustes del juego. En estos casos también pueden existir los administradores, los cuales no juegan al juego sino que lo moderan. En este caso, al tratarse de un videojuego de un único jugador solo habrá un usuario: la persona que juegue.

Jugador: Persona que juega al juego.

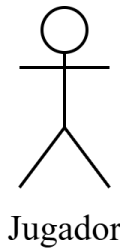
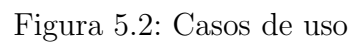


Figura 5.1: Jerarquía de actores

5.2. Modelo de casos de uso

En el gráfico 5.2 se pueden observar los casos de uso de Loop Escape. En el anexo, capítulo 10, se pueden encontrar los diagramas de secuencia para los diferentes casos y subcasos de uso.



- **Iniciar nueva partida:** Obtiene el identificador del personaje seleccionado, crea unos datos nuevos de partida y carga el nivel.
- **Continuar partida:** Utilizando los datos de la partida guardada en el dispositivo, carga el nivel.
- **Ajustes:** Visualiza y/o modifica ajustar ciertos parámetros del videojuego.
- **Cargar nivel:** Carga una nueva escena de Unity. Este subcaso de uso se expande en múltiples subcasos tal y como se muestra en la figura 5.3.

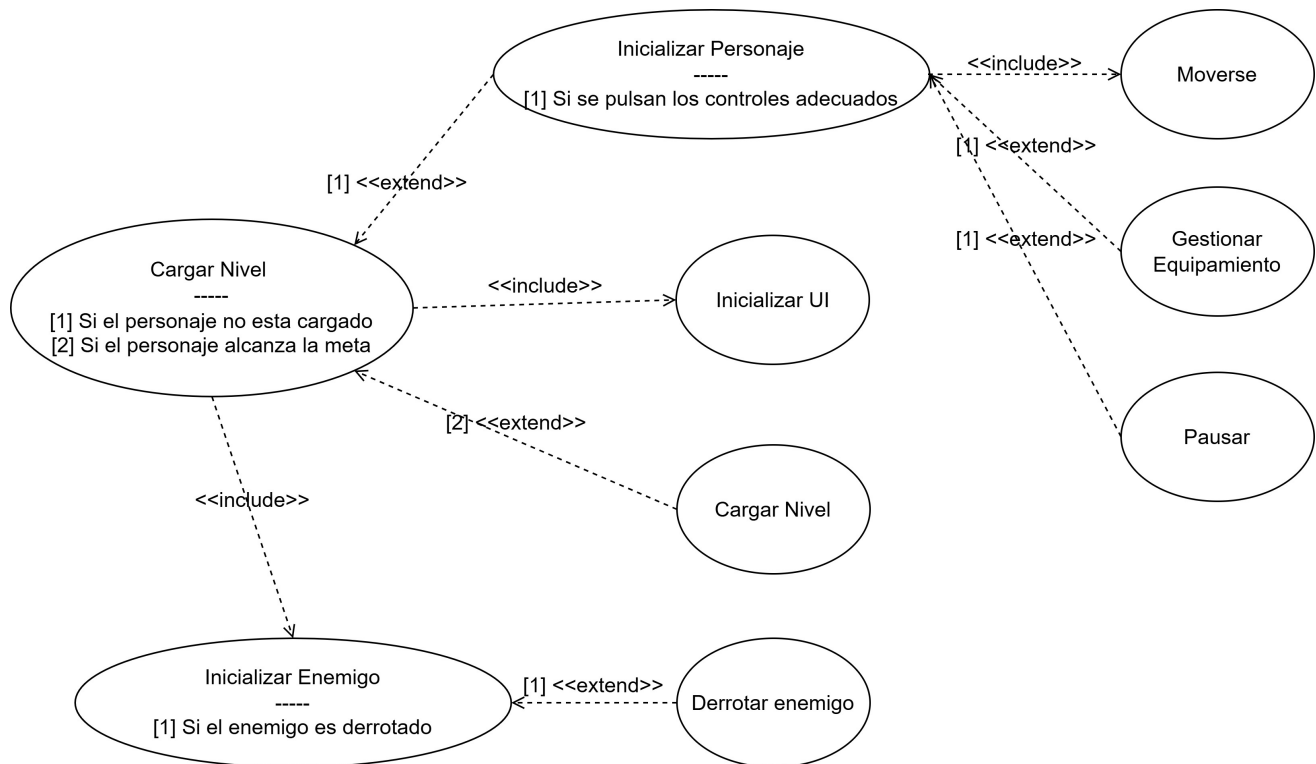


Figura 5.3: Subcasos de uso de “Cargar Nivel”

- **Inicializar personaje:** Cuando se crea el objeto del personaje, se cargan los datos de la partida. Estos datos inicializan los diferentes componentes del personaje, como la vida que tiene o las armas que posee. Estando inicializado, el personaje comienza a verse afectado por las físicas del juego y a responder a los controles del jugador.
- **Moverse:** El personaje camina o salta.
- **Gestionar Equipamiento:** Dependiendo de las acciones del jugador, se cambia de arma, se recogen y/o desechan armas del suelo o se usa un arma para atacar.
- **Pausar:** Se pausa el juego y se abre un menú donde se muestran diferentes datos del personaje y su equipamiento.
- **Inicializar UI:** Se cargan en pantalla los diferentes elementos que muestran información al jugador, como una barra indicando el número de enemigos.
- **Inicializar Enemigo:** Se inicializa el comportamiento de cada enemigo en el nivel.
- **Derrotar Enemigo:** Cuando un enemigo es derrotado, se crea botón para el personaje. Este nuevo objeto contiene un arma la cual puede ser usada por el personaje.

5.3. Modelo de dominio

En la figura 5.4 se pueden ver los diferentes datos que se gestionan en el juego así como sus relaciones.

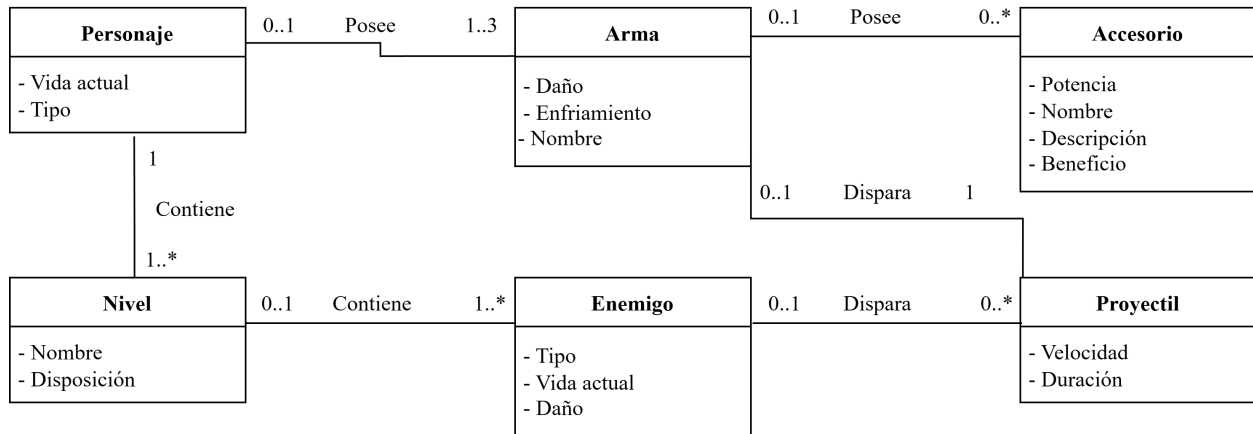


Figura 5.4: Modelo de dominio

A continuación se desarrolla la función de cada una de las entidades.

- **Personaje:** Esta entidad representa el estado del personaje durante una partida, como puede ser la vida actual.
- **Arma:** Un arma es un elemento que permite al personaje atacar. Las armas están definidas por su nombre, el daño que hacen y cada cuánto pueden atacar.

El personaje puede poseer hasta tres armas. Una de ellas es fija y es la utilizada en caso de no tener más armas. Además, el personaje puede poseer dos más.

- **Accesorio:** Guarda datos sobre los beneficios que aporta al personaje además de datos que describen la propia entidad.
- **Nivel:** Un nivel representa un escenario del juego. Los niveles contienen varios enemigos pero únicamente un personaje.
- **Enemigo:** Los enemigos son los obstáculos contra los que se enfrenta el personaje. Cada tipo de enemigo es definido por su identificador y cada uno hace más o menos daño y tiene diferentes valores de vida.
- **Proyectoil:** Los proyectiles guardan la forma en la que se va a comportar este una vez sea disparado. Su comportamiento depende tanto de la velocidad como de la duración.

Tanto las armas como los enemigos tienen relación con los proyectiles. Cada arma tiene un proyectil asociado, este será el proyectil que dispara. En el caso de los enemigos, estos pueden disparar varios tipos de proyectiles o no disparar ninguno.

6. Análisis y diseño

Para poder implementar los diferentes requisitos establecidos, se hace uso de múltiples clases. Estas quedan definidas en el diagrama de clases.

6.1. Diagrama de clases

En las figuras mostradas a continuación se pueden ver varios diagramas de clases del proyecto. Debido a la gran cantidad de clases en el proyecto, los gráficos solo muestran las clases principales del juego. En el anexo, capítulo 10, se encuentra la figura 10.12, la cual contiene un diagrama detallado. En el gráfico se muestran muchas clases, las cuales pueden ser agrupadas basándose en la forma en la que son usadas. A continuación se explicarán varias de las clases o grupos.

ControladorPersonaje es la clase encargada de gestionar la funcionalidad del personaje. Haciendo uso de una gran variedad de clases, las conecta para hacer funcionar al personaje. Estas funciones incluyen inicializar el personaje, detectar las entradas por teclado, gestionar la respuesta a estas acciones y aplicar las físicas entre otras.

El primer grupo de clases son aquellas específicas del personaje, como pueden ser **GestorInput**, **GestorEquipamiento** o **RecolectorLoot**. Tal y como sus nombres indican, gestionan diferentes funcionalidades del personaje, desde gestionar el equipamiento, hasta mover el personaje. En la figura 6.1 se pueden ver estas clases en conjunto con la del personaje.

- **GestorInput** gestiona la entrada, o *input*, por teclado.
- **SaltoJugador** gestiona el salto del personaje.
- **GestorEquipamiento** permite atacar, cambiar entre armas o recoger y desechar armas.
- **RecolectorLoot** detecta el botín alrededor del personaje para que este pueda ser recogido.
- **AnimadorPersonaje** y **GestorSonidosPersonaje** gestionan las animaciones y sonidos del personaje respectivamente.

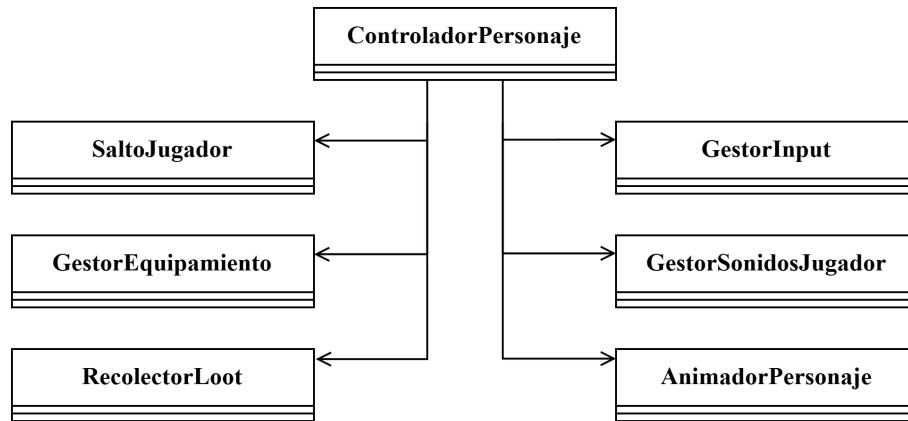


Figura 6.1: Diagrama de clases, parte del personaje

ComportamientoCucaracha es una de las clases de comportamientos existentes. Cada tipo de enemigo posee un comportamiento y clase diferente. Estas clases inicializan los enemigos y gestionan los diferentes estados en los que puede estar cada enemigo. Al igual que con la clase **ControladorPersonaje** hacen uso de múltiples clases para lograr su objetivo.

Loot, o botín en castellano, es una clase que gestiona el comportamiento del botín. Este aparece como una entidad en el juego y almacena la referencia a un arma.

El siguiente grupo de clases son aquellas usadas por las diferentes entidades que aparecen en el juego, entre las que se encuentran el personaje, enemigos y *loot*.

- Dentro de este grupo se encuentra el subgrupo de clases relacionadas con las físicas del juego, como pueden ser **Desplazamiento**, **Gravedad** o **Fricción**.
- **Vida** y **Estado** se encargan de controlar el recibo de daño de la entidad.
- Por último están las clases “sensor” las cuales detectan diferentes elementos. Estas son **SensorSuelo**, **SensorCaída**, **SensorPared** y **SensorVision**. Cada una detecta si la entidad toca el suelo, si se va a caer, si está tocando una pared o si el personaje está en su línea de visión respectivamente.

En la figura 6.2 se pueden ver las clases del *Loot*, personaje y enemigos y sus relaciones con las clases generales.

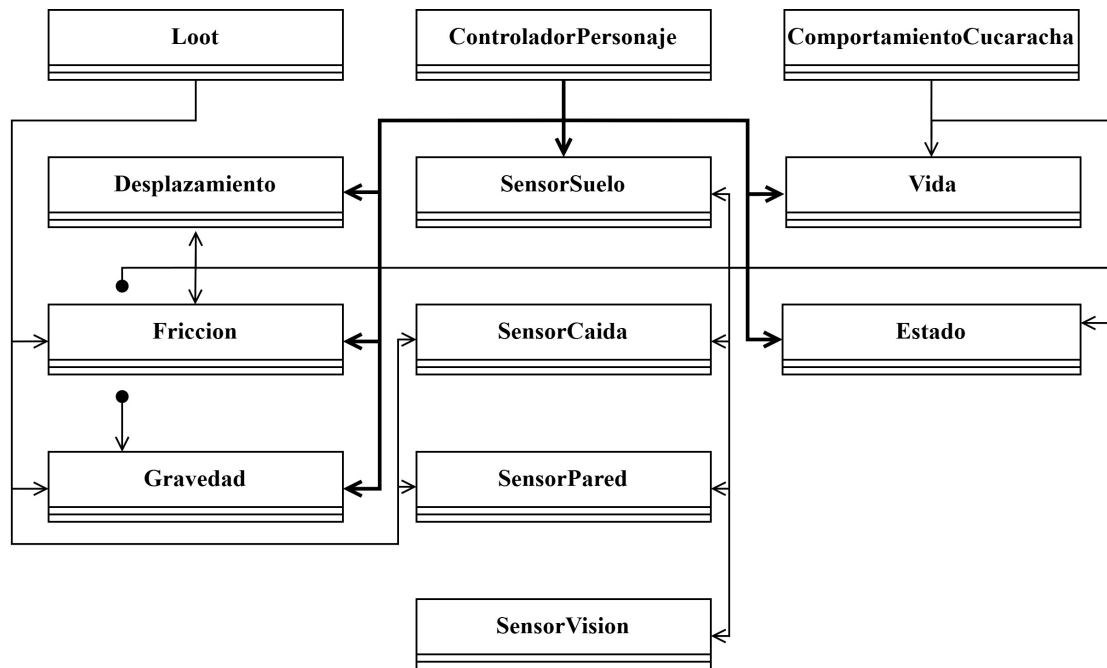


Figura 6.2: Diagrama de clases, clases generales

Cuando un enemigo es derrotado, cabe la posibilidad de que genere algo de botín. Para ello se hace uso de múltiples clases. **DerrotaEnemigo** es una clase la cual comprueba qué componentes posee el enemigo. Si tiene un componente de animación, inicia la animación de derrota y, si tiene la clase **SoltadorLoot**, ejecuta el método para soltar *loot*. Esta clase contiene las probabilidades e información sobre el posible botín. Esta información es pasada al **GeneradorLoot** y esta se encarga de crear el botín. Esto puede ser visto en la figura 6.3.

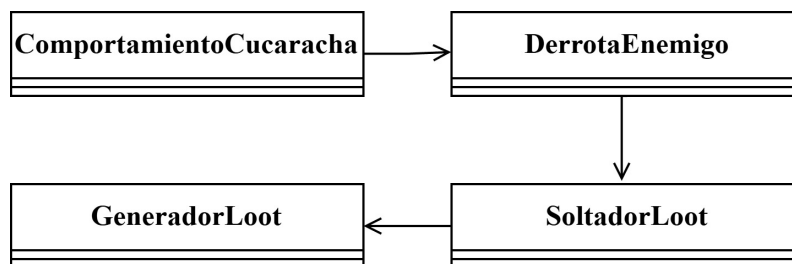


Figura 6.3: Diagrama de clases, derrota de enemigo

Controlando todo el juego se encuentra la clase **ControladorJuego**, la cual gestiona el estado de la partida. Se encarga de controlar si los requerimientos para pasar de nivel han sido cumplidos y hace uso de la clase **SceneManager** de Unity para cargar los diferentes niveles. También se comunica con la clase **GestorPausa** para parar el juego tal y como se ve en la figura 6.4 y hace uso de la clase **GestorUI** para conectarse a los diferentes elementos de la interfaz.

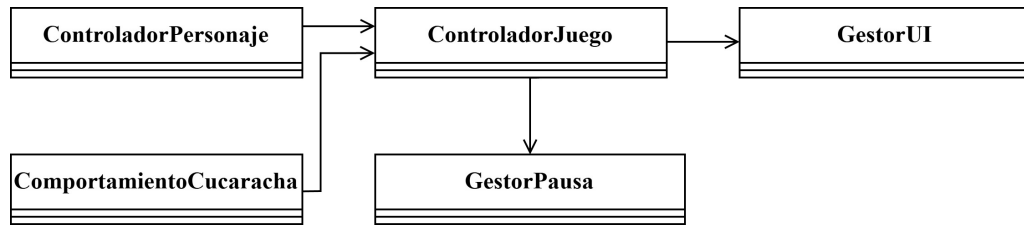


Figura 6.4: Diagrama de clases, controlador

Por último está la clase **Arma**, la cual es capaz de disparar proyectiles. Cada arma se encarga de crear, gestionar y disparar proyectiles, además de controlar los diferentes accesorios.

Proyectil es la clase que gestiona el comportamiento de los proyectiles. Este comportamiento viene dado por las diferentes clases “modificadoras”. Estas clases son **ModCondiciona**l y **ModPosicion**, entre otras. La primera define las condiciones con las cuales un proyectil es destruido. La segunda define el movimiento del proyectil. Estas se muestran en la figura 6.5.

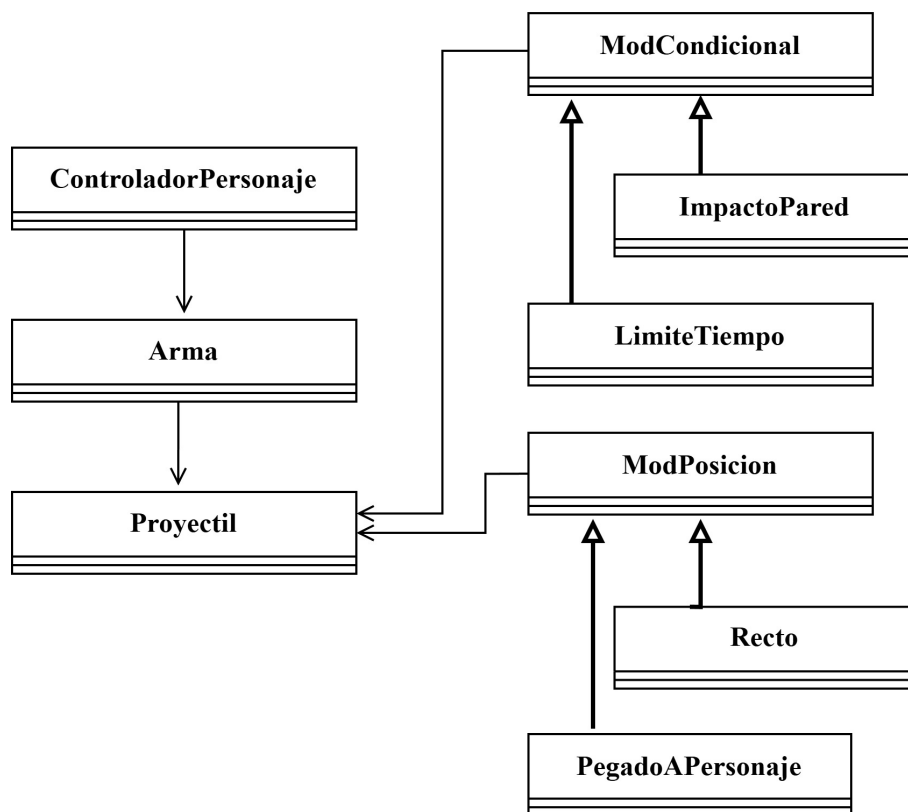


Figura 6.5: Diagrama de clases

7. Desarrollo y evaluación

En este apartado, se llevará a cabo la explicación del desarrollo y las pruebas realizadas junto a los problemas surgidos en este y las soluciones a dichos problemas. Siguiendo la estructura del desarrollo, este capítulo contará con 6 apartados, uno por cada iteración. Además, cada apartado contará con su sección de pruebas.

7.0. El entorno de desarrollo de Unity

Antes de comenzar con el desarrollo, se llevará a cabo una breve introducción sobre Unity para asentar una base sobre la que desarrollar.

Unity funciona mediante objetos y escenas. Se puede entender una escena como un nivel del juego, aunque apartados como los menús pueden tener su propia escena. Cada escena tiene una lista de objetos, o *GameObjects*, que representan todo tipo de elementos, desde el personaje hasta la cámara.

Los *GameObjects* están compuestos de componentes, los cuales establecen ciertos atributos y lógica al objeto. En Unity hay muchos tipos de componentes, siendo los más comunes e importantes para este juego:

- **Transform:** Define la posición, rotación y escala del objeto.
- **Sprite Renderer:** Permite renderizar un *sprite* en pantalla.
- **Script:** Ejecuta el código asignado durante la ejecución de la escena.
- **Rigidbody 2D:** Usado para controlar el objeto con el sistema de físicas 2D de Unity, es decir, el objeto posee atributos como peso, fricción o gravedad.
- **Box Collider 2D:** Según el manual de Unity [7], los componentes *Collider* definen la forma de un objeto para los propósitos de colisiones físicas. Existen otros tipos de *Collider* dependiendo de la forma que se busque.

En Unity y en los videojuegos en general existe un ciclo de ejecución y un orden en el que se llevan a cabo ciertas acciones. Estos ciclos se repiten para cada fotograma generado.

Para explicar el orden de ejecución de eventos de Unity para el ciclo de vida de un *script* [8], se ha creado una versión simplificada del gráfico mostrado en la documentación de Unity. El diagrama de flujo muestra los eventos llamados y su orden. A continuación de la figura 7.1 se explica el funcionamiento de cada apartado.

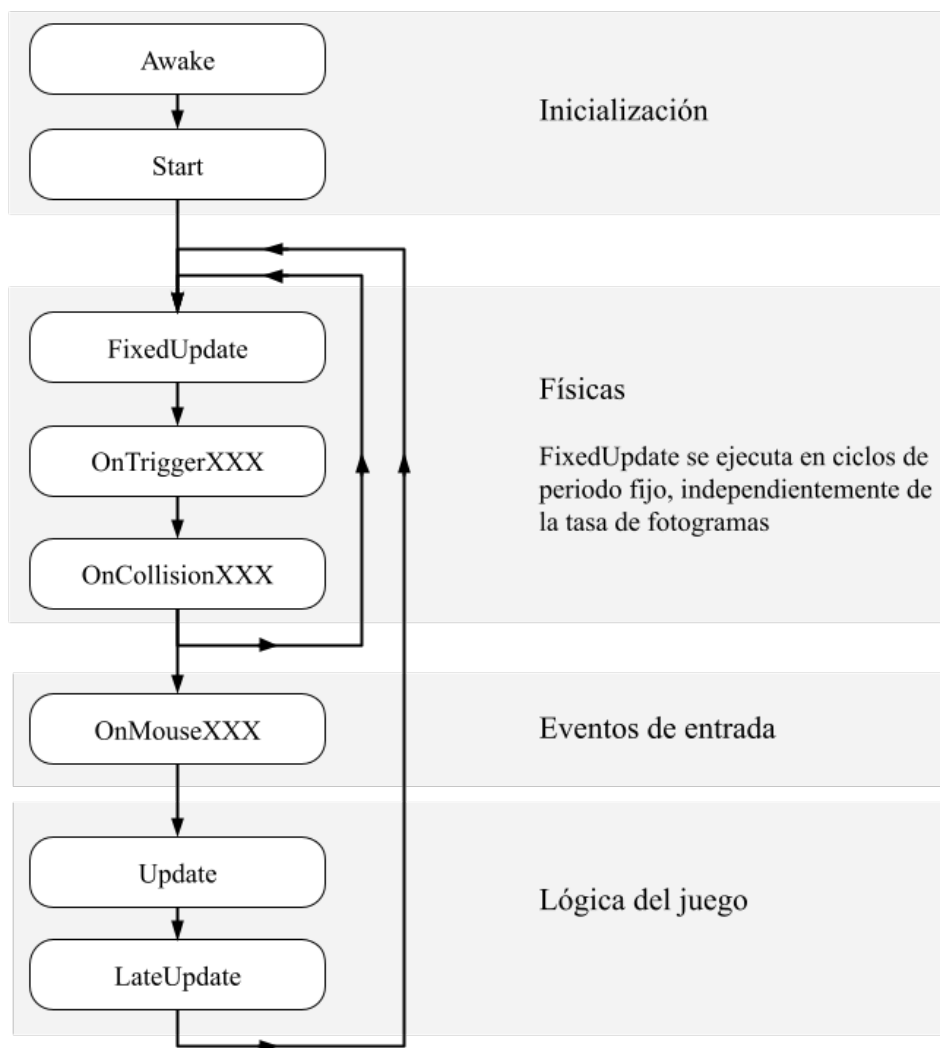


Figura 7.1: Ciclo de ejecución simplificado

- **Awake:** Se ejecuta cuando una nueva instancia del objeto es creada.
- **Start:** Ejecutada antes del primer fotograma.
- **FixedUpdate:** Se ejecuta en intervalos fijos, todos los cálculos de físicas se ejecutan después de este.
- **OnTriggerXXX:** Se dispara cuando ocurre un evento de tipo gatillo.
- **OnCollision:** Llamado cuando dos objetos colisionan.
- **OnMouseXXX:** En este punto del ciclo se reciben las entradas por parte del jugador.
- **Update:** Se ejecuta una vez por fotograma.
- **LateUpdate:** Se ejecuta una vez por fotograma, después de `Update`.

Los *scripts* que controlarán el juego están escritos en C#, un lenguaje de programación de Microsoft. En Unity, al crear un *script*, este es hijo de la clase MonoBehaviour de la cual heredan ciertos métodos. Estos métodos pueden ser llamados por el propio motor en respuesta a eventos, como pueden ser los métodos Start() o Update().

7.1. Iteración 1

En este apartado se verá la primera parte del desarrollo del videojuego, donde se implementan las principales características básicas de forma que se pueda obtener una prueba jugable.

Requisitos de la iteración

- Aspecto jugable
 - El personaje podrá desplazarse lateralmente y saltar.
 - El jugador podrá controlar al personaje mediante un teclado.
 - La cámara sigue al personaje.
 - Implementación de *Buffer-Jumping* y *Coyote-Time*.
 - Una sala, con plataformas y un tipo de enemigo (sala creada a mano).
 - Los enemigos se podrán derrotar, y atacarán al personaje.
 - Al derrotar a los enemigos, aparece la salida.
 - Al entrar a la salida, se felicitará al jugador.
- Aspecto Visual
 - Las plataformas, enemigos y el personaje deberán tener *sprites* con los que se les pueda reconocer.

Para comenzar el desarrollo, se crea un proyecto de Unity en la versión 2022.3.45f1 LTS. Se usa la plantilla *2D (Built-In Render Pipeline)*, diseñada para el desarrollo de videojuegos en 2D.

7.1.1. Movimiento del personaje

Para que el jugador pueda explorar los niveles, el personaje tiene que poder desplazarse por ellos. Para ello requiere las capacidades de moverse lateralmente, saltar y caer. Como previamente se ha mencionado, el personaje será un `GameObject` con componentes específicos. En este caso serán: `Transform`, `Sprite Renderer`, `Script`, `Rigidbody 2D` y `Box Collider 2D`. La lógica del movimiento del personaje se lleva a cabo en el bucle `Update`. En este se ejecutan los subprogramas que controlan y regulan el movimiento. Los pasos seguidos son:

- **Leer input:** Lee y guarda el input del jugador.
- **Saltar:** Dependiendo del input del jugador, hace saltar al personaje.
- **Caminar:** Dependiendo del input del jugador, desplaza el personaje.
- **Fricción:** Desacelera lateralmente al personaje.

- **Gravedad:** Acelera negativamente de forma vertical al personaje.
- **LimitarVelocidad:** Se asegura de que el personaje no supere unos límites de velocidad.

Primero se obtienen las acciones del jugador utilizando la interfaz Input de Unity. Concretamente, si ha pulsado el botón de saltar, la tecla espacio o 'a'/'d' para desplazarse lateralmente.

Si el jugador quiere desplazarse lateralmente, se guarda la dirección en la que se quiere mover, si el jugador intenta saltar, se guarda su intención para procesarla más adelante. Cuando se lee un input, se diferencia entre comenzar a pulsar una tecla y mantener una tecla pulsada. Esto será útil para controlar los saltos.

A la hora de saltar tienen que cumplirse 3 condiciones.

- El personaje tiene que estar tocando el suelo.
- El jugador ha pulsado la tecla para saltar.
- El personaje no ha podido ser atacado hace poco.

Para mejorar la sensación de juego se han implementado dos técnicas que afectan a dos de las condiciones previamente mencionadas, *Jump Buffering* y *Coyote Time*.

Jump Buffering consiste en guardar la intención del jugador de saltar durante un corto periodo de tiempo. Quizás en el ciclo actual el personaje no podría saltar, pero en el siguiente sí. Como a simple vista no se nota la diferencia entre los dos casos, daría la sensación de que el juego no responde correctamente. Guardando la intención varios ciclos se soluciona el problema.

Coyote Time es como se conoce al tiempo que se le da al jugador para saltar después de caer por un bordillo. Teniendo el nombre origen en "El Coyote y el Correcaminos", permite al jugador saltar una vez que se encuentra fuera de la plataforma, dando una sensación similar al caso anterior. En la figura 7.2 se puede ver de forma gráfica el funcionamiento de este método.

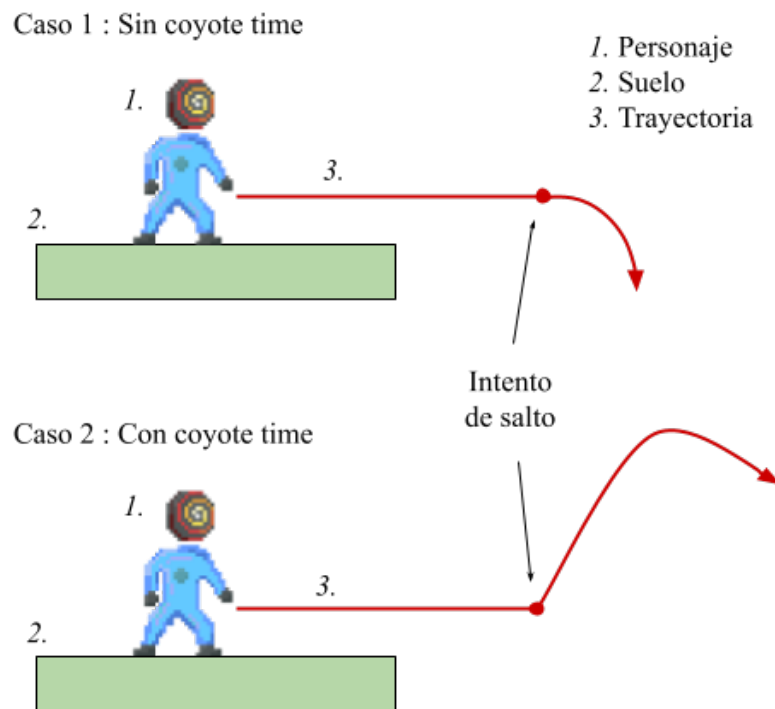


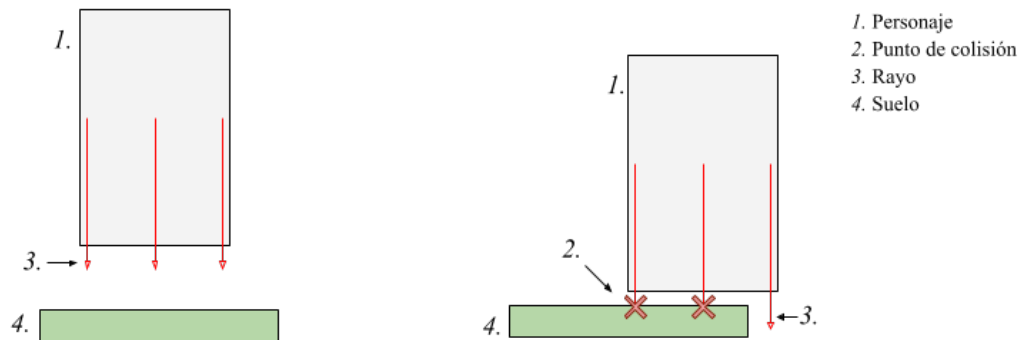
Figura 7.2: Demostración Coyote Time

Una vez se sepa que el personaje va a saltar, se modifica la velocidad vertical de este para que se eleve. Utilizando el componente *Rigidbody 2D*, se puede establecer la nueva velocidad actual, lo cual simula que el personaje se impulsa hacia arriba. Para una mayor precisión del salto, si el jugador deja de pulsar el botón de saltar, la velocidad vertical del personaje se establece a 0, deteniendo el salto.

Para detectar el contacto con el suelo se hace uso de *raycast*. Un raycast consiste en lanzar un rayo, el cual es una línea de longitud finita desde un punto en una dirección [9]. En caso de que el rayo colisione con algún objeto, se puede obtener información de ese objeto. En este caso, se lanzan tres rayos desde el personaje hacia abajo y si colisionan contra el suelo, se considera que el personaje se encuentra en el suelo. Ver figura 7.3. En el editor de Unity, se dibujan los rayos para facilitar el desarrollo.

Caso 1 : Ningún rayo colisiona con el suelo

Caso 2 : Al menos un rayo colisiona con el suelo



Se considera que el personaje está en el **aire**

Se considera que el personaje está en el **suelo**

Figura 7.3: Demostración detección de suelo

Aunque saltar es importante, la otra pieza clave del movimiento es el desplazamiento lateral. Para el desplazamiento del personaje, se han probado varias posibilidades. A la hora de definir el funcionamiento de las físicas en Unity, existen varias formas de hacerlo. Generalmente, se pueden dividir en dos, la primera es hacer uso del sistema de físicas de Unity, y la segunda es crear un sistema propio.

Haciendo uso de componentes dados por Unity, como `Rigidbody2D`, un objeto del juego puede ser afectado por fuerzas como la gravedad, fricción o colisiones con otros elementos. El beneficio de usar este sistema es su simplicidad, dado que ya viene construido. En el caso de realizar las físicas manualmente, la complejidad es mucho mayor, pero el control sobre estas también aumenta. Para este juego, se busca un alto control sobre las físicas de movimiento, como el desplazamiento, gravedad y fricción, a fin de hacer entretenida la experiencia de mover al personaje.

En un principio, se llevó a cabo la implementación utilizando el sistema de físicas de Unity para controlar eventos como la gravedad, fricción y fuerzas que afectan al personaje y a los enemigos. Una vez llegado a la fase de pruebas, surgieron problemas relacionados con este tipo de movimiento. El problema era que al utilizar el sistema de físicas de Unity, se perdía mucho control sobre las físicas y, por tanto, mucho control sobre el movimiento.

Al final, se ha optado por hacer uso de una combinación de ambas. Utilizando como base las físicas de Unity, se implementan nuevas funciones para gestionar el movimiento de los diferentes elementos del juego, mientras que se hace uso del sistema de colisiones dado por Unity. Para la fricción, se desacelera al personaje siempre y cuando este no esté intentando desplazarse, y la gravedad empuja hacia abajo al personaje sumando un valor negativo a su velocidad vertical. Si el personaje supera unos límites de velocidad establecidos, la velocidad de este se establece a un tope.

Un hecho importante a la hora de mover el personaje es que hay ordenadores más rápidos y más lentos, es decir, hay ordenadores que son capaces de realizar más ciclos, o fotogramas, que otros ordenadores en el mismo intervalo de tiempo. Si por cada ciclo se le suma una distancia fija a la posición del personaje, en un ordenador con mayor ratio de FPS y, por tanto, más ciclos por segundo, el personaje se movería más rápido. Por este motivo los cálculos que se realizan son proporcionales a la tasa de refresco.

7.1.2. Movimiento de la cámara

Un elemento clave para el juego es la cámara, un objeto que graba el escenario para que el jugador pueda verlo. Este objeto contiene un componente especial, *Camera*, el cual posee funciones para modificar aquello que se ve en la pantalla.

Para este proyecto, la cámara seguirá al personaje, pero para que dé una buena sensación, se implementará un pequeño retraso entre el personaje y la cámara. El efecto que se quiere conseguir con el retraso es que dé una sensación de velocidad. Si el personaje acelera rápido y la cámara tarda un poco en alcanzarlo, el jugador sentirá que el personaje se está moviendo rápidamente.

La primera versión consistía en mover la cámara a la posición exacta del personaje. Esta se descartó dado que no cumple el efecto de retraso.

En la segunda versión, la cámara se movía a una velocidad constante siguiendo al personaje, dando un efecto de retraso, pero existiendo la posibilidad de que el jugador decida alejarse tanto que el personaje no aparezca en pantalla.

Como solución, en la tercera versión la cámara se movía a diferentes velocidades dependiendo de la distancia al personaje. Se puede representar esto como tres áreas alrededor del personaje. En la primera y más cercana, la cámara no se movía. En la segunda, la cámara se movía a una velocidad similar a la del personaje. En la tercera y la más exterior, la cámara se movía rápidamente para alcanzar al personaje porque se está saliendo del área de visión.

A pesar de conseguir un efecto similar al deseado, la sensación al jugar no es la buscada, así que se han probado más formas de arreglarlo. Al final, la función `SmoothDamp()` de la clase `Vector2`, una clase de Unity, da los resultados buscados.

Esta función recibe como parámetros la posición actual, destino, la velocidad actual a la que el objeto se está desplazando y cuánto tiempo se quiere tardar en alcanzar el destino. En función de eso, devuelve la posición en la que el objeto debe

estar en el siguiente fotograma. Esta función se ejecuta en `LateUpdate()`, la cual se ejecuta después de `Update()`. Esto se hace para asegurar que el personaje ha calculado su nueva posición antes de que la cámara intente alcanzarlo.

7.1.3. Tileset

Un *tileset*, literalmente traducido como conjunto de baldosas, es un conjunto de elementos con los que se forman los escenarios en un videojuego. En videojuegos 2D, este suele ser una imagen separada en diferentes cuadrículas, apareciendo en cada cuadrícula una baldosa. Una vez separadas, se pueden reordenar para crear formas y escenarios complejos. En la figura 7.4 se puede ver una representación de cómo se separan los *tiles* y luego un ejemplo de su unión.

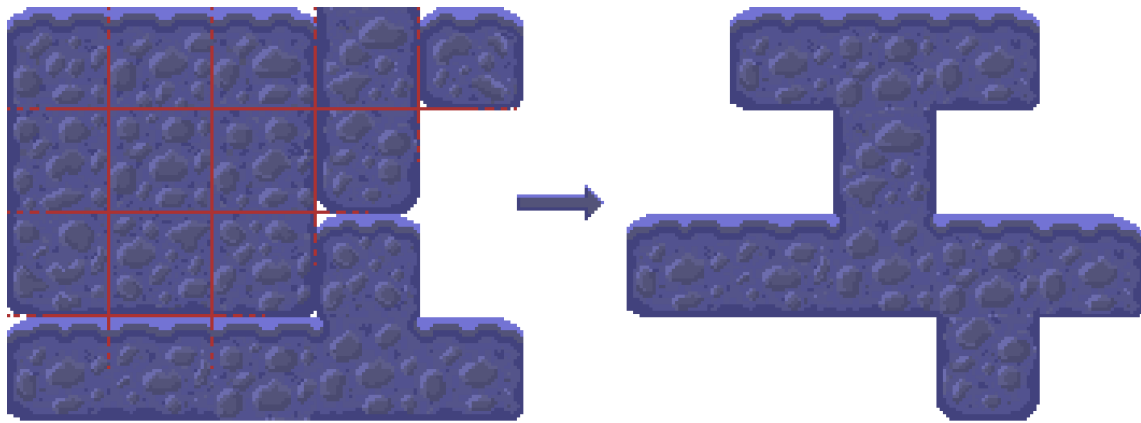


Figura 7.4: Demostración *tileset*

Para implementar los *tilemaps* se utiliza una *Rule Tile*, que permite crear una baldosa, la cual, dependiendo de su entorno, cambia. Este tipo de baldosas programables permite establecer las condiciones con las que aparecerá una baldosa u otra y de esa forma poder crear escenarios rápidamente. Gracias a la guía del paquete [10] se ha configurado sin complicaciones. A estas baldosas se les pueden añadir colisiones, y de esa forma obtener un suelo y paredes por donde se desplazará el personaje. Se pueden combinar diferentes *tilesets* con distintas propiedades y así conseguir escenarios y niveles variados como aquel mostrado en la figura 7.5 el cual combina dos *tilesets*, uno para las plantas y otro para los bloques.

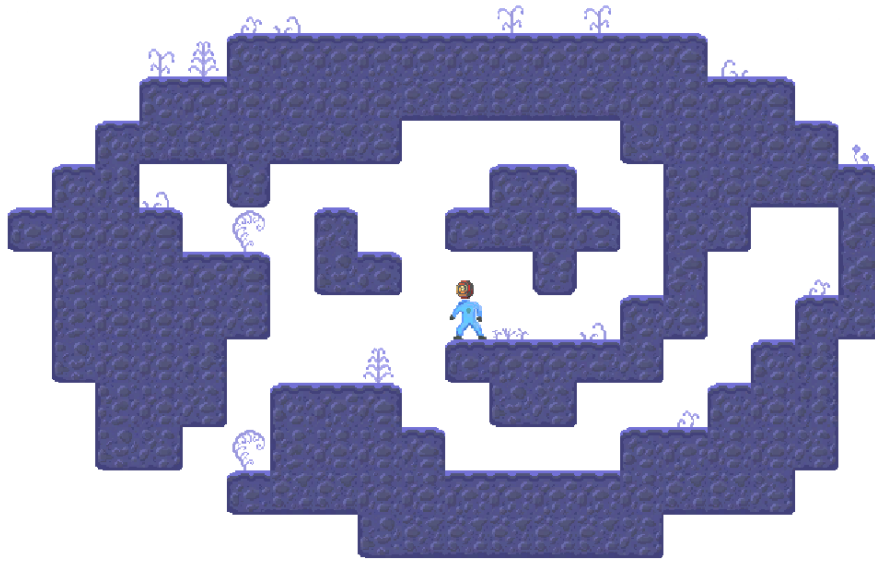


Figura 7.5: Escenario creado utilizando dos *tilesets*

7.1.4. Enemigos

Los enemigos son parte crucial del juego, son el elemento que aporta un desafío a este y lo hace divertido de jugar. Estos seres tienen diferentes comportamientos, pero de una forma u otra pueden suponer un peligro para el jugador. Como primer enemigo se ha implementado un escarabajo o cucaracha, y el funcionamiento de este se generalizará al resto de enemigos en próximas iteraciones. Hay que tener en cuenta que a medida que avance el desarrollo, el funcionamiento de estos puede variar, por lo que aquello explicado a continuación puede no ser el resultado final.

Estos seres también serán `GameObjects` con componentes similares a los del personaje. La única diferencia son los *scripts* que los controlan. Para que puedan interactuar con el jugador, tendrán sus propias colisiones.

El comportamiento de un enemigo es simple: pasea o no hace nada hasta que ve al personaje, entonces le ataca. Más adelante se profundiza en la “visión” de los enemigos. Al acabar el ataque vuelve a esperar y si el jugador le ataca cierto número de veces, este será derrotado. El comportamiento de los enemigos está definido por una máquina de estados. Se han establecido 4 estados:

- **0. Inactivo:** El enemigo está quieto, esperando a ver al personaje.
- **1. Moviéndose:** El enemigo está en movimiento, esperando a ver al personaje.
- **2. Atacando:** El enemigo está en proceso de saltar contra el personaje.
- **3. Derrotado:** El enemigo ha sido derrotado.

En cada uno de los cuatro estados, el enemigo lleva a cabo diferentes acciones y reacciona de forma diferente dependiendo del estímulo. En cada *frame*, dependiendo del estado en el que se encuentra el enemigo, se ejecuta una función, aunque hay funciones generales para todos los estados, como puede ser la detección del suelo o la simulación de la gravedad. En la figura 7.6 se puede ver la máquina de estados del enemigo. A continuación se explicará en detalle cada estado.

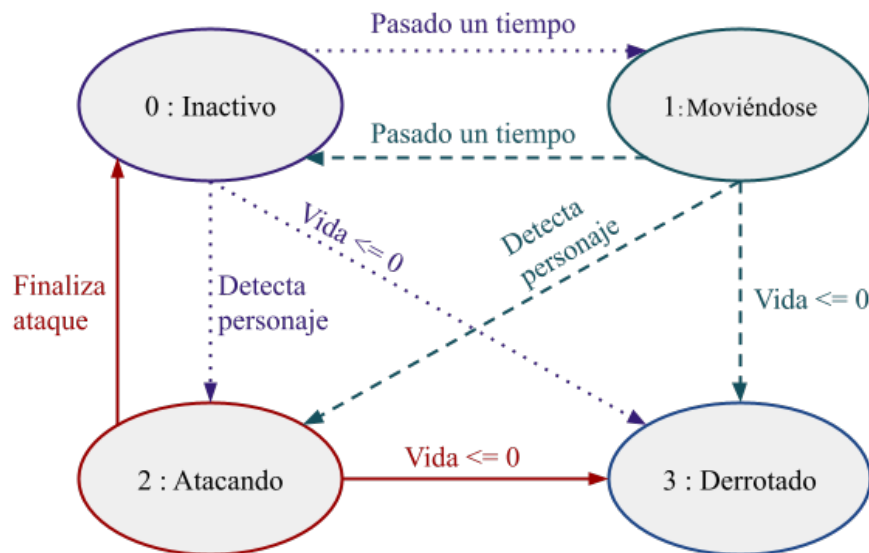


Figura 7.6: Máquina de estados para un enemigo

Estado 0, Inactivo. Cuando el enemigo está inactivo, no se desplaza. Mediante un RayCast detecta si tiene al personaje delante. De ser así, pasa al estado 2, Atacando. Si pasa un tiempo sin detectar nada, pasa al estado 1, Moviéndose, y comienza a desplazarse. Los enemigos guardan un valor que representa la “vida” que tienen, siendo este un valor que disminuye al recibir ataques del jugador. Durante este estado, el enemigo es vulnerable a los ataques y si su vida se baja a 0 pasará al estado derrotado.

Estado 1, Moviéndose. Este estado es similar al 0, con la diferencia de que mientras esté en este estado, el enemigo se desplaza. El funcionamiento del desplazamiento se lleva a cabo de la misma forma que el del personaje. Para dar algo de autonomía al enemigo y que este pueda moverse por el escenario, se han desarrollado unos sensores básicos. Estos son dos, detención de paredes y de bordillos, para dar la vuelta en caso de chocar contra una pared o estar a punto de caer de una plataforma respectivamente. En el grupo de los sensores se puede incluir también la capacidad de detectar al jugador. En la figura 7.7, se pueden ver los rayos que lanza el enemigo, cada uno con una función específica. En el estado de la imagen, se da el caso en el que el enemigo cambiaría de dirección, dado que de seguir caminando, caería de la plataforma.

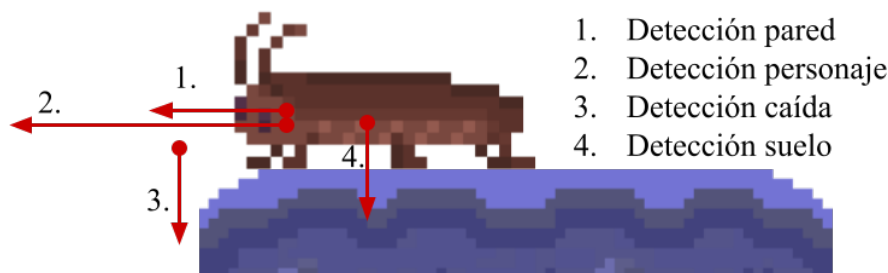


Figura 7.7: Sensores de un enemigo

Estado 2, Atacando. Si el enemigo se encuentra en este estado, está en proceso de atacar al personaje. Al inicializar este estado, se le da un impulso al enemigo para que salte en la dirección en la que se encuentra el personaje. Una vez esté en el aire, se pueden dar dos casos para cambiar de estado. El primero de ellos aterrizar, una vez detectado que está en el suelo, pasa al estado inactivo. El segundo caso es ser derrotado, dado que en este estado puede recibir ataques.

Estado 3, Derrotado. Una vez la vida del enemigo baje a 0, este pasará a estar derrotado. En este estado se borra el `GameObject` del enemigo, haciendo que desaparezca de la escena. Por este motivo, el estado derrotado no tiene implementación más allá de su inicialización. Antes de borrar el `GameObject` se hace una llamada a la clase `GameController` para avisar de su derrota. Esto será relevante más adelante.

Se ha hablado de la posibilidad de que el enemigo reciba daño por parte del personaje, pero no de su funcionalidad. Se considera que un enemigo ha recibido un ataque cuando este entra en contacto con un objeto que esté etiquetado como “arma”. Este contacto se lleva a cabo mediante colisiones, pero a diferencia de las colisiones usadas para el suelo, estas no impactan físicamente al enemigo, simplemente generan un aviso. La misma funcionalidad se usa cuando un enemigo golpea al personaje. Se profundizará este tema en el apartado de combate 7.1.6.

Llegados a este punto, toca introducir un nuevo concepto que se usará a lo largo del desarrollo, los *prefabs*. El enemigo que se ha creado es un único `GameObject`, pero durante una partida tiene que haber varios de este tipo de enemigo. Una solución es copiar y pegar el enemigo, creando objetos individuales. Esta no es una solución óptima porque se quiere poder instanciar los objetos mediante código y de realizarse un cambio en el enemigo, habría que cambiar una por una cada copia.

Los *prefabs* permiten convertir un `GameObject` en una plantilla, permitiendo generar copias del enemigo y colocarlas en el escenario, pero que sigan relacionadas con el original. Cualquier cambio que se haga a la plantilla original se verá reflejado en las copias, pero no a la inversa. Estas plantillas también permiten instanciar nue-

vos objetos durante la ejecución, pudiendo colocar enemigos en el escenario mediante código.

7.1.5. Armas

Teniendo enemigos y un personaje, su interacción es parte crucial del juego, siendo en este caso la interacción mediante combates. Dado que la meta del jugador es acabar con los enemigos y viceversa, se ha de implementar una forma de que el personaje pueda derrotar enemigos y de que estos puedan hacer lo mismo. Para ello, el jugador tiene que poder realizar ataques.

Los ataques son realizados con armas y estas pueden variar desde espadas hasta varitas mágicas. Dado que se busca tener una gran variedad de armas, la implementación tiene que ser flexible para permitirlo.

Un ataque se representa en el juego como una área en la cual los enemigos sufren daño. Esta área puede ser estática, como un corte de una espada, o se puede desplazar, como una flecha. En un principio se ha ideado y probado crear implementaciones individuales para cada arma. Con esto se refiere a tener una plantilla para un ataque de espada, otra de lanza, otra para una flecha... El problema de esta solución es que las armas posibles se ven limitadas por el número de plantillas y querer realizar un cambio que afecte a todas las armas conllevaría modificar cada implementación.

Esta forma de implementarlo tendría sentido si se ve desde el punto de vista de armas reales, porque una espada es claramente diferente a una flecha. Dado que esto es un videojuego, la definición de un ataque puede ser mucho más abstracta. Como solución, se plantean todos los tipos de armas como proyectiles. El ataque de una espada es un proyectil ancho pero que no se desplaza, una lanza es un proyectil más largo que tampoco se desplaza. Una flecha es un proyectil pequeño que se desplaza rápidamente y llegados a este punto, no hay mucha diferencia entre una flecha y una bola de fuego disparada mágicamente. Las formas de los proyectiles pueden ser representados mediante un `BoxCollider2D`. Aunque no tengan todos una forma rectangular, es una forma que se aproxima lo suficiente.

Los ataques por parte del jugador se dividen principalmente en dos *scripts*. Uno asociado al personaje y otro al ataque. Cuando el jugador quiera atacar, puede escoger 8 direcciones, ver figura 7.8. Se puede atacar en horizontal, vertical o en diagonal, dependiendo de las teclas presionadas. ‘j’ y ‘l’ para horizontal, ‘i’ y ‘k’ para vertical y combinaciones de estas teclas para las diagonales. Desde Combate-Jugador, la clase y *script* asociada al personaje, se recibe el *input* y se establece la dirección del ataque.

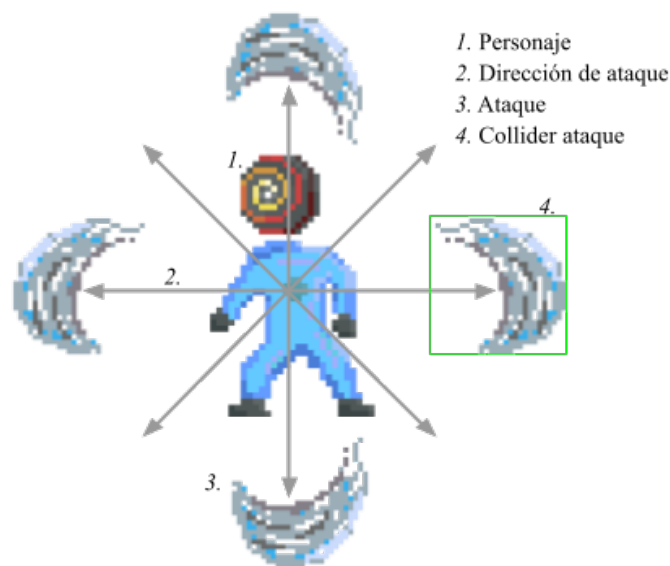


Figura 7.8: Posibles direcciones de ataques

Cuando se intenta atacar, se guarda la intención del jugador de atacar durante un corto periodo de tiempo, al igual que al saltar. Un caso en el que el jugador no podría atacar es durante el tiempo de enfriamiento o *cooldown*. Este es el periodo de tiempo mínimo que tiene que haber entre ataques, para que el jugador no pueda atacar tan rápido como quiera. El valor de este dependerá del ataque. Un ataque puede ser convertido a un *prefab* y de esa forma se crea un nuevo objeto de ataque cuando el personaje ataque. Habiendo recogido la dirección, se instancia el objeto del ataque y se le asigna la dirección.

Para darle el valor de dirección, se obtiene el *script* que controla al proyectil y se usa la función `InicializarDireccion(dirección)`. Usando el mismo *script* se obtiene la duración del tiempo de recarga. Aquí se encuentra uno de los mayores problemas surgidos. Para acceder al *script* que lo controla y llamar a una función de este, es necesario saber el tipo de *script* del proyectil. Como solución, se hace uso de la herencia. Se ha definido una clase abstracta `AbstractAtaque` de la que heredarán todos los tipos de ataque. Dado que los diferentes tipos de ataques serán similares entre sí, gracias a la herencia podrán compartir métodos. Esto se desarrollará en la segunda iteración.

En el *script* `AbstractAtaque` se definen varios métodos útiles para los diferentes ataques. Por ejemplo, `InicializarDireccion()` modifica la rotación del objeto para que se alinee con la dirección recibida y `ColisionPared()` devuelve `True` si el ataque impacta contra el suelo, pared o techo. Para detectar la colisión de un proyectil con el escenario se hace uso de un raycast y no de las cajas de colisiones. De esta forma, los ataques solo se verán detenidos si tocan el escenario con el centro del proyectil. Así, si un ataque cubre un área grande, no daría problemas por rozar el suelo.

A la hora de inicializar la dirección de los ataques, se lleva a cabo una importante acción, normalizar el vector de dirección. Normalizar un vector consiste en cambiar su longitud a 1 manteniendo su ángulo. De no normalizarse los vectores de dirección, los ataques diagonales tendrían un vector de longitud mayor y, por tanto, al multiplicar la velocidad por la dirección, estos se moverían más rápido de lo normal. En videojuegos antiguos esta práctica no se llevaba a cabo y, como consecuencia, hay múltiples juegos en los que moverse diagonalmente es más rápido que moverse de frente.¹

Para la primera arma implementada se usará una espada. `AtaqueEspada` es hijo de `AsbtractAtaque` y controla un proyectil que se desplaza un poco. Este ataque desaparece por sí solo al de 0.15 segundos, por lo que es necesario tener un temporizador que compruebe cuánto tiempo lleva existiendo.

7.1.6. Combate

Una vez implementada la forma en la que los enemigos y el personaje atacan, queda implementar cómo se detectan y reciben los ataques. Dado que la implementación de recibir ataques se hace de forma similar en los enemigos y en el personaje, se explicarán a la vez.

Se ha mencionado que los enemigos tienen un valor que representa su vida o vitalidad y este mismo caso se da para todas las entidades. Este valor podrá incrementar o decrementar durante la partida, a consecuencia de ciertos eventos.

El caso que se estudia a continuación es la consecuencia de un ataque. Cuando una entidad recibe un ataque, su vida decrementa en función del ataque. Si la vida pasa a ser 0 o menor, la entidad será derrotada, de no ser así, entra en un periodo de invulnerabilidad. En este periodo de tiempo, la entidad no podrá recibir daño. Esto soluciona varios problemas. Por ejemplo, si el personaje toca un enemigo, recibe daño. El problema es que si por cada fotograma en el que están en contacto el personaje sufre, en un segundo podría llegar a recibir el equivalente a 60 ataques o más, dependiendo de los FPS actuales. Por parte de jugabilidad, esto no es divertido ni justo.

Para detectar el contacto entre el personaje y enemigos se utilizan colisiones. A diferencia de los *collider* utilizados para definir el espacio físico que ocupa un `GameObjet` y así aplicar las físicas de Unity, estos *collider* son de tipo *trigger* o gatillo, lo que significa que envían una señal al entrar en contacto con otros objetos. `OnTriggerEnter2D` es la función a la cual llama el motor cuando ocurre una colisión de este tipo. Existen variaciones de este método dependiendo del momento

¹<https://tvtropes.org/pmwiki/pmwiki.php/Main/DiagonalSpeedBoost>

de la colisión, inicio, durante y final. Cuando el personaje entra en contacto con un enemigo, llama a la función `ColisionEnemigo()`. Para saber si un objeto es un enemigo, se hace uso de la etiqueta asociada al objeto.

Cuando el personaje es golpeado, se comprueba si se encuentra en un periodo de invulnerabilidad y, de no ser así, se le decrementa el valor de su vida basándose en el daño realizado. En el caso de ser derrotado por este golpe, se avisa al controlador del juego para que acabe la partida.

Para obtener el daño realizado por un objeto, se accede al *script* que lo controla y se llama a una función de este. Al igual que con los diferentes ataques, cada enemigo cuenta con un *script* diferente. Como solución, se hace uso de una interfaz. La interfaz `IControladorEnemigo` declara el método `GetDano()`, de esta forma se puede asegurar que, si es un enemigo, poseerá el método `GetDano()`.

Para dar retroalimentación cuando un enemigo recibe un ataque, se le aplica un ligero impulso en la dirección del ataque. Más adelante esto irá acompañado de otros efectos visuales y sonoros.

7.1.7. HUD

Mientras el jugador juega, hay cierta información que es necesaria que sepa. En este caso, la vida del personaje y el número de enemigos restantes son dos datos a destacar. Un HUD o Head-Up Display consta de pequeños gráficos repartidos por la pantalla que muestran al jugador esa información.

Para llevar a cabo la implementación en Unity se hace uso del objeto *canvas*, el cual permite superponer elementos gráficos sobre el juego. Dado que en esta iteración se realiza una versión simplificada del HUD, los datos se mostrarán mediante textos en las esquinas superiores de la pantalla. La figura 7.9 es una captura de pantalla del juego donde se puede ver el HUD.



Figura 7.9: Captura de pantalla del juego

7.1.8. Controlador del juego

Ante la necesidad de conectar los diferentes apartados del juego, se crea el controlador del juego, un objeto que ejecuta el *script* GameController. Dentro de este *script* se define la lógica del nivel. Esta lógica está compuesta por ciertas normas para cumplir los requerimientos de la primera iteración:

- Si el personaje actualiza su valor de vida, actualiza el contador en pantalla.
- Si el personaje es derrotado, reinicia el juego.
- Si un enemigo muere, actualiza el contador en pantalla.
- Si todos los enemigos mueren, activa una salida para el jugador.
- Si el personaje atraviesa la salida, se le felicita.

La salida es un objeto con un Collider de tipo Trigger, el cual, al detectar una colisión con el personaje, manda un aviso al controlador para que finalice el juego.

7.1.9. Apartado artístico

Para esta iteración es necesario que los elementos sean reconocibles, por lo que necesitan tener diferentes *sprites*. Haciendo uso de la herramienta Aseprite se han creado imágenes con una resolución de 32x32 para representar los objetos.

En un principio se hizo con 16x16, pero quedaba muy limitado y se han tenido que modificar los *sprites* a la nueva dimensionalidad. 7.10

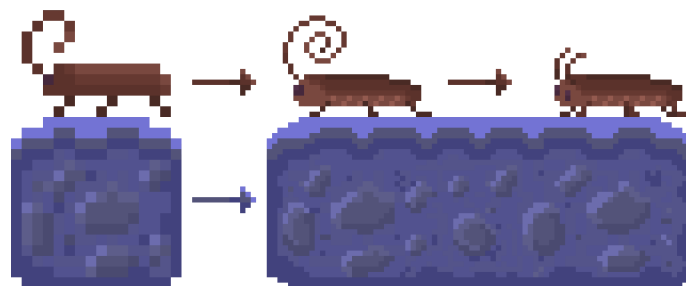


Figura 7.10: Evolución de los *sprites*

7.1.10. Pruebas

Al acabar la implementación, se ha realizado una encuesta entre amigos y familiares con el fin de obtener opiniones sobre el estado actual del juego. Dado que muchas mecánicas del juego sufrirán cambios a lo largo del desarrollo, esta encuesta sirve para encaminar el proyecto y ver los puntos donde el juego carece actualmente.

En la encuesta se realizan varias preguntas, las cuales se pueden puntuar del 1 al 5. Se han recibido 4 respuestas y estas son las puntuaciones medias:

- ¿Consideras que el personaje se mueve acorde a los controles?: 4.5
- ¿La disposición de teclas se hace amena para jugar?: 4.25
- ¿Cómo consideras la dificultad del juego?: 2
- ¿Tienes experiencia jugando a videojuegos similares?: 4
- ¿Consideras la experiencia de juego disfrutable?: 2.75

Las respuestas se alinean con lo esperado, siendo la dificultad y la disfrutabilidad bajas por la falta de contenido del juego. Hay que tener en cuenta que los encuestados pueden no ser completamente imparciales, por lo que los resultados pueden ser mayores a lo que valoraría el público final. A medida que se avance en el desarrollo, se espera mantener o subir esos valores.

Además, se incluyen preguntas de respuesta abierta por si la persona encuestada quisiese dar algún comentario. Como resumen de estas respuestas se destacan la incomodidad de la cámara y las físicas de los enemigos y jugador. La solución a estos dos problemas ha sido explicada durante sus respectivos apartados.

7.2. Iteración 2

Teniendo una base sobre la que trabajar, en la segunda iteración el objetivo es refinar los apartados existentes y añadir variedad.

Requisitos de la iteración

- Aspecto jugable
 - Añadir al menos 3 armas.
 - Poder equipar, desequipar y cambiar de arma.
 - Añadir al menos 3 enemigos.
 - Añadir botín.
 - Añadir accesorios a las armas.
- Aspecto visual
 - Añadir animaciones a los elementos del juego.

7.2.1. Composición

Durante la segunda iteración, se ha llegado a la conclusión de que la estructura de código utilizada hasta el momento era insostenible. En un videojuego hay muchas funciones que se repiten en diferentes elementos y una buena gestión es importante para evitar repetir código innecesariamente.

Como solución, se ha hecho uso de una técnica de diseño, la composición. La composición consiste en definir bloques individuales de funcionalidad que pueden ser agrupados para crear funcionalidades más complejas. Para implementar la composición, se han seguido los pasos indicados en una guía de Gamedevbeginner [\[11\]](#).

Un ejemplo simple es la detección del suelo. Esta se usa en el personaje, los enemigos y más adelante en el botín, pero cada uno de estos elementos posee su propia definición de esta función. En la composición, se crea un componente, en este caso un “detector de suelo”, el cual puede ser usado por cualquier elemento para saber si está tocando el suelo. En la introducción a Unity se ha mencionado cómo los objetos ganan funcionalidad mediante componentes y es esa misma forma de estructurar el funcionamiento la que se ha implementado.

Dado que se ha llegado a esta conclusión a mitad de la iteración, se explicarán a continuación algunos cambios realizados sobre la primera iteración. Más adelante se explicarán las novedades en sus propios apartados.

Durante este proceso se han repasado las funcionalidades existentes y se han reducido a componentes. En caso de encontrar una funcionalidad repetida, se ha modificado el componente creado para que se pueda usar en múltiples casos.

Gracias a esto se puede ahorrar mucho trabajo en caso de querer cambiar algo. Por ejemplo, en la primera iteración se cambió la forma de mover al personaje y enemigos. En su momento se tuvo que cambiar individualmente cada uno, ahora únicamente habría que cambiar los componentes asociados a la física del juego.

También soluciona el problema de la vida y cómo detectar el daño. En vez de que los enemigos tengan que implementar una interfaz para asegurarnos de que tengan la función adecuada, se ha creado un componente de estado. Este componente también lo usa el personaje e indica el estado de un objeto. En conjunto con el componente Vida, gestionan la vida e invulnerabilidad de un objeto. Con estos cambios, cuando un ataque detecta que ha entrado en contacto con un objeto, busca el componente Estado. Si tiene el componente, intenta hacerle daño. Si no tiene el componente, significa que se trata de un objeto al que no se le puede hacer daño.

En la figura 7.11 se puede ver cómo cada tipo de objeto utiliza una combinación diferente de componentes para realizar sus funciones. En la imagen se muestran algunos de los componentes de tres categorías generales, siendo estas las categorías física, sensorial y de estado. La bola azul vista en la imagen es la representación gráfica del botón que se explicará más adelante.

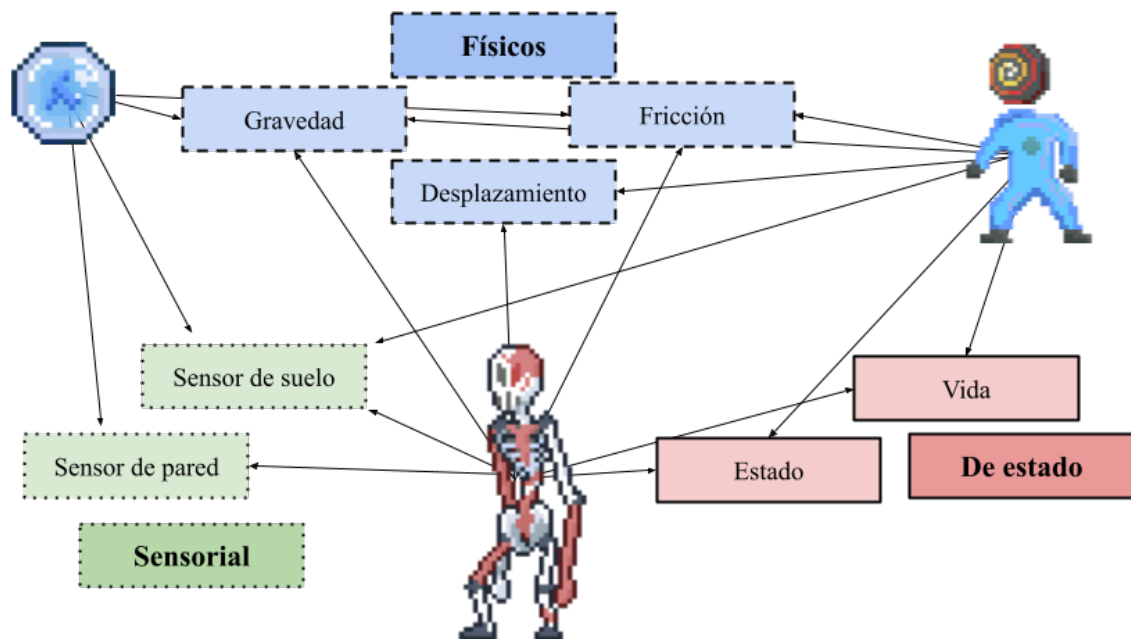


Figura 7.11: Diferentes objetos utilizando diferentes componentes

7.2.2. Armas

Para tener variedad entre partidas, se han implementado diferentes armas. Como se ha definido previamente, un arma consiste en la propia arma y el proyectil que esta dispara.

Se han implementado 5 armas, tres de cuerpo a cuerpo y dos a distancia. Para que cada una funcione diferente, lo que se modifica es el comportamiento de sus proyectiles. Al pasar de herencia a composición, se han eliminado las subclases de proyectil y los diferentes métodos se han pasado a componentes. Se ha decidido nombrar estos componentes como modificadores y se dividen en grupos dependiendo de su función. Actualmente hay cuatro grupos:

- **Modificador de posición:** Definen cómo se desplaza el proyectil.
- **Modificador gráfico:** Afectan al apartado visual.
- **Modificador condicional:** Definen las condiciones de destrucción de un proyectil.
- **Modificador de destrucción:** Establecen qué pasa al destruir el proyectil.

Combinando diferentes modificadores se pueden obtener diferentes tipos de proyectiles. En la figura 7.12 se puede ver cómo para crear diferentes comportamientos se les asigna diferentes modificadores. Por ejemplo, para el crear el boomerang se combinan funciones sinusoidales para que vuelva a la posición inicial. Además, la imagen se va rotando para que se parezca más a un boomerang real.

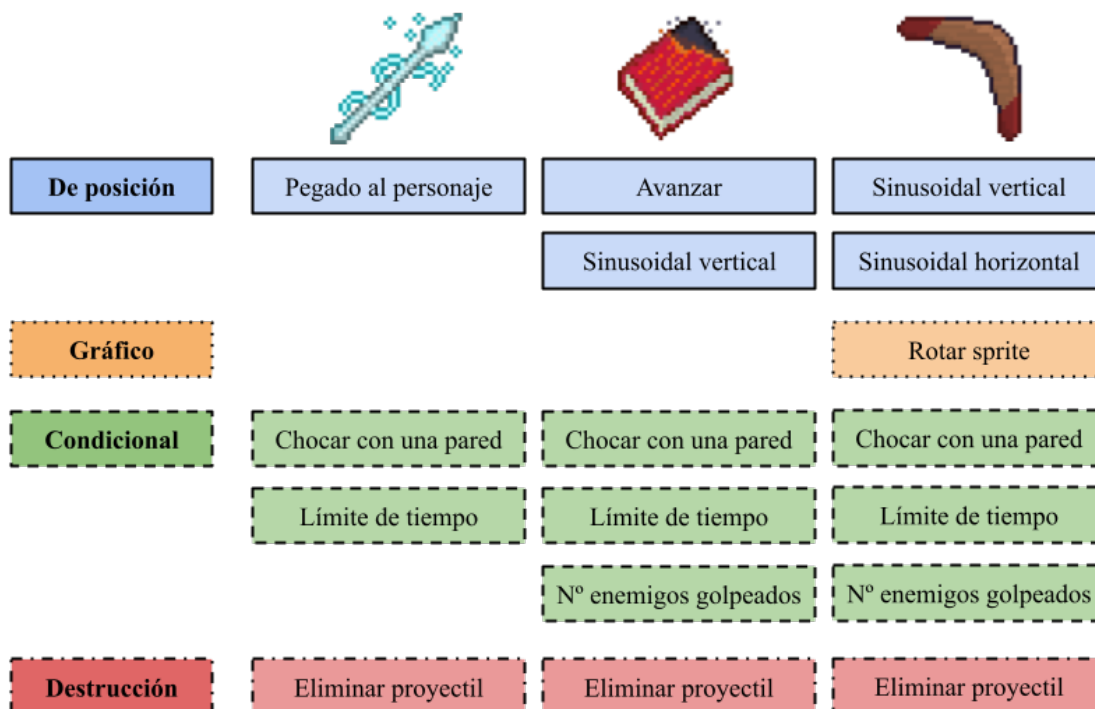


Figura 7.12: Diferentes proyectiles utilizando diferentes modificadores

7.2.2.1. Object Pooling

Cada vez que el jugador usa un arma, se crea un proyectil y cuando este cumple su función, se destruye. Generar y destruir objetos constantemente no es eficiente, por lo que se hace uso de la técnica llamada *Object Pooling* [12]. En esta técnica, en vez de crear y destruir constantemente objetos nuevos, se instancian los objetos una vez y luego se activan y desactivan a necesidad. Los proyectiles se guardan en una lista y cuando es necesario obtener un proyectil, se busca uno desactivado de la lista.

Generalmente, al usar esta técnica se genera inicialmente una cantidad fija de proyectiles para luego ser usados. Dado que el jugador puede decidir coger un arma del suelo y luego volver a soltarla, no tiene sentido crear múltiples proyectiles desde el principio dado que se desperdiciarían recursos en generar unos proyectiles que quizás no se usen. Como solución, solo se crean proyectiles si se intenta usar un proyectil y no queda ningún objeto desactivado en la lista.

7.2.2.2. Accesorios

Los accesorios son complementos que van asociados a las armas. Estos accesorios aportan beneficios para el jugador. Por ejemplo, se ha creado un accesorio que aumenta la velocidad a la que se desplaza el personaje, o que aumenta la vida máxima de este. Para que un accesorio esté activo, es decir, que aplique el beneficio, el arma al que está asociado tiene que estar en reserva. En el apartado 7.2.3 se explica en profundidad cómo se gestiona el equipamiento, por el momento, solo es necesario saber que el personaje puede tener dos armas, una en uso y otra en reserva.

Para poder aplicar los beneficios, se accede al componente en cuestión, ya sea el componente “desplazamiento” para la velocidad o “vida” para la vida máxima y se modifican. Los accesorios se han implementado como componentes, de esta forma, a la hora de generar un arma se le pueden asociar tantos accesorios como se quiera y el propio arma lo gestiona.

Los accesorios tienen un valor llamado “potencia” que representa si el beneficio dado es mayor o menor. Un accesorio de aumento de velocidad de potencia 1 aumenta la velocidad en un 10 % y un accesorio de potencia 2 un 20 %. Cada accesorio tiene una función con la cual calcula el beneficio basándose en la potencia. En el caso de la velocidad la función es:

$$beneficio = 10\% \times potencia$$

Para tratar los beneficios las variables previamente almacenadas mediante *floats*, valores como la vida máxima o la velocidad, han sido sustituidas por una

nueva clase. Esta clase, llamada ValorModificable almacena internamente el valor original, los modificadores aplicados y el valor resultante.

7.2.3. Gestión de equipamiento

A lo largo de una partida, quien juegue encontrará múltiples armas, por lo que una buena gestión de estas es necesaria. El personaje puede llevar consigo 2 armas. Una de ellas, la principal, es la que está en uso y la que genera los proyectiles al atacar. La otra, la secundaria, está en reserva y aporta los beneficios de sus accesorios al jugador. El personaje puede tener como mínimo un arma, la cual es considerada como principal.

Las armas se gestionan con dos botones. Un botón, actualmente asociado a la tecla 'l', se encarga de cambiar de arma y otro botón, la tecla 'i', se encarga de soltar y recoger armas.

Cuando se cambia de arma, la principal pasa a estar en reserva y la secundaria pasa a estar en uso. Al hacer el cambio se activan los beneficios de una y se desactivan de la otra.

Al pulsar el botón de soltar y recoger armas, se pueden dar diferentes casos dependiendo de dos factores: el número de armas que porte el personaje y si hay algún arma en forma de botín alrededor del personaje.

Si solo tiene un arma y no hay botín, no ocurre nada. Si tiene dos armas y no hay botín, se suelta el arma principal y la secundaria pasa a ser la nueva principal. Al desechar un arma se genera un orbe de botín. En el apartado 7.2.4 se ahonda más en el funcionamiento del botín.

Si tiene un arma y hay botín cerca, recoge el botín y lo equipa como arma principal. Si tiene dos armas y hay botín cerca, suelta el arma principal y recoge el botín.

Previamente, las acciones de recoger armas y soltarlas se llevaban a cabo con dos botones diferentes, pero no era cómodo ni intuitivo de usar.

7.2.4. Botín

Al jugar un videojuego, una forma común de recompensar a los jugadores y ayudar en su progreso es el botín o *loot*. En este caso, el botín son las armas que el jugador puede conseguir derrotando enemigos.

Cuando un enemigo es derrotado, este hace una llamada al gestor de botín el cual se encarga de generar una recompensa en caso de ser necesario. Los enemigos tienen asociado un nivel de dificultad y una probabilidad de generar *loot* al ser derrotados. Basándose en esa probabilidad, el generador de botín decidirá aleatoriamente generar *loot* y en caso de querer generarlo, el arma se decidirá aleatoriamente entre todas las armas posibles. Cuanto mayor sea la dificultad del enemigo, el gestor añadirá al arma más accesorios con mayor potencia.

Al generar un orbe de botín, se le aplica un impulso hacia arriba y hacia un lado. Dado que tiene añadido el componente de gravedad, baja hacia el suelo y cuando se detecta que el orbe va a tocar el suelo utilizando el componente “sensor de suelo”, se le aplica otro impulso hacia arriba pero esta vez de menor fuerza. Esto se repite dos veces para simular que el orbe rebota al caer al suelo.

Cuando un orbe de botín es generado, se le asocia la imagen del arma que contiene además de la propia arma. Cuando el personaje se acerca a un orbe, este se eleva ligeramente de la superficie y muestra la imagen del arma que contiene. En caso de haber varios orbes, únicamente se eleva aquel más cercano al personaje. De esta forma, el jugador puede saber cuál de los orbes a su alrededor recogería en caso de pulsar el botón de recoger.

Para no tener que calcular constantemente la distancia del personaje a todos los orbes, el personaje tiene asociado un círculo a su alrededor que representa el rango a partir del cual puede recoger el botín. Cuando un orbe entra en contacto con el círculo, se añade a una lista. En esa lista están todos los orbes cercanos al jugador y se calcula cuál es el más cercano de la lista al jugador para que este pueda elevarse. En la figura 7.13 se pueden ver los diferentes estados en los que puede estar un orbe de botín.

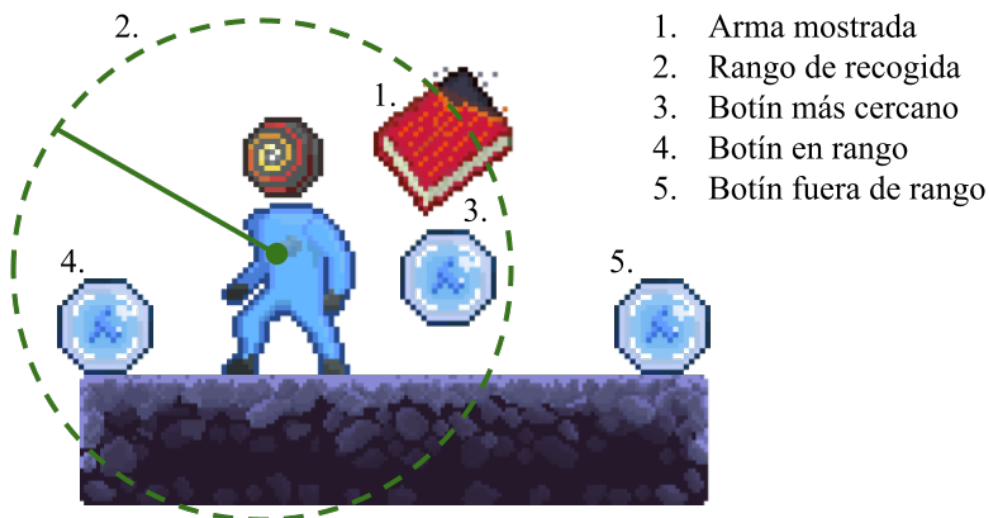


Figura 7.13: Rango de recogida de botín

7.2.5. Enemigos

En la primera iteración se creó una versión básica de un enemigo, la cucaracha. En esta iteración se ha tomado esa base para crear tres enemigos. Una cucaracha, un *slime* y un esqueleto. Estos se pueden ver en la imagen 7.14.



Figura 7.14: Tipos de enemigos

Cada enemigo tiene un comportamiento y formas de atacar diferentes. Gracias a los componentes creados, se pueden reutilizar e intercambiar comportamientos para generar nuevos enemigos. Por ejemplo, el esqueleto y la cucaracha utilizan el mismo componente de desplazamiento que el personaje, pero el *slime* utiliza uno el cual hace que avance realizando pequeños saltos.

El enemigo base veía lanzando un rayo hacia delante y en esta iteración se ha actualizado el componente de visión de los enemigos. Gracias al nuevo componente, la visión de un enemigo se puede configurar mediante 4 parámetros.

- Longitud: La longitud de los rayos.
- Ángulo de visión: El ángulo del cono que representa la visión del enemigo.
- Número de rayos: Entero impar que representa el número de rayos que forman el cono.
- Atraviesa paredes: Si el enemigo ve a través de las paredes.

Configurando estos parámetros se pueden crear diferentes comportamientos, como puede ser que el esqueleto vea de lejos o que el *slime* vea de cerca, pero en todas las direcciones. En la figura 7.15 se puede ver una representación gráfica.

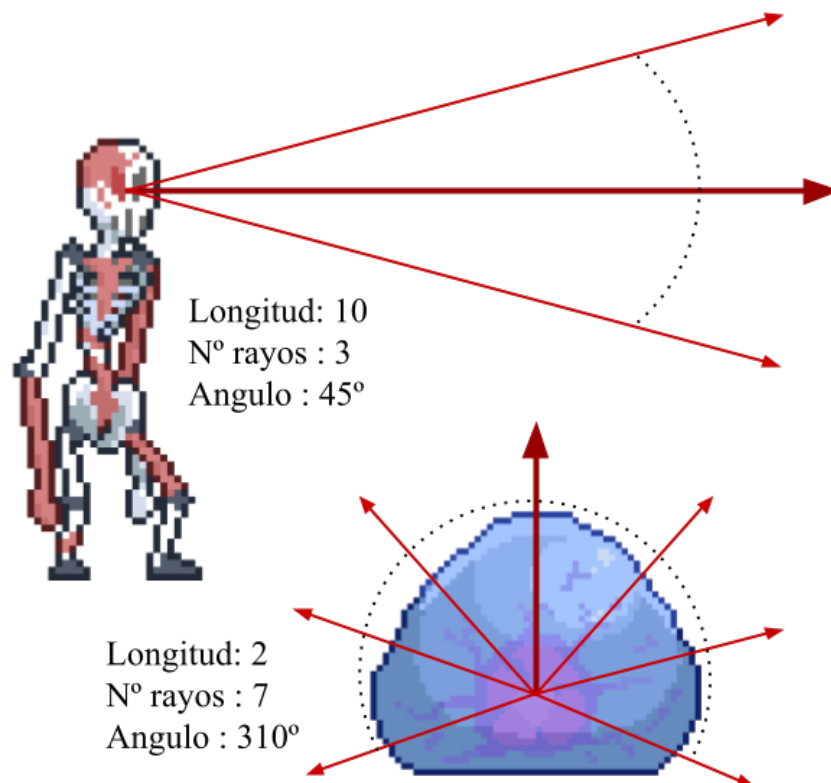


Figura 7.15: Visión del esqueleto y *slime*

A diferencia de la primera iteración, no todos los enemigos atacan directamente cuando ven al personaje. El esqueleto entra en un estado llamado “enfadado” en el que su velocidad aumenta y trata de acercarse al personaje. Si consigue acercarse, entonces inicia el ataque.

Los ataques de los enemigos también han sufrido cambios. Previamente, el contacto con los enemigos causaba daño, pero esa función ha sido eliminada. Ahora los enemigos solo hacen daño cuando están atacando. Estos ataques se han definido como entidades independientes del enemigo. Los ataques son definidos por el área de efecto y su comportamiento. El área de efecto es un *Collider2D* de tipo *trigger* el cual permanece desactivado hasta que el enemigo ataca. Dependiendo del comportamiento se han creado tres versiones:

- Golpe: Activa el área de efecto.
- Placaje: El enemigo salta y activa el área de efecto.
- Embestida: El enemigo avanza y activa el área de efecto.

El esqueleto usa el golpe, la cucaracha el placaje y el *slime* la embestida. Cada uno de estos ataques puede ser configurado de diferentes formas. Por ejemplo, en el placaje se puede definir la dirección del salto y en la embestida si el enemigo deja de avanzar si llega a un borde o si rebota al llegar a una pared.

Cuando se inicia un ataque, este entra en un estado de carga. Este pequeño tiempo antes de realizar el ataque da tiempo al jugador a reaccionar. Cuando acaba el tiempo de carga, comienza el ataque, ya sea saltando o simplemente activando el área de efecto del ataque. En la figura 7.16 se muestran los diferentes *hitbox*, o áreas de colisión, utilizados por cada enemigo.



Figura 7.16: *Collider* de los ataques de enemigos

7.2.6. Animaciones y apartado gráfico

Para esta iteración se han implementado animaciones para el personaje, enemigos, proyectiles y botín. En los videojuegos 2D, las animaciones suelen guardarse en forma de *spritesheet*. Esto es una imagen, la cual contiene una serie de imágenes dentro. En el caso de un *spritesheet* de la animación del personaje corriendo, contiene cada fotograma necesario para formar la animación como se puede ver en la figura 7.17. Unity es capaz de recortar los *spritesheet* para obtener *sprites* individuales.



Figura 7.17: *Spritesheet* del personaje corriendo

Dependiendo del elemento a animar, se han creado diferentes animaciones. En el caso de los proyectiles y orbes de botín, se ha creado una animación que se repite en bucle para cada objeto.

Los enemigos requieren animaciones para cuando están quietos, moviéndose, cargando, atacando y siendo derrotados. El personaje requiere animaciones para caminar, estar quieto, saltar y ser derrotado.

A la hora de hacer las animaciones, se han modificado algunos *sprites* creados en la iteración anterior. Como se ha podido ver en previas imágenes, todos los elementos tienen un borde oscuro. Esto se ha hecho para generar un mayor contraste visual y poder diferenciar fácilmente los elementos del fondo.

Para implementar las animaciones en Unity, se ha usado el propio sistema de animación de Unity siguiendo los pasos de un tutorial de Brackeys [13].

A los objetos con animaciones se les asigna una máquina de estados, la cual define la animación mostrada. En el caso del personaje, las animaciones dependen de si el jugador intenta moverse lateralmente, si el personaje está tocando el suelo y la velocidad vertical del personaje. Con estos atributos se puede saber si el personaje está corriendo, saltando o cayendo y mostrar una animación acorde. En el caso de los enemigos, las animaciones son gestionadas desde sus propios *scripts* de comportamiento.

7.2.7. Pruebas

Durante el desarrollo de la iteración se han implementado múltiples sistemas que interactúan constantemente entre sí. Por este motivo, a medida que se han implementado ciertos aspectos, se han encontrado errores en otros. Además, se han realizado pruebas creando diferentes escenarios donde colocar varios enemigos y estudiar su comportamiento.

En estos escenarios se ha analizado el rendimiento del juego, probando a añadir 100 enemigos de diferentes tipos en una plataforma pequeña. También se ha estudiado cómo reaccionan estos enemigos al personaje en diferentes casos. Por ejemplo, un caso consiste en poner dos plataformas, una con el personaje y otra con los enemigos. En este caso, la cucaracha salta hacia el personaje mientras que el esqueleto y *slime* se mantienen en su plataforma. Esto corresponde con el comportamiento esperado.

En el caso de los orbes de botín, se ha mencionado que estos se elevan cuando el personaje se acerca. Internamente, los orbes tenían dos estados, elevado y en el suelo. El problema de esta máquina de estados era que no tenía en cuenta todas las posibilidades, como que el personaje se aleje mientras se esté elevando. Tras realizar pruebas con múltiples orbes se encontró que varios quedaban elevados, lo cual era un error. En un principio se pensó que la causa de este error era la gestión de la lista, lo cual llevó a estar horas sin encontrar nada. Al final se descubrió la causa y se modificó la máquina de estados para incluir todos los estados en los

que se puede encontrar un orbe. Ahora las acciones de subir y bajar representan dos nuevos estados, y son capaces de reaccionar correctamente a que el personaje se aleje.

Otro de los problemas detectados ha sido un comportamiento inesperado de los enemigos. Cuando una pared separa al personaje de un enemigo, se podía dar el caso de que el enemigo intente atacar al personaje. Este problema es causado por la forma en la que se manejan los rayos de visión de los enemigos. Dado que los rayos únicamente detectan al personaje, ignoran los suelos y paredes. Como solución, si un enemigo detecta al personaje, primero se comprueba que no haya primero un muro en la línea de visión.

7.3. Iteración 3

El objetivo de la tercera iteración consiste en obtener un producto jugable. Teniendo las bases del juego, en esta fase se añaden interfaces, menús y una forma de progresar entre niveles a fin de tener un juego, aunque limitado, completo.

Requisitos de la iteración

- Aspecto jugable
 - Añadir el *Heads Up Display* (HUD).
 - Añadir menús de inicio, pausa, derrota y finalización del juego.
 - Habilitar el avance entre niveles.
 - Crear tres niveles, uno de ellos siendo un tutorial.
 - Añadir una forma de combinar armas.
- Aspecto audiovisual
 - Añadir sonidos.
 - Diseñar los menús.

7.3.1. HUD

El HUD o *Head-Up Display* es aquella información que se muestra al jugador sobre el estado de la partida durante el juego. Esta se muestra mediante gráficos y generalmente es visible al jugador en todo momento. En la figura 7.18 se pueden ver diferentes ejemplos de HUD en videojuegos populares.

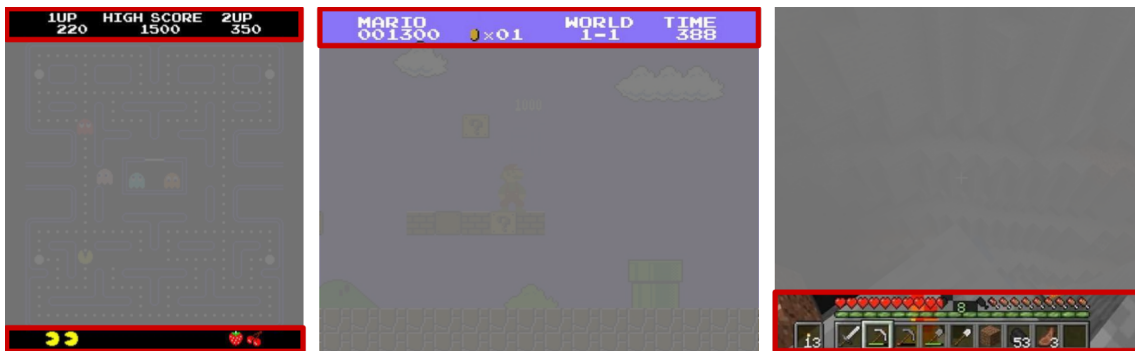


Figura 7.18: HUD en Pac-Man, Super Mario Bros y Minecraft

En el caso de Loop Escape la información mostrada es:

- Barra de vida del personaje.
- Barra de vida de los enemigos.

- Arma equipada como principal.
- Enfriamiento/*cooldown* del arma principal.
- Arma equipada como secundaria.
- Accesorios activos.
- Número de enemigos restantes.

Esta información se tiene que mostrar por encima del resto de elementos del juego, para que se vea claramente a la hora de jugar. Para ello se hace uso del componente *canvas* de Unity. El *canvas* es una capa donde se colocan todos los elementos relacionados con la interfaz. Esta capa se coloca la última a la hora de renderizar el juego, de forma que siempre se vea. Diferentes elementos son colocados en diferentes capas y esto ayuda a manejar su interacción y renderizado. En la figura 7.19 se pueden ver tres de las capas que se utilizan en el juego.

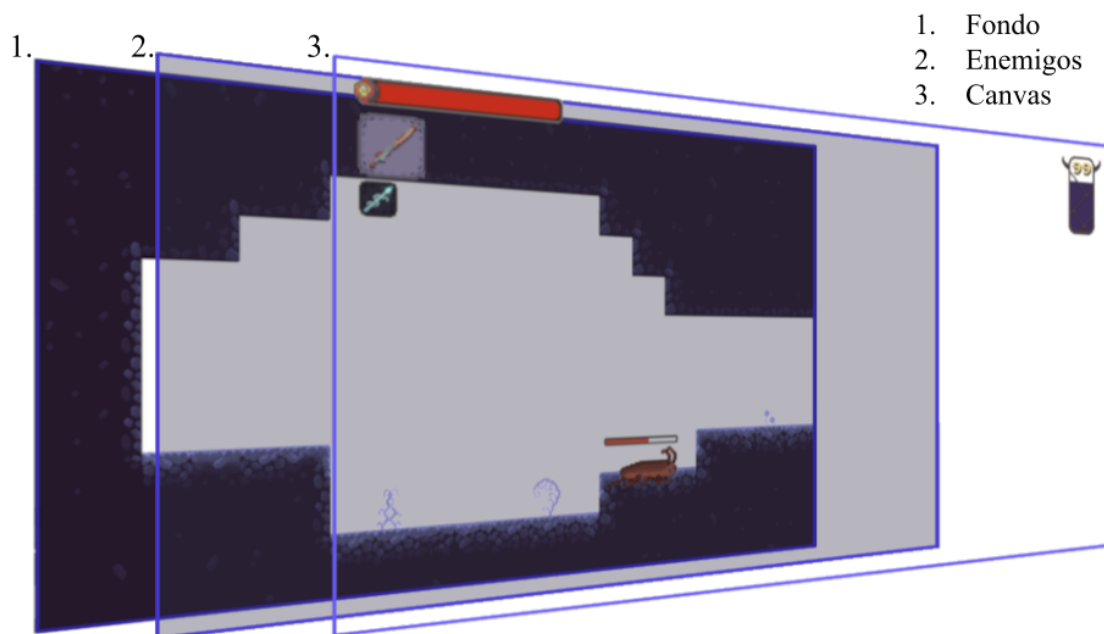


Figura 7.19: Tres capas mostradas en perspectiva

7.3.1.1. Barra de vida

Para crear la barra de vida del personaje se ha tomado como referencia un video de Brackeys [14]. Una barra de vida es un elemento que muestra de forma visual un valor actual respecto al máximo, como una botella transparente en la que se puede ver cuánta cantidad de líquido tiene.

Para generar la barra se hace uso del componente slider, el cual permite crear una barra de desplazamiento, comúnmente usada en elementos como los selectores de volumen. Modificando esta para que el jugador no pueda interactuar con ella, se

convierte en un medidor fácil de entender.

Para controlar la barra de vida se ha implementado la clase `BarraDeVida`, y esta puede ser asociada al componente Vida de una entidad, el cual actualizará el valor mostrado por la barra de vida cuando sea necesario.

Al añadir la barra de vida como una opción para el componente Vida, se puede crear una barra de vida más pequeña la cual sigue a cada enemigo y asociarla al componente. Esta barra de vida se encuentra en un pequeño *canvas* que se añade a cada enemigo.

En la iteración 2 se añadió un componente el cual invierte la dirección de un objeto, para que parezca que está mirando en la dirección contraria. Este cambio se aplica al enemigo, y todos los objetos asociados como hijos giran también. El problema de esto es que el *canvas* asociado al enemigo también se da la vuelta, lo cual es algo que se quiere evitar. En consecuencia ha sido necesario reestructurar la jerarquía de objetos del enemigo, de forma que se separan los componentes a los que debe afectarle el giro y a los que no.

7.3.1.2. Armas

Bajo la barra de vida del personaje se añaden las armas que este porta. Se muestran dos imágenes, una grande y una pequeña, siendo la grande en la que se ve el arma principal. En la figura 7.20 se puede ver gráficamente. Para mostrar cuánto queda hasta poder atacar con el arma principal se ha implementado algo similar a la barra de vida. Se ha añadido un *slider* con la diferencia de que este es vertical, y el encargado de su actualización es el propio arma. Cuando se equipa un arma como la principal, se le pasa la referencia a este *slider*. Cada vez que el arma calcule cuánto queda para que pueda volver a ser usada, actualiza el *slider*. En la figura 7.20 el arma principal está oscurecido de mitad para abajo, indicando que falta la mitad del tiempo de enfriamiento para poder volver a atacar.

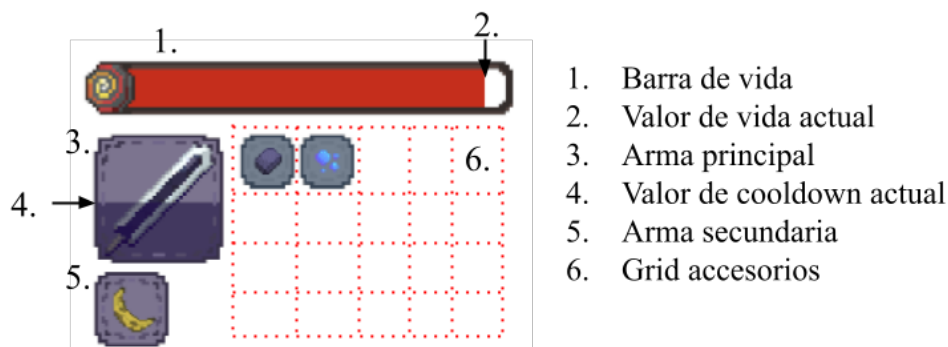


Figura 7.20: Esquina izquierda del HUD mostrando información sobre la vida y armas

Cuando se cambia de un arma a otra, se cambian las imágenes de cada recuadro y se actualizan los accesorios equipados. Los accesorios se muestran a la derecha de las armas y bajo la barra de vida. Gracias a un componente *grid* se puede establecer una cuadrícula, la cual se encarga de colocar los objetos hijos en la posición adecuada. En la imagen se ha representado este *grid* mediante puntos.

Estos accesorios se pasan desde el arma hasta el *script* que gestiona los accesorios del HUD. Para cada accesorio obtenido se crea un objeto gráfico al cual se le asigna la imagen correspondiente.

7.3.1.3. Contador de enemigos

En la esquina superior derecha se puede ver el contador de enemigos. Este es un slider que va bajando a medida que se derrotan enemigos. Cuando se inicia un nivel, los enemigos avisan al gestor del juego de su creación, y cuando son derrotados, de su derrota. El gestor actualiza el contador y al llegar a 0, el contador se convierte en una flecha que apunta a la salida del nivel.

A la hora de hacer la animación se ha descubierto una utilidad de Unity, las *corrutinas* o *coroutines*. Las *corrutinas* son funciones que se ejecutan por completo antes de retornar un valor, aunque la ejecución tome varios fotogramas. En el ejemplo que da Unity [15] se muestra una función la cual disminuye la opacidad de una imagen en bucle hasta que desaparece. Si este bucle se encontrase dentro de una función normal, este bucle se recorrería en un fotograma, haciendo desaparecer la imagen instantáneamente. En cambio, si se utiliza una *corrutina*, se puede añadir una espera dentro del bucle para que solo ejecute un ciclo por fotograma. Lo bueno es que esta espera no bloquea la ejecución del punto desde donde se la ha llamado. Esto es similar a cómo funciona el método *Update*, pero permite hacer funciones como la ejemplificada mucho más sencillas.

En el caso del contador de enemigos, se ha creado una función para rotar el contador de forma que deja de verse en pantalla y se sustituye por la flecha que apunta a la salida. En la figura 7.21 se puede ver una demostración gráfica.



Figura 7.21: Rotación del contador de enemigos

La fecha es rotada en función del ángulo entre el personaje y la salida. Para ello se obtiene la posición de ambos elementos, se calcula el vector de dirección de uno a otro y al fin se obtiene el ángulo.

Aprovechando las corrutinas se ha implementado una respuesta visual cuando una entidad es golpeada. Al recibir un golpe, el componente Estado busca el componente encargado de los efectos visuales y en caso de encontrarlo, este ejecuta una corrutina. La corrutina cambia el color de la entidad a rojo, y tras una espera de unos milisegundos vuelve a la normalidad.

7.3.2. Menús

A lo largo del juego aparecen múltiples menús y estos se dividen en dos tipos. Aquellos relacionados con el menú principal y aquellos que aparecen durante una partida.

En el menú principal se encuentran las opciones como jugar, salir del juego, acceder a las opciones y créditos. Durante la partida se puede acceder al menú de pausa, menú de derrota y menú de victoria.

Previamente se ha explicado el funcionamiento de las escenas y en este punto se hace uso de ello. El menú principal es una escena aparte donde no existe el objeto del personaje ni los enemigos. Este menú se ha creado utilizando un canvas y se puede ver en la figura 7.22.

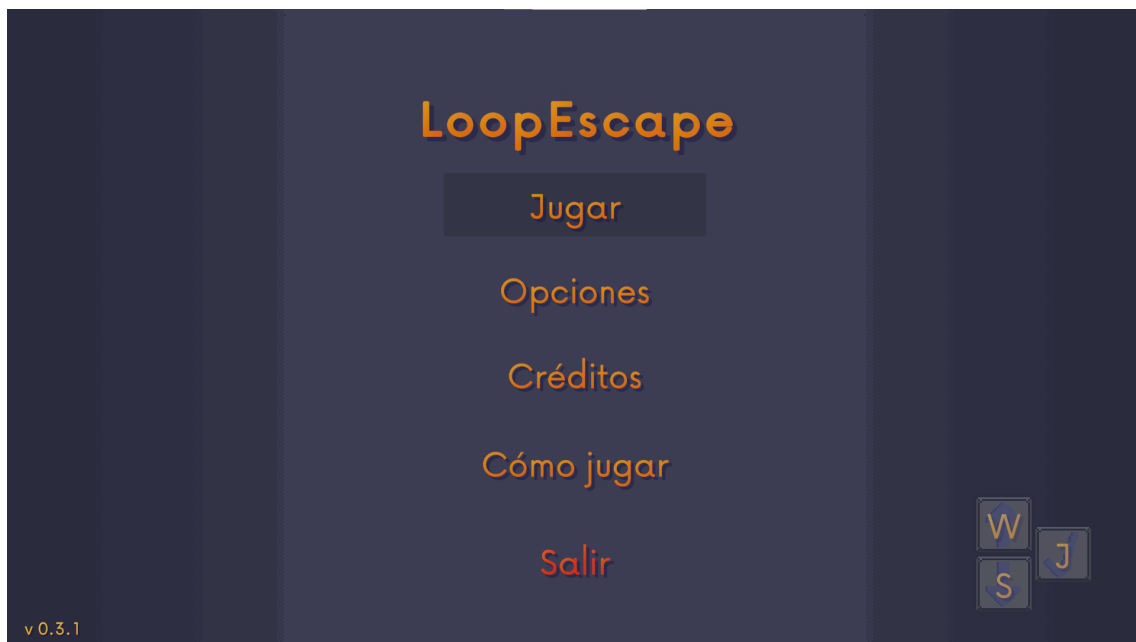


Figura 7.22: Menú principal

Los menús están compuestos de elementos como textos, botones o imágenes. Estas interfaces se han diseñado para que se puedan navegar tanto con el ratón como con el teclado.

Los menús de partida están añadidos en el mismo canvas que el HUD, pero se mantienen desactivados hasta que su uso sea necesario.

Para entrar a cualquiera de los menús de partida, el juego se pausa. La gestión de la pausa está inspirada en un video de Brackeys [16]. Para pausar el juego, se cambia la escala a la que se ejecuta el juego a 0, estableciendo el parámetro `Time.timeScale` a 0. En la primera iteración se ha mencionado que, para que el juego avance a la misma velocidad en diferentes dispositivos, las velocidades y temporizadores están relacionados con un parámetro, `Time.deltaTime`, el cual indica el tiempo entre fotogramas. Al bajar la escala de tiempo a 0 todas las físicas internas y las acciones relacionadas con `Time.deltaTime` quedan congeladas.

La gestión de la pausa se realiza desde su propia clase, `GestorPausa`, la cual está asociada al objeto `GameController`. Cuando el jugador pulsa la tecla 'p' o 'esc', este manda un aviso al `GameController`, el cual accede al `GestorPausa` y pausa o continúa el juego. `GestorPausa` tiene un booleano estático que indica si el juego está pausado o no. Gracias a eso todos los elementos pueden saber si el juego se encuentra pausado. A la hora de pausar el juego, aparece el menú mostrado en la figura 7.23.



Figura 7.23: Menú de pausa

7.3.3. Niveles y progreso

Para esta iteración se han creado tres niveles, siendo el primero un tutorial. Cada nivel es una escena de Unity diferente, por lo que todos los elementos que se han diseñado hasta ahora tienen que poder usarse en las múltiples escenas. Esto incluye el personaje, HUD, cámara... Para ello se han creado *prefabs* de estos elementos, gracias a esto, cualquier cambio realizado en el *prefab* original se ve reflejado en las copias añadidas a cada escena o nivel.

Al hacer este cambio, se ha tenido que cambiar como se manejan las referencias entre varios elementos. Los elementos ahora convertidos en *prefab* han sido creados en una escena y ciertas referencias puestas a mano. Es decir, el jugador necesita una referencia a la barra de vida del HUD para poder mostrar su vida actual. Esta referencia estaba asignada a mano desde el editor de Unity, pero al pasarlo a un *prefab* sería necesario añadir las referencias a mano para cada escena. Como solución, se ha creado una clase que gestiona las referencias necesarias del HUD, y diferentes elementos pueden acceder a esta clase para obtener las referencias automáticamente.

Con este cambio también ha sido necesario configurar los botones de la interfaz. Estos botones son los que llaman al GameController para volver al menú principal o continuar el juego. Gracias al foro de Unity [17], se ha encontrado que añadiendo un *listener* a un botón se pueden establecer las acciones que se ejecutarán una vez pulsado el botón.

Para pasar de nivel se carga la escena del siguiente nivel, y por lo tanto, se carga un nuevo *prefab* del personaje. Al hacer esto, se pierde todo el progreso del nivel anterior, como la vida actual o armas en posesión. Como solución, se hace uso de la función `DontDestroyOnLoad()`. La carga de una nueva escena elimina todos los objetos de la escena actual. Llamando a `Object.DontDestroyOnLoad()` se preserva el objeto durante la carga [18].

En este punto, el personaje no se borra al pasar de nivel, pero se sigue creando el *prefab* del personaje, por lo que aparecen dos personajes. Una solución sería únicamente añadir el *prefab* del personaje al primer nivel, pero eso causaría que no se puedan jugar el resto de niveles de forma individual. Además, dado que el objeto del personaje se mantiene entre niveles, se mantienen todos sus datos, incluyendo la posición del objeto, por lo que el personaje aparece en sitios inesperados.

La solución, un punto de aparición. En vez de añadir el *prefab* del personaje a cada nivel o solo a uno, no se añade a ninguno. En cambio, se añade el nuevo objeto, punto de aparición, en la posición en la que se quiere que aparezca el personaje. Al iniciar la escena se busca si hay un personaje, si no hay, crea una copia del *prefab*, si hay, se mueve a la posición del punto de aparición.

Dado que el personaje se mantiene activo al pasar entre escenas, es necesario que vuelva a obtener referencias a objetos como el HUD, el cual se destruye y regenera en cada nivel.

7.3.4. Altares

En el tutorial se enseñan las mecánicas del juego, como desplazarse, saltar, atacar o cambiar de arma. Estos mensajes son mostrados cuando el personaje se acerca a un objeto con forma de símbolo de interrogación. Este es uno de los tres tipos de altares, los cuales se pueden ver en la figura 7.22.



Figura 7.24: Tipos de altares

El brillo azul es una imagen separada y normalmente oculta. Cuando el personaje se acerca a un altar se muestran las líneas azules para indicar que es un elemento con el que se puede interactuar. Además, se abre una ventana la cual cumple diferentes funciones.

En el caso del altar de mensaje, muestra un mensaje. El altar de nivel permite avanzar al siguiente nivel presionando un botón indicado. Este altar permanece oculto hasta derrotar a todos los enemigos del nivel. El último altar es el de armas y este permite combinar armas.

A la hora de combinar armas es necesario tener un arma principal y una secundaria. Cuando se presiona el botón indicado, ambas armas son extraídas del personaje y se genera un orbe de botón con el resultado.

El problema de crear un orbe de *loot* con el resultado de la combinación es que el personaje se queda sin armas. Esto es un problema dado que, por como está diseñada la gestión de armas, el personaje no puede no tener ningún arma. Al iniciar la partida, se da al personaje un arma simple para cumplir esta regla.

Para solucionarlo ha sido necesario añadir un arma auxiliar. Esta arma son unos guantes y al atacar, el personaje pega un puñetazo. Al añadir el arma auxiliar ha sido necesario cambiar la lógica utilizada al recoger, soltar y atacar para incluir el caso donde el personaje no tiene ningún arma, exceptuando la auxiliar. El arma auxiliar no puede ser desechada, ni combinada con otras armas, y si el jugador encuentra un arma, la auxiliar se desactiva hasta que sea necesario.

7.3.5. Sonidos

En este punto del juego ya hay mecánicas y animaciones, pero falta un elemento crucial: sonidos.

En un videojuego, los sonidos pueden servir para dar información al jugador o simplemente para crear un ambiente más inmersivo. En el caso de la información, toda la información que se da de forma auditiva también es mostrada de forma visual, para no dificultar la experiencia a personas con problemas auditivos.

Cada enemigo necesita múltiples sonidos, como puede ser al atacar, ser atacado o derrotado, cuando se mueven o cuando ven al personaje. El jugador tiene sonidos al caminar, saltar, aterrizar, ser golpeado... Además, cada arma tiene sonidos propios al ser equipada o atacar.

Todos estos sonidos son creados combinando sonidos de diferentes páginas, las cuales ofrecen sonidos con licencia de uso *Creative Commons* 0 o de autoría. Estas páginas son Sonniss ² y Freesound³. Estos sonidos son combinados y modificados en Audacity para crear el resultado final.

Para reproducir sonidos se utiliza un componente de Unity. Este componente permite reproducir distintos sonidos alterando el volumen y el tono. Dado que hay sonidos como el uso de un arma que se repiten constantemente, se modifica el volumen y tono de forma aleatoria para otorgar variedad.

Los sonidos se gestionan en un componente nuevo. Durante su uso se le pasa un identificador del sonido y este se comunica con el componente de Unity. También es importante gestionar la pausa de los sonidos para que no sigan ejecutándose mientras el juego está pausado.

Durante la implementación de los sonidos se han descubierto varios errores, los cuales pasan desapercibidos a simple vista. Al querer reproducir un sonido con ciertas acciones, se han encontrado casos donde un método se reproduce varias veces

²<https://sonniss.com/>

³<https://freesound.org/>

en vez de solo una. Por ejemplo, al recoger un arma nueva, las armas se equipan y se desequipan múltiples veces internamente, lo que provoca más sonidos de los necesarios. Visualmente no es notable, pero al reproducir los sonidos, se escuchan varios sonidos a la vez.

Un caso similar se da con los orbes de botín. Cuando un orbe rebota contra una pared, se le da la vuelta a la velocidad horizontal para simular el rebote. Cuando la velocidad llega a 0 gracias a la fricción, si toca una pared va a dar la vuelta, pero va a seguir tocando dado que no tiene velocidad para alejarse, entonces se vuelve a reproducir el sonido. Esto ocurre constantemente hasta que toca el suelo y deja de comprobar los rebotes.

El problema es que se comprueba si toca una pared a ambos lados, no únicamente en la dirección en la que se está moviendo. Debido a esto también existe el caso donde el orbe se queda pegado a la pared dado que se le está dando la vuelta a la velocidad constantemente. Visualmente se puede ignorar este error, pero al añadir sonidos pasa a ser una molestia.

Como solución, únicamente se comprueba si hay pared en la dirección en la que el orbe se está desplazando.

7.3.6. Pruebas

Para realizar las pruebas se ha compilado el juego y se ha dado a probar a varias personas. Antes de realizar las pruebas, se ha pedido a los encuestados que lean el cuestionario que contestarán tras jugar, para que sean conscientes de los detalles en los que deben fijarse. En la encuesta se pide la opinión de la persona encuestada sobre:

- Facilidad de controlar el personaje.
- Dificultad del juego.
- Experiencia previa con videojuegos similares.
- Facilidad de entender cómo funciona el juego.
- Errores durante la ejecución.
- Sugerencias para futuras versiones.

De esta encuesta, los resultados importantes son aquellos que mencionan dificultades del juego o eventos que creen que no deberían ocurrir. Aparte de los cambios y mejoras propuestos en las encuestas, a continuación se incluyen también otros cambios realizados para mejorar o arreglar la experiencia de juego:

Cámara

El tercer y último nivel del juego consiste en descender entre plataformas derrotando enemigos hasta encontrar el altar de nivel en el fondo.

El problema es que si el personaje tiene que saltar a una plataforma por debajo de lo que muestra la cámara, el jugador no saltará porque no sabe si hay una plataforma. Para solucionar este problema se ha añadido una función para mover la cámara. Si el personaje está quieto y se mantienen las teclas w/s, la cámara sube o baja un poco. Dado que la cámara sigue la posición del jugador, se suma o resta una constante a esa posición para que la cámara suba o baje.

Ataques con identificador

Cuando un proyectil del personaje impacta a un enemigo, este entra en un estado de invulnerabilidad. De esta forma se evita que el enemigo reciba daño del proyectil por cada fotograma en el que estén en contacto.

Si el jugador realiza dos ataques seguidos y el segundo proyectil impacta al enemigo estando este en estado de invulnerabilidad, el enemigo no sufrirá daño. Para solucionar esto, cuando un proyectil es generado, es asignado un identificador. Una vez golpea al enemigo, este se vuelve invulnerable durante el tiempo establecido, alrededor de un segundo, a los impactos provenientes del proyectil con ese identificador. Para conseguirlo, cada enemigo tiene una lista de pares de identificadores y la hora en la que han sido golpeados. Al recibir un ataque nuevo, se recorre la lista comprobando si tienen invulnerabilidad y a la vez eliminando identificadores antiguos de la lista.

El personaje se mueve solo

A lo largo de la partida puede ocurrir que tras detener el personaje, este se moviese un poco, como si estuviese vibrando. Este error lleva en el juego desde el principio, pero no se ha solucionado hasta el momento dado que se creía un error del editor de Unity.

En un principio se creía que la causa podría ser el sistema de entrada, input, de Unity. Este no es discreto sino continuo, es decir, al pulsar una tecla el valor fluctúa de 0 a 1. Para evitar ruido, se considera que el jugador está pulsando la tecla una vez que supere un umbral. Para comprobar si este es el caso, se ha creado una nueva interfaz en el HUD, la cual muestra el valor actual del input horizontal. Tras varias pruebas se concluyó que el problema no era el input sino la fricción.

Cuando el jugador deja de pulsar las teclas de movimiento, se comienza a aplicar fricción al personaje. Esta fricción se calcula restándole un valor fijo a la velocidad actual. Dependiendo de la velocidad, positiva hacia la derecha o negativa

hacia la izquierda, se le resta o suma ese valor. El error ocurre cuando se suma o resta el valor y la velocidad cambia de signo. Es decir, en vez de restar y que la velocidad llegue a 0, hace que se desplace en dirección contraria, por lo que se vuelve a aplicar la fricción, y la velocidad vuelve a cambiar de signo. Esto causa que el personaje vibre y se desplace un poco.

Como solución, si al modificar la velocidad esta baja de 0, se fuerza que se quede a 0, asegurándose de que el personaje frene definitivamente.

Quitar el ratón de la pantalla

Durante la partida el ratón se puede ver en la pantalla aunque no haya nada con lo que pueda interactuar. Esto es confuso, por lo que el ratón se esconde si el juego no se encuentra en un menú.

Hacer más fácil de entender los altares

Durante el tutorial se pueden encontrar varios altares de mensajes. El texto de estos ha tenido que ser modificado dado que ciertos mensajes no dejaban claro qué debería hacer el jugador. Además, el altar de nivel se ha modificado para que la forma de usarlo se intuya más fácilmente. En un principio, si el personaje se mantenía cerca del altar por unos segundos, se avanzaba de nivel. Ahora aparece un mensaje similar al altar del mensaje y se indica un botón. Al mantener el botón aparece una barra de carga y, al llenarse, se avanza de nivel.

7.4. Iteración 4

Originalmente, el objetivo de la cuarta iteración consistía en la implementación de generación procedimental de salas. Llegado este punto, se ha decidido alterar el orden de los paquetes de trabajo para priorizar aquellos que acercan en mayor medida el desarrollo a un proyecto finalizado. Aunque la generación procedimental de salas es un gran añadido al videojuego, elementos como los jefes o el sistema de guardado son mucho más prioritarios.

Por estos motivos, la generación procedimental pasará a ser un objetivo secundario, y en esta iteración se implementarán los jefes. Hay que tener en cuenta que los objetivos de esta iteración han sido ligeramente modificados para compensar el cambio. Este añadido es la necesidad de crear dos niveles más para poner entre los dos jefes que se van a crear.

Requisitos de iteración 4

- Aspecto jugable
 - Crear dos salas para los jefes.
 - Diseñar e implementar el comportamiento de dos jefes.
 - Crear dos armas únicas las cuales serán el botín de los jefes.
- Aspecto audiovisual
 - Diseñar el aspecto y animación de los jefes y armas.
 - Componer los sonidos de los jefes y armas.

7.4.1. Calamar Vampiro

El primer jefe a implementar será el Calamar Vampiro. Inspirado por el pequeño cefalópodo de aguas profundas con el mismo nombre, este jefe flotará por la sala lanzando proyectiles de tinta. En la figura 7.25 se puede comparar el jefe con una foto del animal real sacada por el equipo de National Geographic⁴.

⁴www.nationalgeographic.com/animals/article/120929-vampire-squid-deep-ocean-animals-science-monterey



Figura 7.25: Calamar Vampiro : Jefe y animal

Este enemigo tiene dos formas diferentes de atacar. En la primera de ellas lanza burbujas de tinta en un patrón de espiral. En la segunda, hace caer meteoritos de tinta desde el cielo, los cuales al impactar con el suelo, crean olas de tinta. El ataque que use dependerá del estado en el que se encuentre.

El comportamiento del calamar es simple. Desciende lentamente hasta que detecta que está cerca del suelo, entonces se impulsa hacia arriba y vuelve a descender. También puede detectar las paredes para alejarse de estas. Esto se consigue gracias a los componentes “Gravedad”, “SensorSuelo”, “SensorPared” e “Impulso” previamente creados.

Cuando se impulsa hacia arriba, realiza el ataque en espiral y una vez haya realizado el ataque en espiral un número fijo de veces, se impulsa hasta salir del área de la cámara. En este estado realiza el ataque de los meteoritos de tinta. Una vez acabado ese ataque, descende y se repite el ciclo. En la figura 7.26 se puede ver una representación gráfica de los ataques y a continuación serán explicados en más detalle.

Para diseñar el ataque en espiral se ha utilizado una corrutina. Esta dispone de un bucle en el cual crea proyectiles y los dispara aumentando un poco el ángulo en que se mueven por cada ciclo del bucle. Entre ciclos del bucle se hace una espera. Los proyectiles generados utilizan la misma lógica que los del personaje, por lo que han sido modificados para poder indicar si deben afectar al personaje o a los enemigos. El ataque en espiral utiliza como proyectil las burbujas, pero podría utilizar cualquiera de los proyectiles creados, como la bola de fuego.

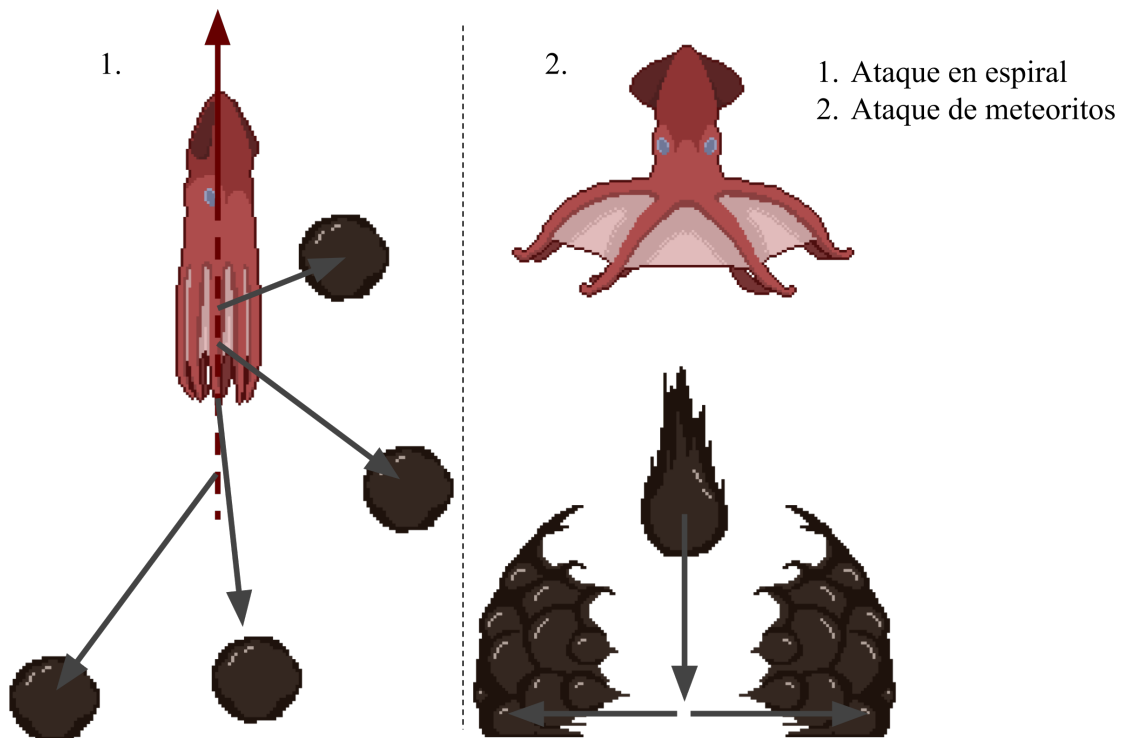


Figura 7.26: Ataques del calamar vampiro

El ataque de los meteoritos consiste en generar proyectiles que apunten hacia abajo a una altura concreta. Es necesario fijar la posición de las paredes de la sala para generar los proyectiles dentro y no fuera de esta. Cuando los proyectiles tocan el suelo, se destruyen. A la hora de destruirse se utiliza un nuevo modificador de proyectiles, el cual genera dos nuevos proyectiles apuntando a ambos lados.

Estos proyectiles tienen un modificador especial, el cual hace que al aparecer se peguen al suelo. Para lograr esto, primero se comprueba si el centro del proyectil se encuentra dentro del suelo o fuera. Utilizando la función *Collider2D.OverlapPoint*⁵ se puede saber si el punto dado se encuentra dentro de un área de colisión. El uso de esta función ha sido inspirado en la solución ideada por Ginxu, un desarrollador de videojuegos español, en uno de sus videos [19]. Sabiendo si el proyectil se encuentra dentro o fuera del suelo, se lanza un rayo hacia arriba o abajo para encontrar la altura a la que deberá moverse el proyectil. Teniendo ese punto, se calcula la posición que debe tomar el centro del proyectil para que la parte inferior de su área de colisión coincida con el suelo.

Como botón especial, el calamar suelta una varita que genera burbujas de tinta, aprovechando el proyectil ya creado.

⁵<https://docs.unity3d.com/6000.0/Documentation/ScriptReference/Physics2D.OverlapPoint.html>

7.4.2. Mandoble Volcánico

El segundo jefe a implementar es una espada que vuela por el escenario rebotando en las paredes. Su comportamiento es simple. Inicia clavada en el suelo o pared y cuando el personaje se acerca, comienza a salir del suelo. Una vez sale, la espada vuela rebotando en las paredes, techo o suelo. Mientras está rebotando el *sprite* de la espada rota. Si el *sprite* de la espada apunta al personaje, la imagen de la espada deja de rotar y se desplaza en la dirección en la que ha visto al personaje. Avanza hasta que se clava en el escenario y el ciclo se repite. En la figura 7.27 se puede ver una representación del ciclo de ataque.

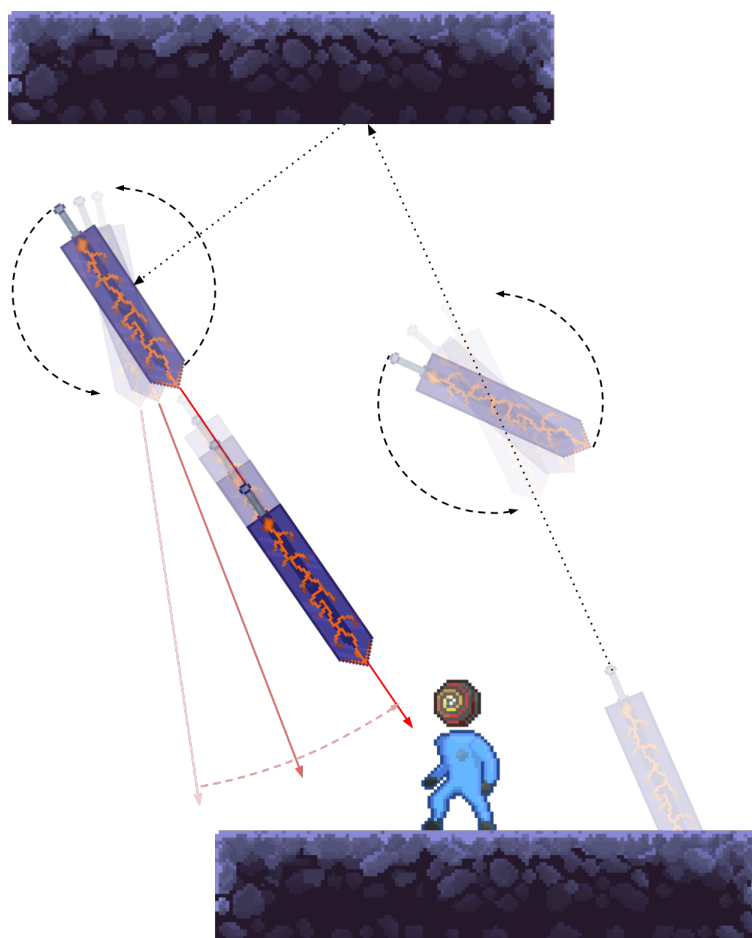


Figura 7.27: Comportamiento del mandoble volcánico

La espada hace daño al personaje mientras está volando, por lo que deberá esquivarla mientras rebota por el escenario. Para permitir que el jugador se acostumbre al enemigo, pero que este suponga un desafío, la velocidad a la que se desplaza la espada aumenta en función de la vida que tenga el jefe. Cuanto menos vida, más rápido.

Para simular el rebote de la espada, esta lanza un rayo en la dirección en la que se está desplazando. Si detecta una colisión, se sabe que ha chocado con una pared, suelo o techo, pero no se sabe cuál de ellos. En función de la dirección de la superficie, la espada deberá cambiar su dirección hacia un lado o hacia otro. Es importante decidir correctamente la nueva dirección dado que, en caso de rotar en la dirección equivocada, la espada atravesaría la pared y se saldría del escenario.

Para llevar a cabo la comprobación, se lanzan dos rayos más, uno por encima de la espada y otro por debajo. Hay que tener en cuenta que “arriba” y “abajo” dependerán de la dirección del vector de desplazamiento. Para este ejemplo se entiende que la espada se desplaza hacia arriba a la izquierda. En la figura 7.28 se presenta un gráfico para acompañar la siguiente explicación. Se calcula el vector perpendicular a la dirección de la espada para encontrar la posición desde donde se disparan los rayos. Teniendo ambos rayos, se comprueba cuál de los dos ha detectado una pared. Si el rayo de abajo ha detectado una colisión, la espada pasa a moverse en la dirección perpendicular que no lleva a la pared. Si ninguno de los rayos detecta una pared, la dirección de movimiento se invierte.

En el caso de que ambos rayos detecten una pared o techo, se comprueba cuál de los rayos está más metido en la pared.

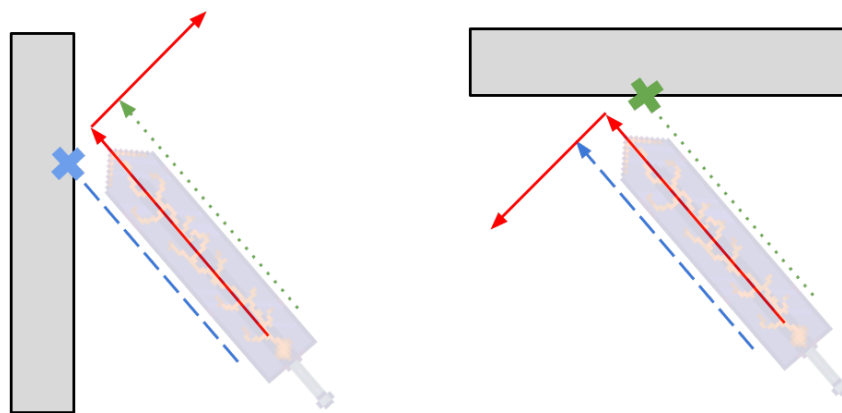


Figura 7.28: Detección de paredes o techos

Dado que la espada no requiere ninguna animación, se ha creado un componente el cual cambia el *sprite* de un objeto en función de su vida actual. Se han creado varias versiones de la espada en diferentes estados de destrucción y gracias al componente, el aspecto de la espada va cambiando.

También se ha aprovechado el componente Trail de Unity para que la espada deje un rastro similar a un cometa. Como se muestra en la figura 7.29, esto facilita el seguimiento de la espada cuando se mueve muy rápido. Como resultado no previsto, añadir un rastro a la espada ha ayudado a la hora de diseñar la detección de paredes.

Cuando la espada se desplaza muy rápido, es difícil seguirla y cuando se sale del escenario no se sabe dónde ha rebotado ni qué caso se ha dado. En la figura 7.29 se puede ver cómo siguiendo el rastro de la espada, se encuentra un rebote donde la dirección calculada saca a la espada del escenario.

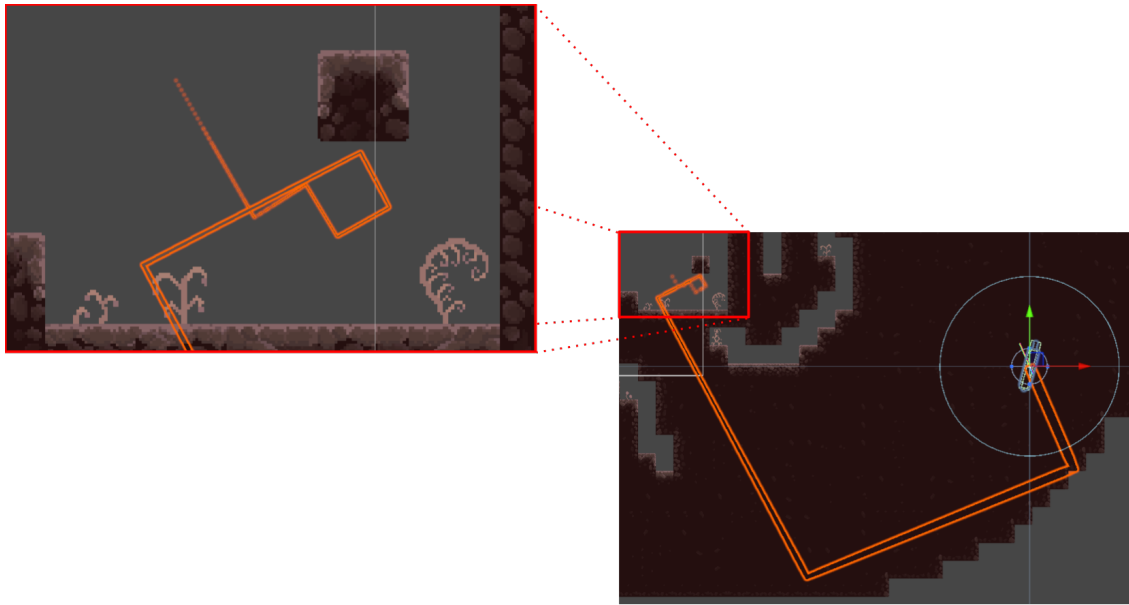


Figura 7.29: Rastro de la espada

Como botín, este jefe suelta un mandoble volcánico. Este mandoble tiene el mismo aspecto que el jefe, pero su comportamiento es como el del resto de espadas.

7.4.3. Interfaz

Al combatir con un jefe, en vez de mostrar su barra de vida sobre el enemigo, se muestra en una posición fija de la pantalla. En este caso, la barra de vida junto con el nombre del jefe se encuentra en la parte inferior de la pantalla. Gracias a un nuevo componente se recoge la referencia de la barra de vida desde el gestor de referencias del HUD y se asigna al componente Vida del jefe, para que muestre la vida actual de este.

7.4.4. Nuevos niveles

Para compensar por el cambio en el orden de las iteraciones y a consecuencia de comentarios en las pruebas anteriores indicando los saltos de dificultad en los niveles, se han creado y rediseñado los niveles actuales hasta tener los siguientes:

- Tutorial
- Nivel 1 y 2 (fáciles)

- Jefe Espada
- Nivel 3 y 4 (difíciles)
- Jefe Calamar

En los niveles 3 y 4 se ha aprovechado el comportamiento de la espada voladora para crear un nuevo enemigo. A diferencia del jefe, estos enemigos se mueven más lentos y tienen menos vida, haciendo el desafío de derrotarlos menor a enfrentarse al jefe original.

7.4.5. Pruebas

En esta iteración, el aspecto más propenso a fallos es el movimiento de mandoble volcánico. Para probar su funcionamiento correcto, se ha modificado el nivel del jefe para que la espada se encuentre en un túnel estrecho. Al no tener un amplio espacio para maniobrar, la espada es más propensa a salirse del escenario, dando oportunidad de analizar la causa de este comportamiento. Como se ha mencionado previamente, gracias al rastro dejado por la espada, el análisis se ha hecho más ameno.

Además, tras recibir *feedback* de varios jugadores, se han modificado los valores de los atributos de los jefes a fin de ajustar la dificultad, la cual se considera muy baja. En general, gracias a que utilizan la misma base que los enemigos, esta iteración se ha desarrollado con pocas complicaciones.

7.5. Iteración 5

En quinta y última iteración, exceptuando objetivos secundarios, se tiene como fin añadir los últimos detalles que hacen del proyecto un juego. En esta, se implementa el sistema de guardado para que el progreso del juego no se pierda al cerrar la aplicación, la opción de jugar con diferentes personajes, un sistema de progresión entre partidas y un menú de ajustes.

Requisitos de iteración 5

- Aspecto jugable
 - Sistema de guardado de partidas.
 - Selección de personaje.
 - Sistema de progresión.
 - Ajustes y accesibilidad.
- Aspecto audiovisual
 - Diseñar dos nuevos personajes.
 - Diseñar los nuevos menús.

7.5.1. Sistema de guardado de partidas

A lo largo de una partida se puede dar el caso de querer dejar de jugar. Cuando esto ocurre, la persona que esté jugando no querrá perder todo el progreso a la hora de cerrar la aplicación. Gracias al guardado de datos se puede guardar el progreso y volver a cargarlo al reiniciar la aplicación.

Para guardar los datos se hace uso de un archivo JSON, *JavaScript Object Notation*. Este archivo se encuentra en texto plano, no cifrado, y es fácilmente legible. Los datos del juego no se cifran para dar libertad de modificarlos al usuario. Si se tratase de un juego multijugador, sería importante cifrar los datos dado que al poder modificarlos existe la posibilidad de obtener una ventaja de forma injusta sobre el resto de jugadores. Dado que es el propio jugador el único que se ve afectado por la modificación de estos, se dejan en texto plano de forma que el usuario pueda experimentar con los límites del juego. Además, durante el desarrollo es conveniente poder modificar manualmente los datos a la hora de hacer pruebas.

En este archivo se van a guardar diferentes datos:

- La vida del personaje.
- El arma principal.

- Los accesorios del arma principal.
- El arma secundaria.
- Los accesorios del arma secundaria.
- El nivel actual.

Más adelante puede ser necesario añadir más tipos de datos. Estos se añadirán al mismo archivo. La vida del personaje se guarda como un *float*, pero el resto de elementos no son objetos serializables; es decir, no pueden convertirse directamente a JSON. Esto se debe a que las armas y accesorios son objetos de Unity, no objetos simples como texto o números.

Para poder guardar las armas y accesorios, es necesario saber el tipo de arma o accesorio, además de la potencia de este último. Como solución, se ha creado una lista de identificadores aprovechando la estructura *enum* de C#. Esta estructura permite asociar números enteros a palabras clave, como pueden ser ESPADA_OXIDADA o LIBRO_ARDIENTE. De esta forma, en el JSON se guarda el identificador numérico asociado a cada arma. Tanto para el arma principal como para la secundaria, se guardan dos listas. Una con los identificadores de los accesorios y la otra con sus potencias. Sabiendo cómo van a estar guardados los datos, es necesario leerlos y escribirlos desde el juego.

7.5.2. Selección de personaje

Para añadir variedad a cada partida, se han añadido dos personajes con los que se puede jugar. Estos personajes difieren del original en las estadísticas de sus componentes, como puede ser la velocidad o la vida máxima. En la figura 7.30 se pueden ver los diseños de los tres personajes.



Figura 7.30: Personajes seleccionables

A la hora de comenzar una nueva partida, se abre una interfaz donde se puede seleccionar el personaje con el que se quiere jugar. A la hora de crear el personaje, se utiliza el *prefab* correspondiente al personaje seleccionado. El tipo de personaje está asociado a un identificador y este es guardado en el JSON junto con el resto de datos de la partida.

Cada uno de estos personajes cuenta con todas las animaciones del personaje original adaptadas para mantener los nuevos diseños.

7.5.3. Progresión

En los videojuegos, es común encontrar sistemas de progresión. Estos otorgan beneficios al jugador a medida que este juega al juego. Una forma usual de hacer esto es bloqueando parte del contenido del juego e ir desbloqueándolo a medida que se jueguen varias partidas. De esta forma se incentiva jugar más de una vez. En este caso, se ha decidido bloquear los personajes seleccionables y, únicamente permitir su uso tras superar ciertos desafíos. Así, al iniciar nuevas partidas en el futuro, se podrá jugar con personajes los cuales no estaban disponibles al iniciar el juego por primera vez.

Para desbloquear los personajes, el jugador deberá derrotar a ambos jefes del juego, desbloqueando un personaje por cada jefe. Esto se ha implementado modificando los altares de nivel para que puedan otorgar recompensas.

En los datos del juego se guarda una lista con los identificadores de los personajes desbloqueados. En la figura 7.31 se puede cómo el personaje sin desbloquear se muestra más oscuro que el resto.



Figura 7.31: Menú de selección de personaje

7.5.4. Ajustes y accesibilidad

Para aumentar el público objetivo del juego y permitir adaptarlo a personas con distintos gustos y necesidades, inicialmente se han planteado 4 ajustes: Elección entre pantalla completa o ventana, idioma, reasignación de controles y velocidad de juego. En la figura 7.32 se puede ver el resultado final de la interfaz de ajustes.

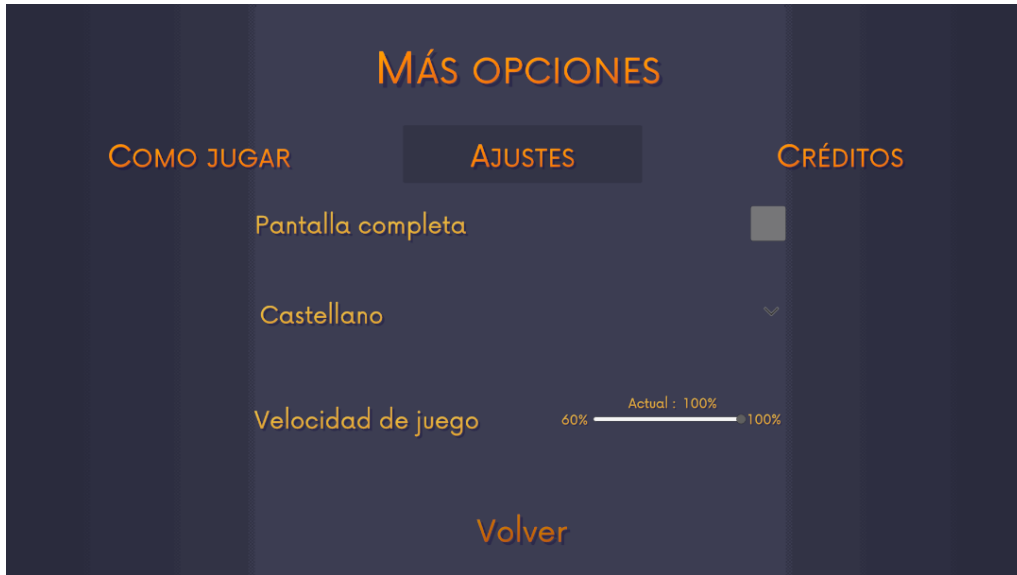


Figura 7.32: Interfaz de opciones

7.5.4.1. Pantalla completa

El juego se ejecuta automáticamente en pantalla completa. Hay personas las cuales, debido a no poder usar un teclado físico, necesitan utilizar un teclado digital que se muestra en pantalla [20]. Si el juego se encuentra en pantalla completa, esta posibilidad se pierde, por lo que desde el menú de ajustes se permite cambiar entre pantalla completa o modo ventana. Esta selección y los ajustes explicados a continuación se guardan junto con los datos del juego.

7.5.4.2. Selección de idioma

Una forma sencilla de aumentar la demográfica con acceso a un juego es añadir la posibilidad de cambiar de idioma. Se ha implementado un sistema que permite cambiar de idioma entre una lista preestablecida de estos, actualmente castellano e inglés.

Para implementar el cambio de idioma, se ha hecho una lista con todos los textos que se utilizan en el juego y se han dividido en dos categorías: textos fijos y de variables. Los textos fijos son aquellos que, sin importar la situación muestran el

mismo contenido. Por ejemplo, el botón de “Continuar partida” siempre va a mostrar una frase con ese significado, variando únicamente en el idioma.

Los textos variables son aquellos que no siempre tienen el mismo contenido. Por ejemplo, al pausar el juego se muestran los datos de las armas, como puede ser su nombre y descripción. Dependiendo del arma equipada en el momento, el contenido del texto varía. Lo mismo ocurre con las descripciones de los accesorios.

Los textos se guardan en archivos de texto plano para que puedan ser leídos por el juego durante la ejecución. Las traducciones se han colocado en una tabla, guardando en cada línea una lista de traducciones para un texto. Si se quieren añadir más idiomas, únicamente hay que añadir columnas, y si se quieren añadir más textos, hacen falta más líneas.

Al cargar el juego, se cargan todos los textos en una matriz, y cuando un elemento, como un botón, necesita un texto, recoge de la matriz la traducción adecuada. El proceso encargado de los idiomas guarda el identificador del idioma actual, mientras que el botón guarda el identificador del texto. Entre ambos se obtiene la traducción adecuada.

Desde el menú de ajustes se tiene acceso a una *DropDown List* donde se puede seleccionar el idioma.

7.5.4.3. Reasignación de controles

Para facilitar diferentes formas de juego, la reasignación de controles es una importante opción [21]. Esta permite cambiar las acciones ejecutadas al pulsar determinadas teclas o controles.

En Unity existen dos formas de detectar cuándo y cómo el jugador pulsa los controles. La original, *Input Manager*, y usada actualmente consiste en comprobar en cada fotograma si cierta tecla acaba de pulsarse, estaba pulsándose o se ha liberado. En cambio, el nuevo sistema de Unity, *Input System Package*, funciona mediante eventos, ejecutando funciones cuando se pulsan ciertos controles.

Para poder llevar a cabo una implementación robusta de reasignación de controles, sería necesario hacer uso del nuevo sistema. Esto se debe a que el nuevo sistema maneja el uso de diferentes controles de forma más efectiva. Si se quiere cambiar de un sistema a otro, es necesario refactorizar el código a fin de que funcione mediante eventos, a diferencia de comprobar constantemente el estado de las teclas.

Esta opción se ha valorado, pero el coste supera en gran medida al beneficio, por lo que se ha decidido dejar la reasignación de controles de lado. En proyectos futuros, la selección de un tipo de sistema u otro deberá plantearse desde un principio para evitar este problema.

7.5.4.4. Velocidad de juego

A fin de hacer el juego más accesible para personas con discapacidades motoras o cognitivas, se ha añadido una opción para ralentizar el juego [22]. Esto aumenta el periodo de tiempo disponible para reaccionar a ciertos eventos, como esquivar un ataque, haciendo del juego una experiencia más disfrutable.

Unity consta de la variable *Time.timeScale* la cual se modifica para aplicar este cambio. Esta misma es la que se usa a la hora de pausar el juego, donde se modifica la velocidad del juego a 0.

7.5.5. Pruebas

Para guardar los ajustes y la partida actual se hace uso de un archivo de texto. Durante las pruebas se han modificado los diferentes valores guardados en este archivo para comprobar que son cargados correctamente. Al probar con valores incorrectos, como puede ser indicar que se está utilizando el personaje número 5, cuando solo existen tres personajes, ocurren errores. Lo mismo ocurre cuando se rompe el formato JSON. Para solucionarlo, se ha añadido una comprobación para cada valor, y en caso de ser incorrecto, se establece al valor por defecto. Si el formato JSON es incorrecto, se reinician los datos del juego. Dado que se da al jugador libertad de modificar el archivo de guardado, es necesario tomar este tipo de medidas.

Dado que esta es la iteración final, se ha hecho un cuestionario a fin de dar a probar el resultado final del proyecto. Esta encuesta es similar a la de la tercera iteración, haciendo más hincapié en las interfaces y menús. Los resultados son generalmente buenos y, aunque sea un juego de baja dificultad, el consenso es que el juego es fácil de controlar y jugar.

8. Conclusiones

En este TFG, se ha creado un videojuego para Windows utilizando Unity. El juego es del género *rougelike* y consiste en superar niveles a medida que se derrotan enemigos. A lo largo del juego, se pueden encontrar varias armas, enemigos y jefes a fin de crear una experiencia de juego agradable. A continuación, se desarrollan las conclusiones extraídas durante la creación de este proyecto.

8.1. Objetivos

En un principio se habían establecido una serie de objetivos principales y secundarios. A continuación se detalla si se han completado y la razón de considerarse así.

Objetivos principales

Crear un videojuego ejecutable en sistemas Windows. Este es el objetivo más básico del proyecto y ha sido cumplido. Loop Escape es un videojuego completo, el cual se puede jugar en sistemas Windows. Gracias a Unity, en un futuro se podría exportar a otros dispositivos, a fin de hacer el juego más accesible.

Generación procedimental de salas. A mitad del desarrollo, se revisaron los objetivos y se realizaron cambios. Este objetivo pasó a ser secundario y finalmente no se ha abordado. El cambio en la planificación se dio por querer priorizar aquellas tareas que avanzan en mayor medida el desarrollo hacia un producto finalizado. La generación de niveles es importante, pero no deja de ser una “mejora” de los niveles normales. En consecuencia, se diseñaron nuevos niveles para añadir variedad.

Lograr una experiencia de juego fluida. La fluidez de un juego es una métrica subjetiva, la cual puede entenderse como “cómo de cómodo es jugar a un juego”. Esto puede verse afectado por la facilidad de manejar al personaje, el movimiento de la cámara o las interfaces del juego. Para valorar el cumplimiento de este objetivo, se ha hecho uso de las respuestas a la encuesta realizada sobre la versión final del juego. De esta encuesta se han valorado las respuestas a las preguntas:

- ¿Consideras que el personaje se mueve acorde a los controles?
- ¿La disposición de teclas se hace amena para jugar?
- ¿Consideras la experiencia de juego disfrutable?
- ¿Consideras la interfaz del juego fácil de entender y usar?

Estas preguntas pueden ser valoradas de 1 a 5, siendo 1 el valor más bajo y 5 el más alto. Gracias a las respuestas de 10 personas se ha obtenido el gráfico mostrado en la figura 8.1. Además de las 4 preguntas mostradas, se han realizado dos más, las cuales permiten conocer el contexto de los encuestados. Estas preguntas obtienen información sobre la dificultad percibida y la experiencia previa con videojuegos similares. Generalmente, los encuestados tienden a tener poca experiencia con videojuegos similares y consideran que es un videojuego de dificultad media.

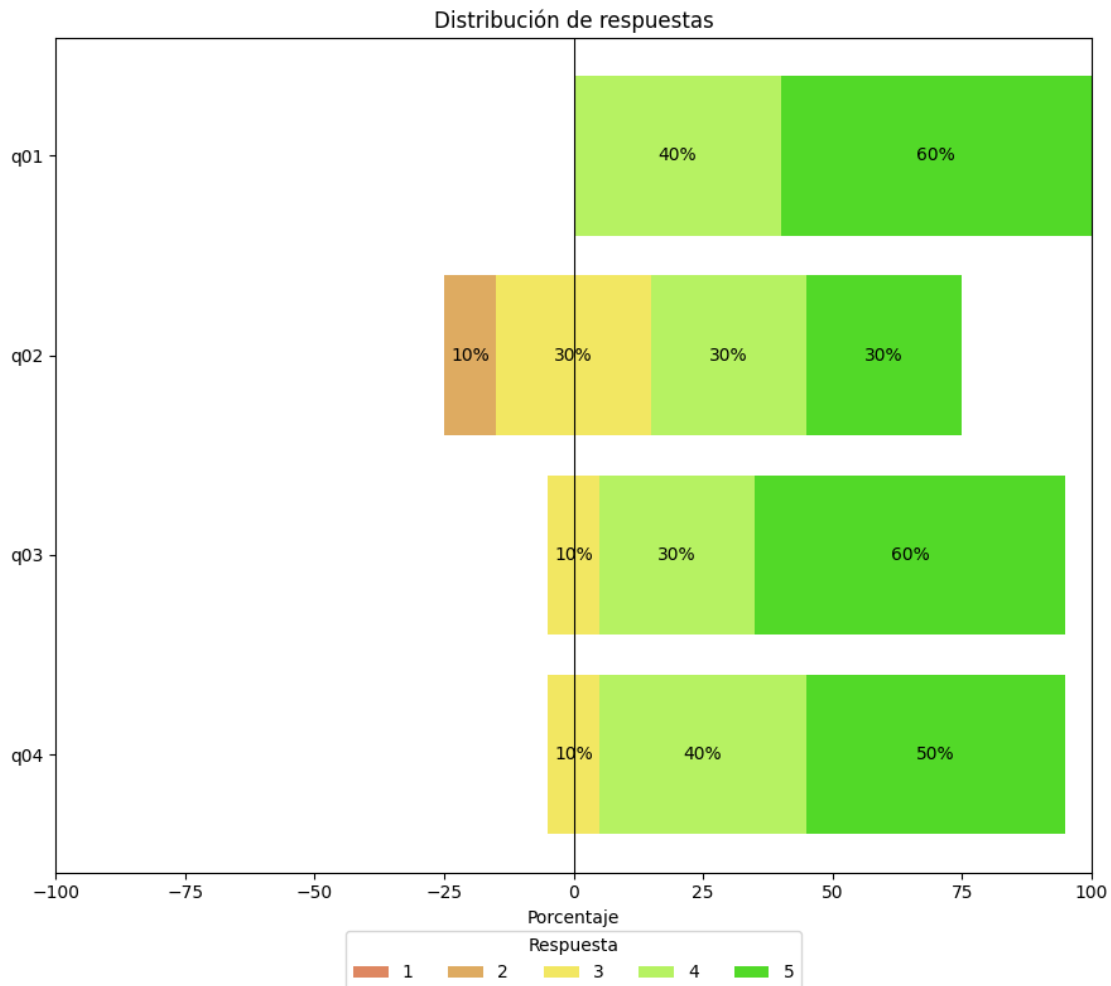


Figura 8.1: Gráfico de Likert sobre la encuesta final

Añadir variedad de elementos en el videojuego. Al igual que la fluidez del juego, la variedad de elementos es una métrica que depende de la opinión de cada individuo. A lo largo del proyecto, se añadieron 8 armas, 6 accesorios, 4 enemigos y 2 jefes. Aunque puede no parecer mucho, gracias a la base creada durante la implementación de estos, la creación de nuevos elementos no requiere de gran esfuerzo. Durante el desarrollo se decidió únicamente implementar esa cantidad de elementos para priorizar otros apartados. Es por esto que se marca como completado y se pueden añadir más elementos como trabajo a futuro.

Objetivos secundarios

Ajustes de accesibilidad, selector de idioma. Este objetivo consiste en hacer el juego más accesible para las personas, permitiendo modificar varios parámetros del juego. En la última iteración, se añadieron ajustes para modificar factores como el idioma o la velocidad de juego, dando por completado el objetivo.

Diferentes tipos de salas. A fin de aumentar la variedad de experiencias, en un inicio se planteó la existencia de diferentes tipos de salas. Estas podrían variar de salas con recompensas como armas, altares especiales o salas con desafíos especiales. Al final, se ha decidido no implementar esta funcionalidad, pero no se descarta para futuras iteraciones.

8.2. Planificación

Hasta comenzar el desarrollo de este TFG, los trabajos realizados han sido de una escala mucho menor a la planteada en los TFG. Durante el desarrollo de este proyecto se ha notado cómo ciertos contratiempos que previamente se veían como pequeños problemas pasan a ser grandes inconvenientes.

Uno de estos problemas es la planificación temporal y del trabajo. Donde antes se podían cometer errores de días, ahora pasan a ser semanas, retrasando el proyecto y forzando revisiones del planteamiento inicial. Un caso de esto se ha dado en la iteración 4, la cual inicialmente se dedicaría a la implementación de generación procedimental de salas y finalmente se ha descartado.

En un principio, se había estimado un coste total de 398 horas, siendo el coste real de 354 horas. En la figura 8.2 se puede ver un gráfico donde se compara el tiempo estimado de cada grupo de paquetes de trabajo en comparación al tiempo real.

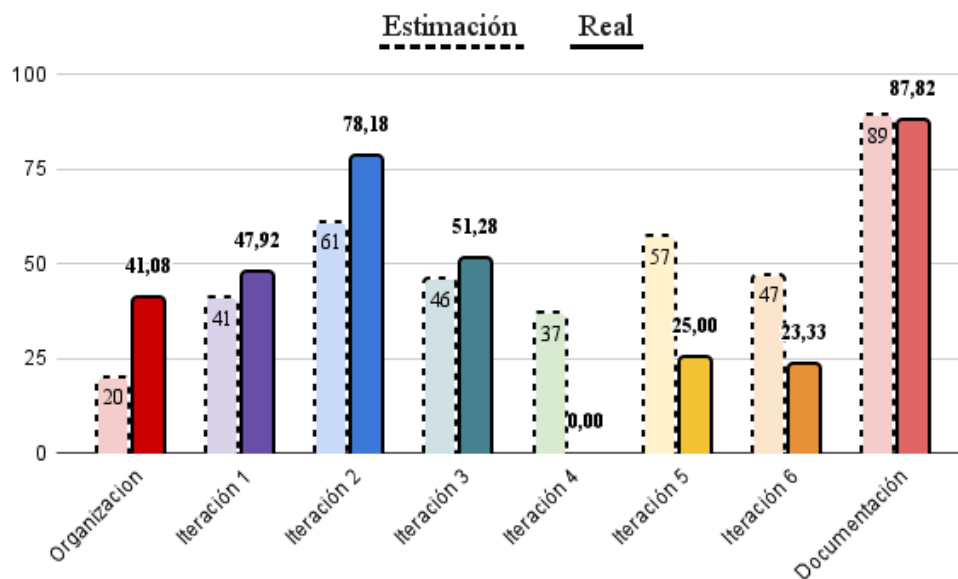


Figura 8.2: Estimación horaria contra realidad

En el gráfico se puede ver cómo el coste real de las tres primeras iteraciones no se aleja mucho de la estimación inicial, habiendo estimado un coste menor al real con un error promedio del 18 %. Dejando de lado el paquete de trabajo descartado, los dos últimos paquetes de trabajo del desarrollo tienen error de estimación del 114 %, habiendo sobrestimado en gran medida la cantidad de trabajo que conllevarían ambos paquetes. En el caso del paquete donde se crean los jefes, se cree que al poder reutilizar gran parte del desarrollo de los enemigos normales, se ha reducido mucho tiempo. En el caso de la última iteración se produjo una sobreestimación del tiempo necesario debido a la falta de experiencia previa en tareas como la gestión de los ajustes y el guardado. Su desarrollo fue más sencillo de lo esperado. Además, la experiencia ganada en cada iteración reduce el tiempo de trabajo de las siguientes.

En un futuro, a la hora de estimar la duración de proyectos, se tomarán en cuenta más factores además de la experiencia previa. A fin de mejorar la predicción, se investigará en mayor profundidad las tareas sobre las que menos conocimiento se tenga, para ser conscientes del trabajo que conllevan.

A lo largo del proyecto, se han realizado múltiples tareas que pueden ser agrupadas en 4 grupos: Documentación, organización, desarrollo y creación de elementos audiovisuales. En la figura 8.3 se puede ver cómo se han distribuido entre los cuatro grupos.

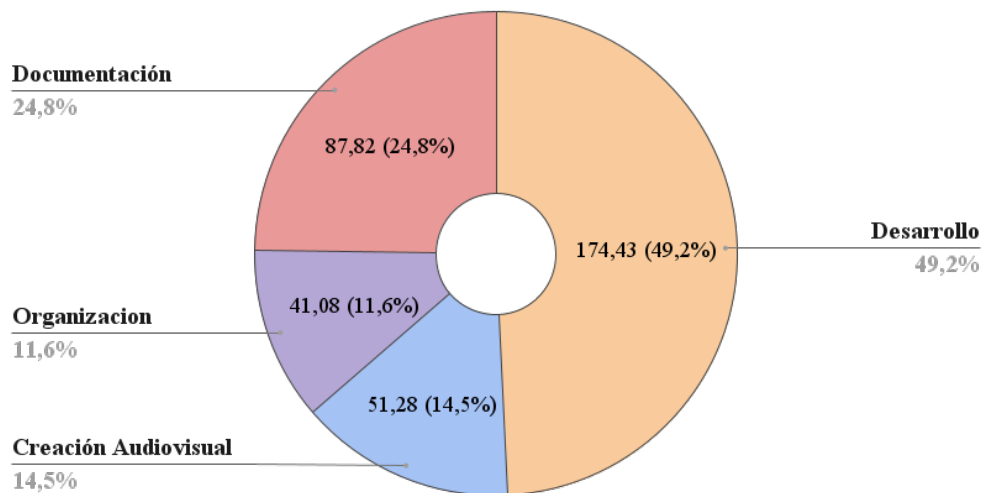


Figura 8.3: Distribución horaria

En el gráfico se puede ver cómo las tareas de documentación y organización, es decir, las tareas más administrativas, toman casi el 30 % de las horas del proyecto. Esa duración es indicativa de la importancia de estas tareas para el cierre satisfactorio de un proyecto.

8.3. Desafíos surgidos

En todos los desarrollos ocurren imprevistos, estos afectan al proyecto en diferentes formas y es por eso que se intentan prever. A pesar de esto, la prevención puede fallar o puede ocurrir algo que no fuese previsto. En este proyecto han ocurrido dos problemas que han afectado al desarrollo, el *overscoping* y la necesidad de refactorización.

El *overscoping* consiste en tratar de abarcar demasiado en cuanto a carga de trabajo se refiere y no priorizar las tareas más importantes del proyecto. A pesar del análisis inicial de cada funcionalidad, el plan de prevención no funcionó y se dio este riesgo. Como contingencia se llevó a cabo una reevaluación de la planificación donde un objetivo principal pasó a ver secundario.

La refactorización consiste en modificar internamente la estructura del código a fin de mejorarlo sin cambiar la funcionalidad de este. Este riesgo no se identificó en su momento, por lo que no se realizaron medidas de prevención. En este caso, la refactorización se realizó en la segunda iteración. En esta, a la hora de implementar las nuevas funcionalidades del paquete de trabajo, se llegó a la conclusión de que el código no era sostenible. Es decir, aunque funcional, la estructura utilizada dificultaría añadir nuevos elementos sobre esa base. Para evitar complicaciones en

próximas iteraciones, se rehízo el código a fin de utilizar composición, una técnica donde se crean pequeños módulos de funcionalidad, los cuales se combinan para crear funcionalidades complejas. Dado que este cambio se realizó al inicio de la segunda iteración, la cantidad de trabajo adicional a realizar fue de 9 horas.

8.4. Trabajo futuro

Aunque el proyecto haya concluido, hay muchas cosas que pueden añadirse al videojuego. Esto es cierto para muchos juegos en el mercado actual, los cuales reciben actualizaciones constantes a lo largo de los años. En el caso de este proyecto, las actualizaciones pueden ser categorizadas en tres apartados: Revisión, añadir contenido y añadir funcionalidades.

La primera categoría, revisión, hace referencia a modificar el contenido actual del juego para mejorarlo. Un ejemplo de esto es la mejora de los comportamientos de los enemigos. Aunque actualmente son funcionales, los enemigos podrían llegar a tener comportamientos más complejos, los cuales otorgarían una mejor experiencia y desafío.

A la hora de añadir contenido, el objetivo es utilizar los sistemas actuales para añadir variedad. Por ejemplo, añadir nuevas armas, accesorios o niveles. Las bases de estos ya están creadas, pero faltaría diseñar nuevos elementos para implementarlos. En el caso de los niveles, únicamente hace falta diseñar un nuevo escenario, pero las armas requieren animaciones y sonidos nuevos.

Por último, se ha categorizado la incorporación de nuevas funcionalidades. Este grupo es generalmente considerado como contenido, pero para este ejemplo se va a definir como la creación de funcionalidades extra, como se han añadido hasta ahora en cada iteración. En esta categoría se puede incluir la generación procedimental de salas.

A lo largo del desarrollo se han tenido múltiples ideas, las cuales han quedado en el tintero para evitar ahondar más en el problema de *overscoping*, el cual, como previamente se ha mencionado, se había detectado que ya se estaba produciendo.

En caso de seguir trabajando en el juego, añadiría más armas y enemigos para aumentar la variedad del juego. Hay muchas ideas que no he añadido, pero que tengo diseños y *sprites* ya hechos, por lo que me gustaría verlos en el juego. En la figura 8.4 se pueden ver múltiples *sprites* de armas, proyectiles y enemigos, los cuales no se pueden encontrar en el juego. Después implementaría la generación procedimental

de salas, ya que es una función con muchas posibilidades y me parece un tema de estudio interesante.



Figura 8.4: Diseños sin implementar

9. Bibliografía

- [1] UPV/EHU: *Reglamento de convivencia de la Universidad del País Vasco/Euskal Herriko Unibertsitatea*. <https://www.ehu.eus/es/web/gardentasunataria/universidad-del-paC3ADs-vasco/euskal-herriko-unibertsitateko-elkarbizitza-araudia>, 2025.
- [2] Unity: *Comparación de planes de Unity*. <https://unity.com/es/products/compare-plans>, 2024.
- [3] Servicio Público de Empleo Estatal: *Análisis de las características de la oferta de empleo de Desarrolladores de videojuegos*. <https://www.sepe.es/SiteSepe/contenidos/observatorio/perfiles/pdf/DesarrolladoresDeVideojuegos.pdf>, 2013.
- [4] Itchio: *Introducing open revenue sharing*. <https://itch.io/updates/introducing-open-revenue-sharing>, 2015.
- [5] Steam: *Steam Direct Product Submission Fee*. <https://store.steampowered.com/sub/163632>.
- [6] Epic Games: *Epic Games Store Launches Self-Publishing Tools for Game Developers and Publishers*. <https://store.epicgames.com/en-US/news/epic-games-store-launches-self-publishing-tools-for-game-developers-and-publishers>, 2023.
- [7] Unity: *Colliders*. <https://docs.unity3d.com/es/2018.4/Manual/CollidersOverview.html>, 2019.
- [8] Unity: *Order of execution for event functions*. <https://docs.unity3d.com/Manual/ExecutionOrder.html>, 2024.
- [9] Unity: *Ray*. <https://docs.unity3d.com/ScriptReference/Ray.html>, 2024.
- [10] johnsoncodehk y DreadBoy y AVChemodanov y DoctorShinobi y n4n0lix: *Rule Tile*. <https://docs.unity3d.com/Packages/com.unity.2d.tilemap.extras@4.0/manual/RuleTile.html>, 2023.
- [11] John French: *How to use script composition in Unity*. <https://gamedevbeginner.com/how-to-use-script-composition-in-unity>, 2022.
- [12] Unity Technologies: *Introduction to Object Pooling*. <https://learn.unity.com/tutorial/introduction-to-object-pooling>, 2019.
- [13] Brackeys: *2D Animation in Unity (Tutorial)*. <https://youtu.be/hkaysu1ZN8?si=dbveGPZHEGiextU0>, 2019.
- [14] Brackeys: *How to make a HEALTH BAR in Unity!* <https://www.youtube.com/watch?v=BLfNP4SciA>, 2020.

- [15] Unity: *Coroutines*. <https://docs.unity3d.com/es/2018.4/Manual/Coroutines.html>, 2019.
- [16] Brackeys: *PAUSE MENU in Unity*. <https://www.youtube.com/watch?v=JivuXdrIHK0>, 2025.
- [17] AyAMrau: *Calling function on button click via script*. <https://discussions.unity.com/t/4-6-ui-calling-function-on-button-click-via-script/114835/2>, 2014.
- [18] Unity: *Object.DontDestroyOnLoad*. <https://docs.unity3d.com/ScriptReference/Object.DontDestroyOnLoad.html>, 2025.
- [19] Ginxu: *Perdí 5 horas con este BUG por no leer el manual de Unity*. <https://youtu.be/OZpM4TBoMqw?si=qfrWMin-Lc-LcsEzt=335>, 2025.
- [20] Game Accessibility Guidelines: *Support windowed mode for compatibility with overlaid virtual keyboards*. <https://gameaccessibilityguidelines.com/if-producing-a-pc-game-support-windowed-mode-for-compatibility-with-overlaid-virtual-keyboards/>, 2016.
- [21] Game Accessibility Guidelines: *Allow controls to be remapped / reconfigured*. <https://gameaccessibilityguidelines.com/allow-controls-to-be-remapped-reconfigured/>, 2016.
- [22] Game Accessibility Guidelines: *Include an option to adjust the game speed*. <https://gameaccessibilityguidelines.com/include-an-option-to-adjust-the-game-speed/>, 2016.

10. Anexo

10.1. Diagramas de secuencia

Continuar partida

Utilizando los datos de la partida guardada en el dispositivo, se carga un nivel. El nivel está identificado por un nombre y se carga la escena/nivel indicada haciendo uso de la clase *SceneManager* de Unity. El diagrama de secuencia se puede ver en la figura 10.1.

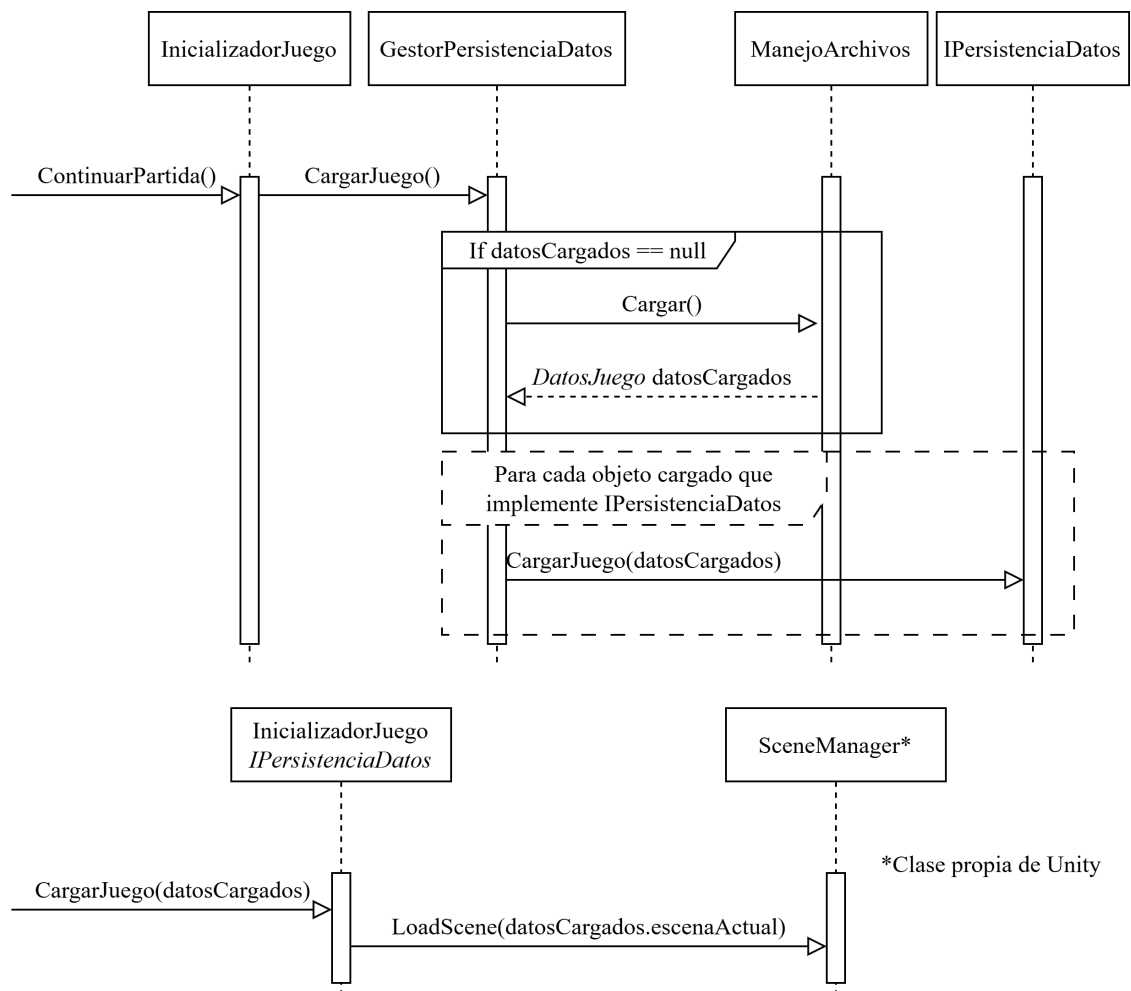


Figura 10.1: Diagrama de secuencia: Continuar partida

Iniciar nueva partida

Tal y como se muestra en la figura 10.2, al iniciar una nueva partida, se recoge el personaje seleccionado en la interfaz y se crea un nuevo paquete de datos para una partida nueva. Estos nuevos datos mantienen los datos permanentes como los ajustes. Los nuevos datos se guardan en un archivo JSON.

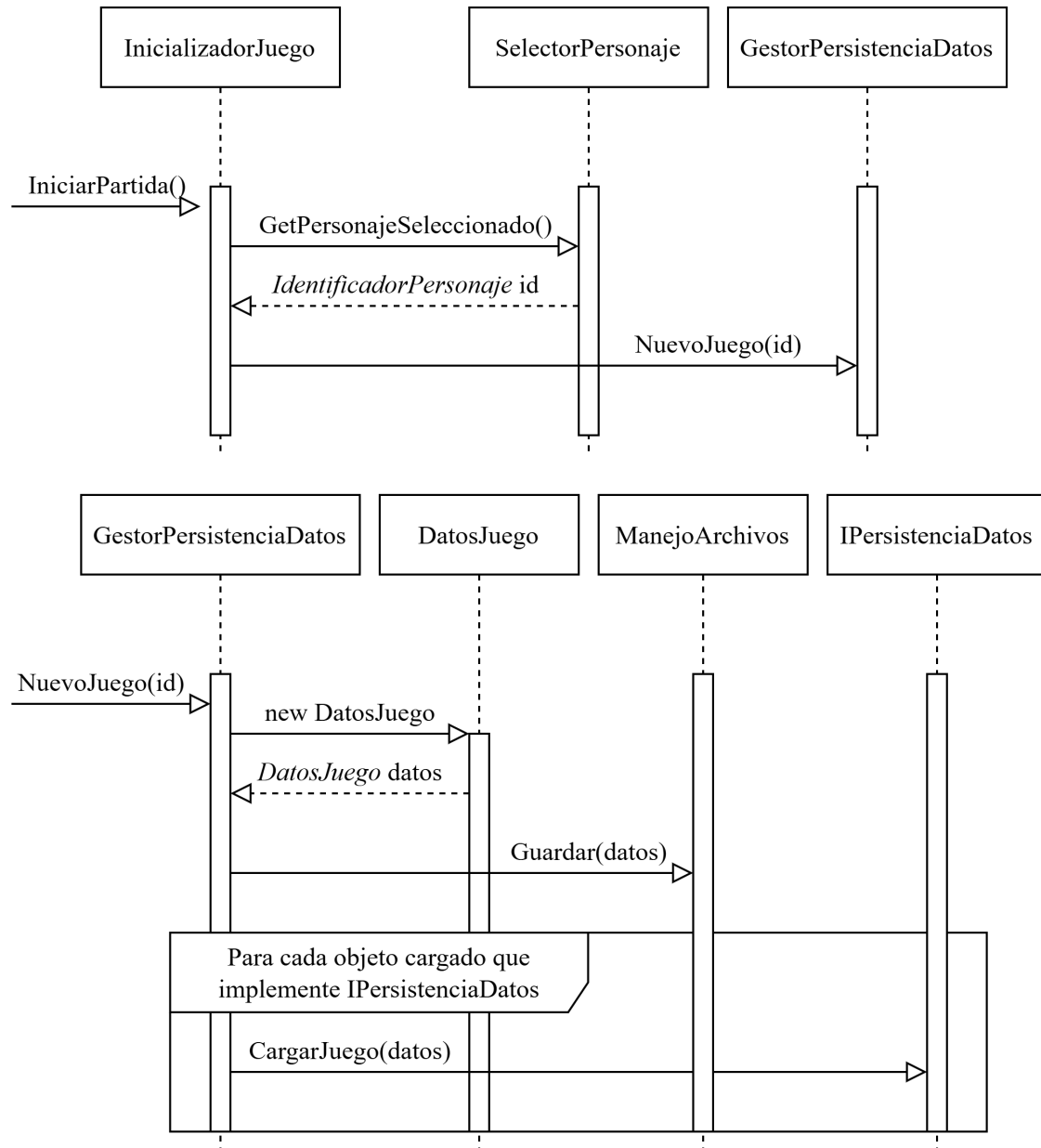


Figura 10.2: Diagrama de secuencia: Iniciar nueva partida

Ajustes

Desde la interfaz de ajustes se puede ver y modificar el idioma, velocidad y si el juego está en pantalla completa. GestorIdioma es una clase la cual guarda todas las traducciones y cuando un objeto necesita hacer uso de un texto, pide la traducción a esta clase. En la figura 10.3 se puede ver el diagrama de secuencia indicado.

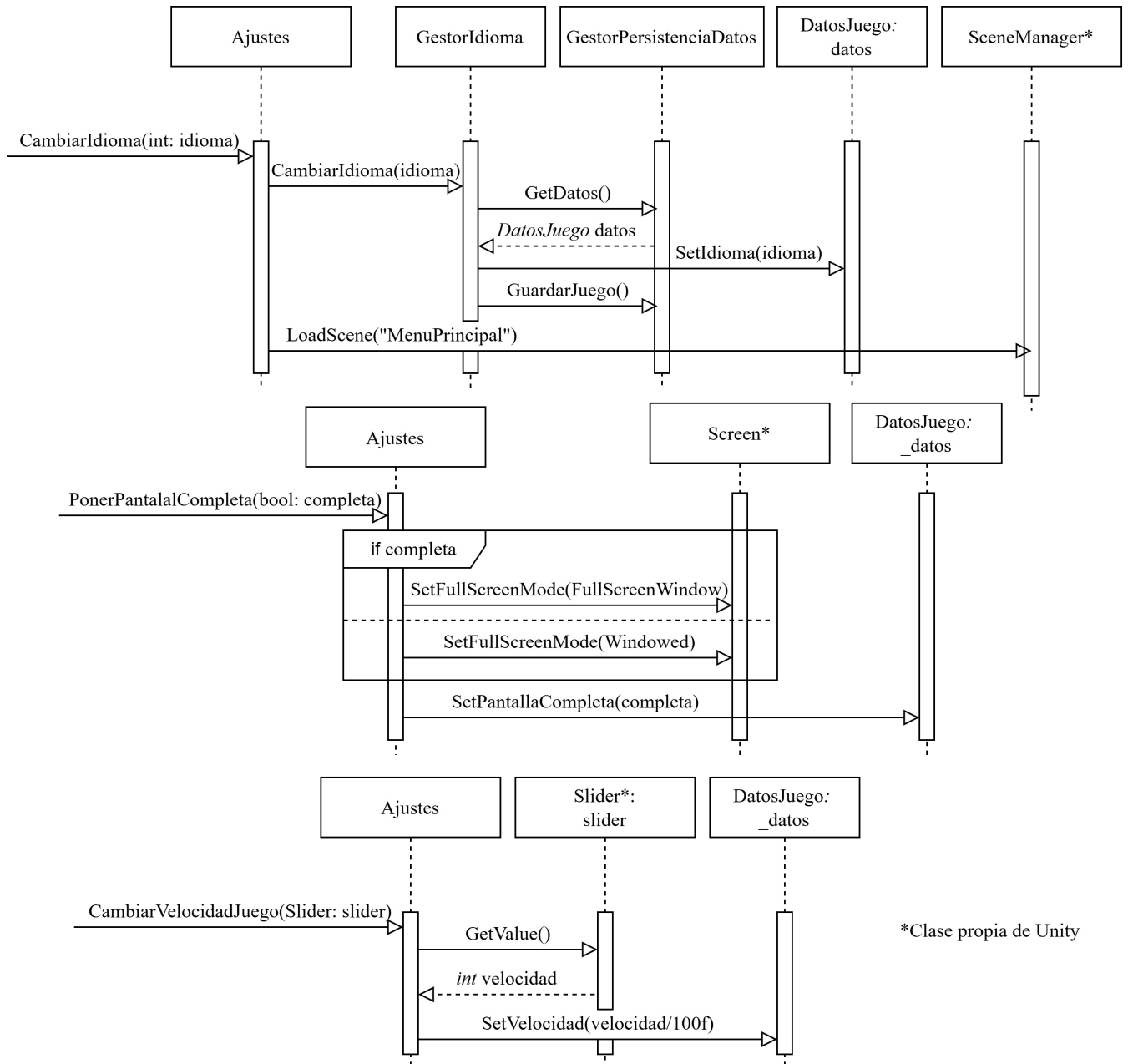


Figura 10.3: Diagrama de secuencia: Ajustes

Cargar nivel

Carga una nueva escena de Unity. En la figura 10.4 se pueden ver las diferentes llamadas que realiza Unity al cargar una escena. *Awake* y *Start* se dan cuando un objeto se está iniciando y *Update* es llamado en cada fotograma.

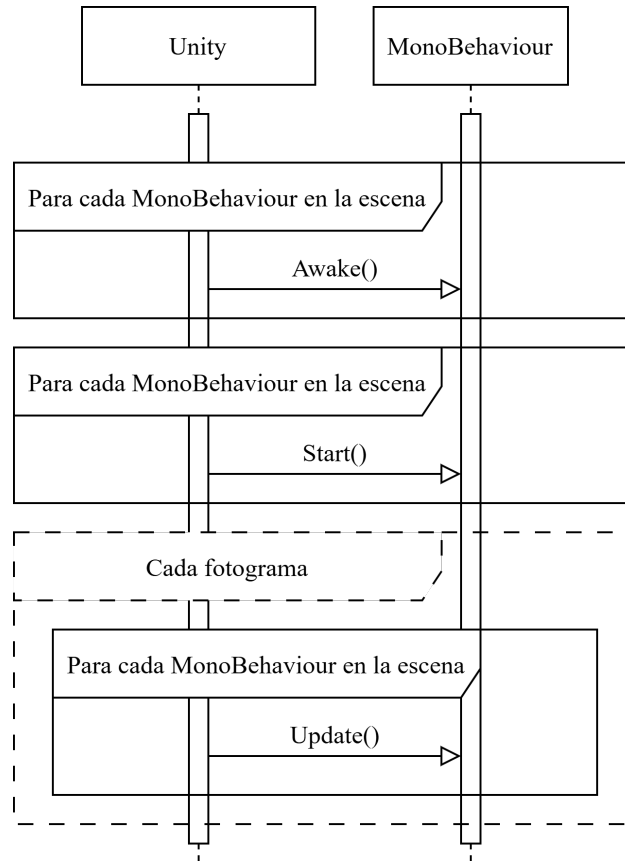


Figura 10.4: Diagrama de secuencia: Cargar nivel

Inicializar Personaje

A la hora de inicializar el personaje, el primer paso es obtener las referencias a los diferentes componentes que se van a utilizar. Esto se hace en el método *Awake*. En el método *Start* se recogen los datos de la partida para inicializar los diferentes componentes, como puede ser la vida o las armas equipadas. Esto puede ser visto en la figura 10.5. Una vez inicializado, en cada fotograma se gestionan las animaciones y sonidos, entre otras acciones que se mostrarán en próximos diagramas.

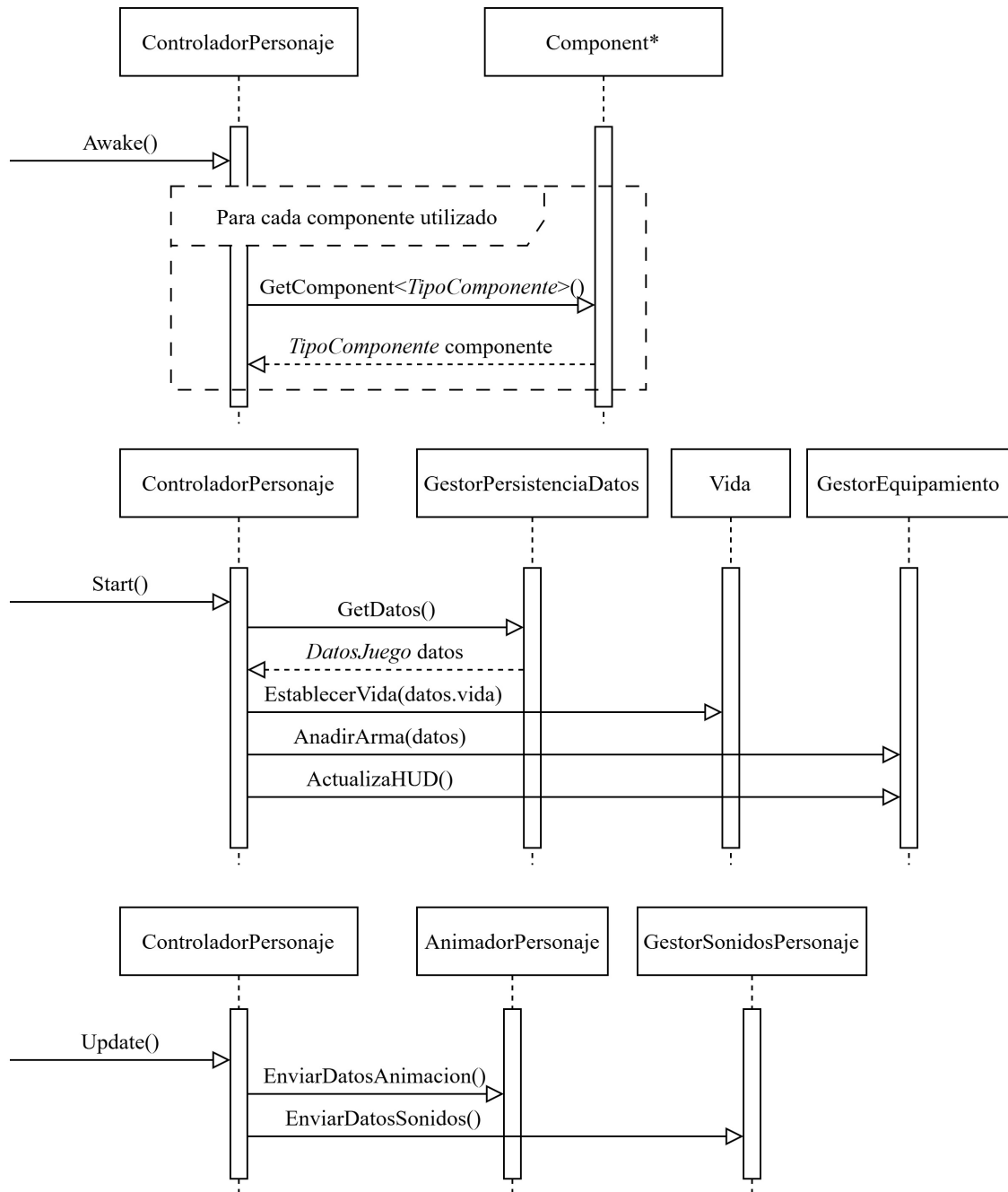


Figura 10.5: Diagrama de secuencia: Inicializar personaje

Inicializar UI

A la hora de comenzar un nivel, se establecen los valores de los diferentes elementos en pantalla para que puedan ser visualizados. En la figura 10.6 se muestra el proceso llevado a cabo para colocar las imágenes de las armas y sus accesorios en pantalla.

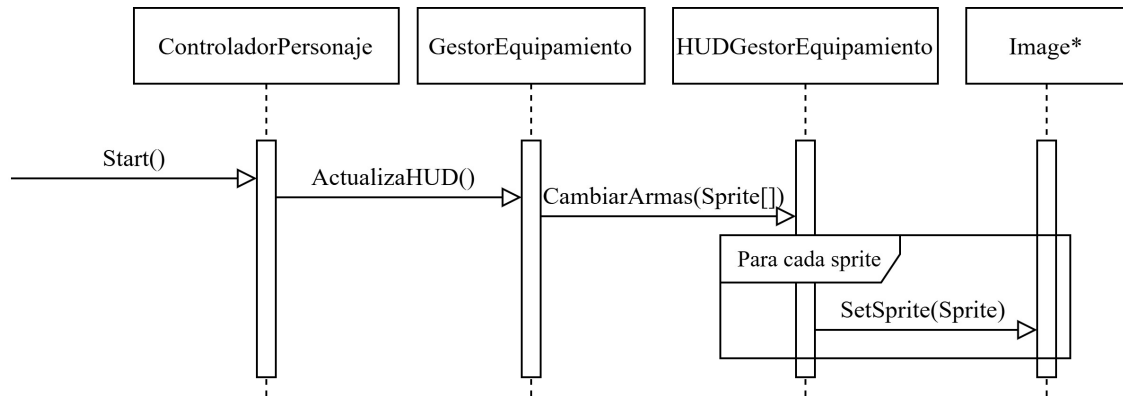


Figura 10.6: Diagrama de secuencia: Inicializar interfaz

Pausar

Para pausar el juego se hace uso de la clase *Time* de Unity, la cual gestiona el paso del tiempo en el juego. A la hora de pausar el juego, se ralentiza al mínimo y al reanudar, se establece a la velocidad indicada en los ajustes. En la figura 10.7 se puede observar el diagrama de secuencia.

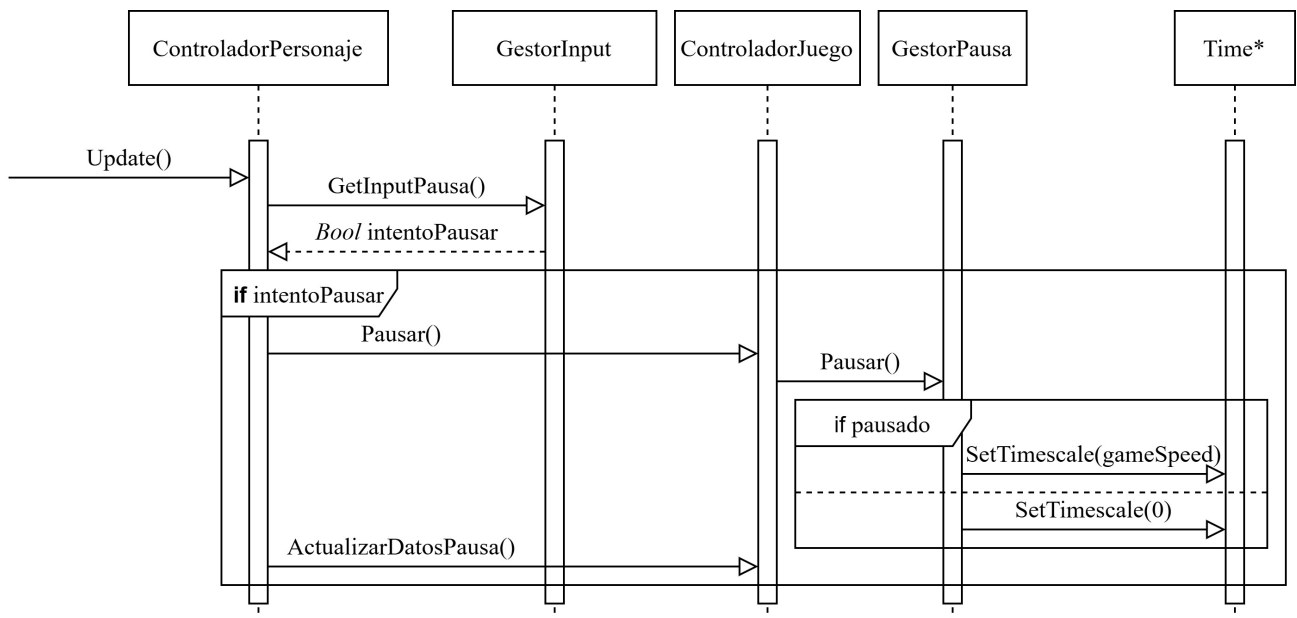


Figura 10.7: Diagrama de secuencia: Pausar

Gestionar Equipamiento

La gestión del equipamiento conlleva realizar varias acciones. Estas pueden ser atacar, cambiar de arma y recoger o desechar un arma. En el segundo diagrama de la figura 10.8 se puede ver como un arma instancia un proyectil para poder dispararlo.

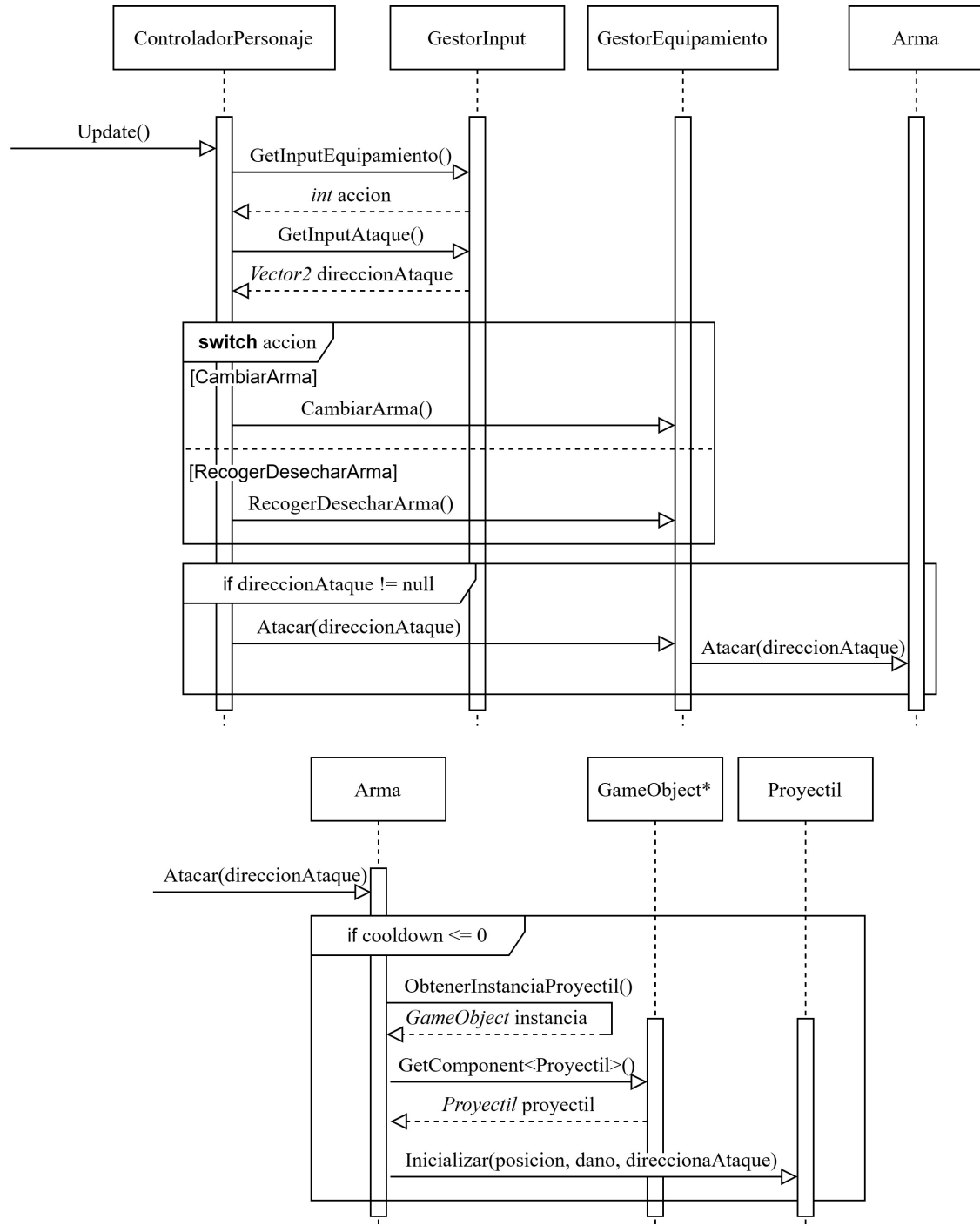


Figura 10.8: Diagrama de secuencia: Gestionar equipamiento

Inicializar Enemigo

Cuando se carga un nivel, cada uno de los enemigos inicia su comportamiento. Dependiendo del enemigo, los pasos que seguirán son diferentes aunque similares. En la figura 10.9 se muestra el caso de la cucaracha. Los enemigos comienzan en estado inactivo. Una vez inicializado, en cada fotograma, dependiendo del estado, el comportamiento es diferente.

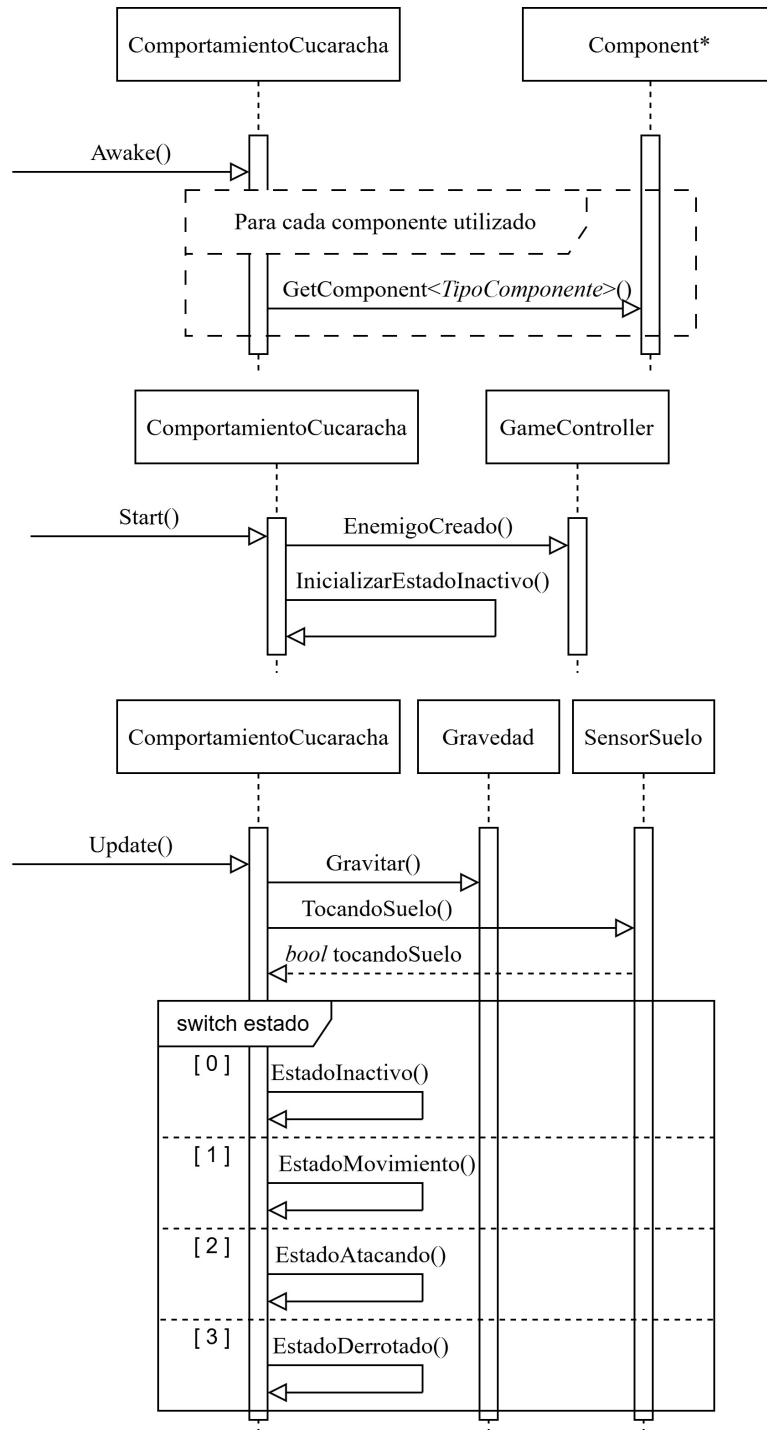


Figura 10.9: Diagrama de secuencia: Inicializar enemigo

Derrotar Enemigo

Cuando la vida de un enemigo llega a cero, se lanza el método `Derrotado()`, el cual se encarga de generar el botín, iniciar la animación y sonidos de derrota y de destruir el objeto del enemigo. En la figura 10.10 se puede cómo en el segundo diagrama se genera una instancia de botín y el arma que esta contendrá.

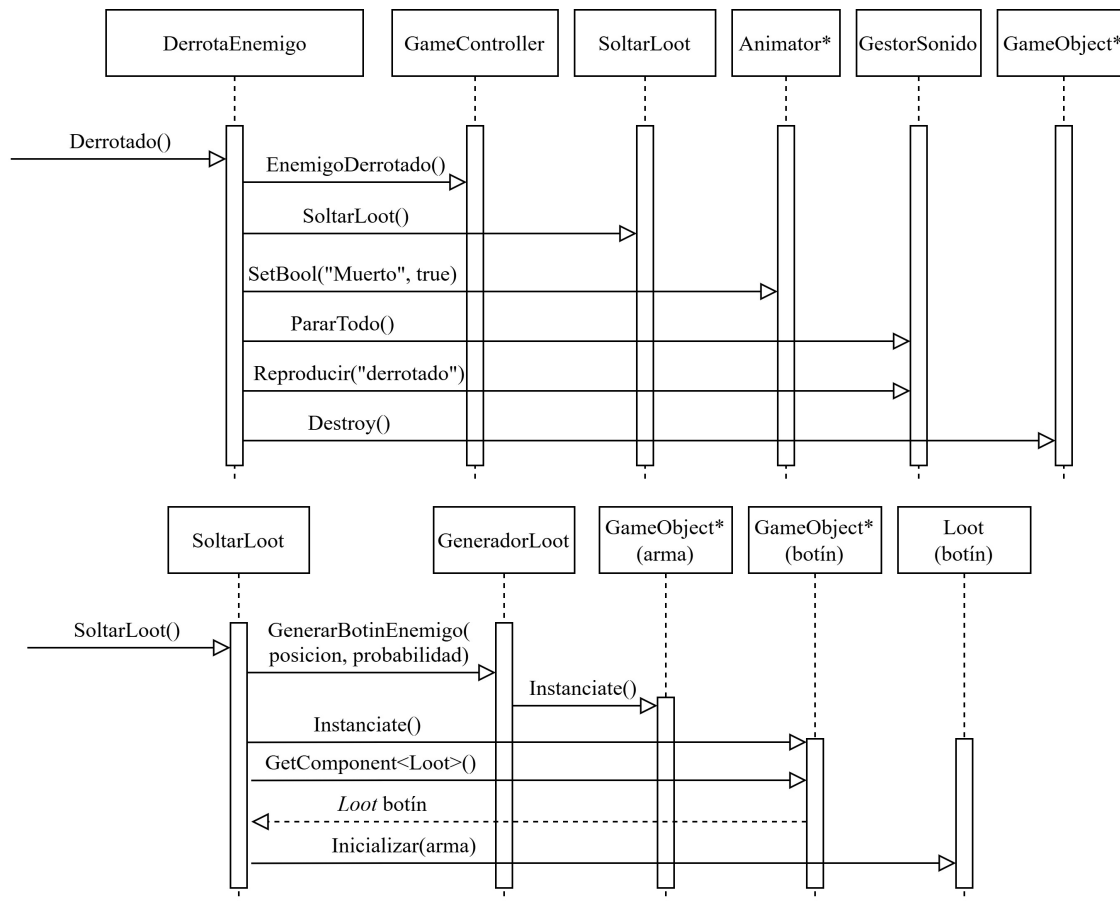


Figura 10.10: Diagrama de secuencia: Derrotar enemigo

Moverse

Moverse consiste en caminar y saltar. En la figura 10.11 se pueden ver tres diagramas. En el primero se gestiona el *input* del jugador el cual es redirigido al componente correspondiente. Además, se aplican las físicas como la gravedad o fricción. A la hora de caminar, se modifica la velocidad del componente *RigidBody2D* basándose en un valor de aceleración. Al saltar, la velocidad es modificada verticalmente, dando un impulso inicial.

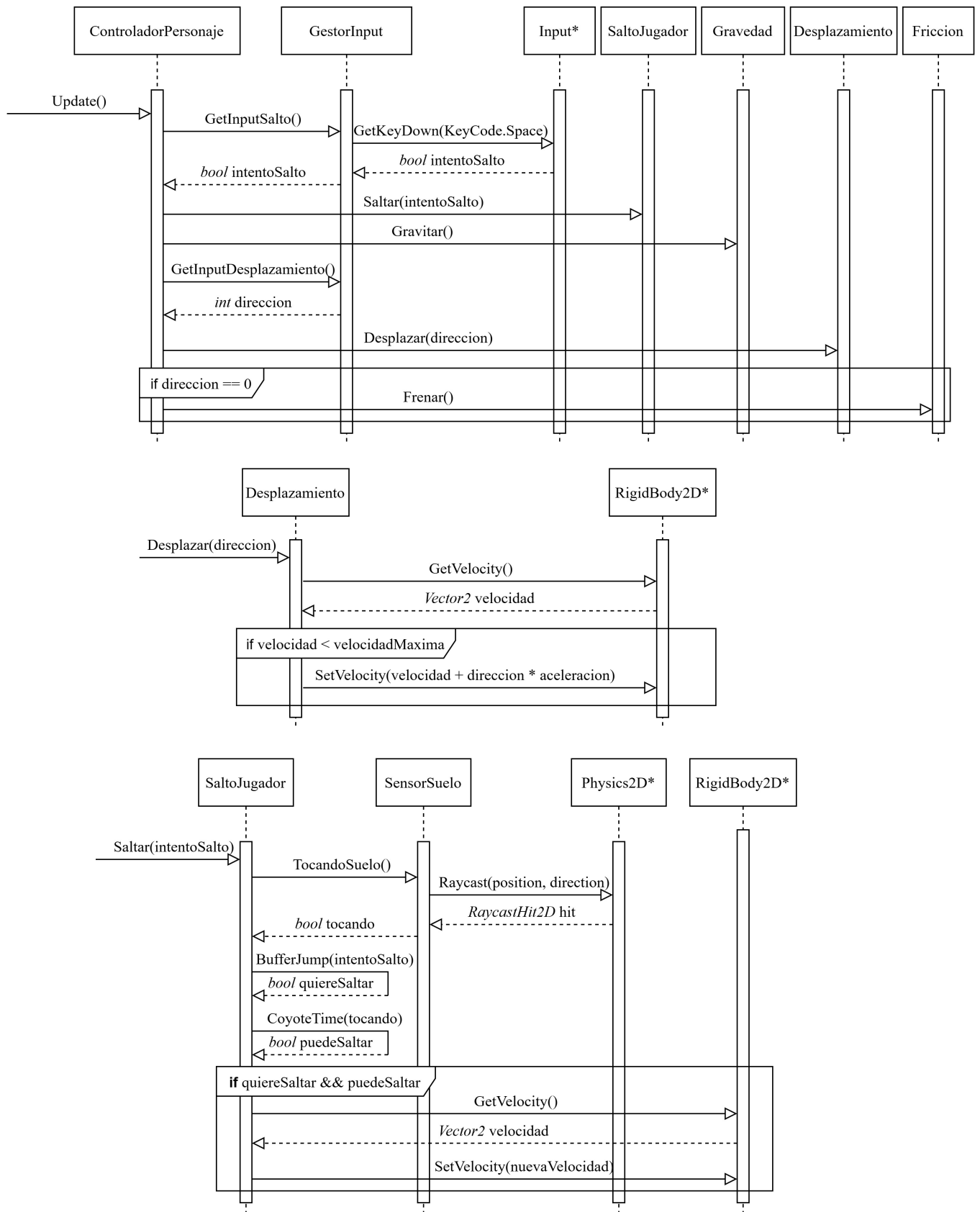


Figura 10.11: Diagrama de secuencia: Moverse

10.2. Diagrama de clases

En la figura 10.12 se muestra el diagrama de clases más detallado.

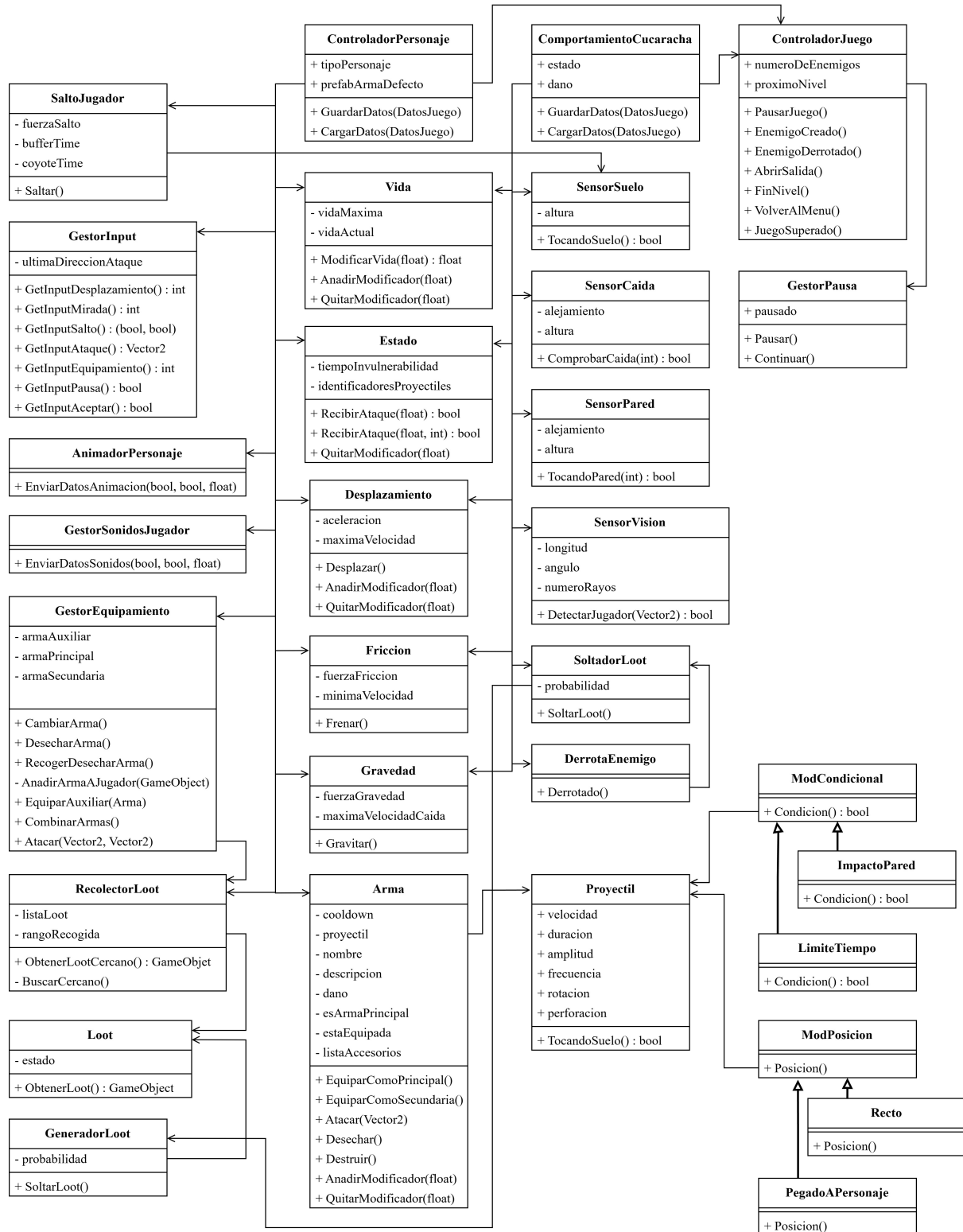


Figura 10.12: Diagrama de clases